

파이썬 프로그래밍

(시각화)

· 아나콘다 환경 설정 및 주피터 노트북 사용법

아나콘다란 수학, 과학관련 패키지들, 머신러닝이나 데이터 분석 등에 사용하는 여러 가지 패키지가 기본적으로 포함된 파이썬 배포판이다. 또한 아나콘다는 파이썬 가상 환경을 구축하는데도 유용하게 사용할 수 있다. 내부적으로 conda라는 환경/패키지 관리자가 존재하며 이 conda를 통해 패키지를 설치하거나 가상 환경을 관리할 수 있다.

· 아나콘다 설치

<https://www.anaconda.com/>

· 주피터 노트북 정의

웹브라우저(크롬) 환경에서 파이썬 코드를 작성하고 단계적으로 실행 가능하도록 하는 개발자 도구이다.

· 주피터 노트북 실행 및 폴더 생성

[Anaconda]-[Jupyter Notebook] 프로그램 실행한 후 [PythonDataWorkspace]-[Pandas] 폴더를 생성한다.

· 주피터 노트북 사용법

- 1) 초록색 : 편집모드
- 2) 파랑색 : 명령모드 (편집모드 상태에서 ESC)
- 3) 각 Cell의 Line Numbers : [View]-[Toggle Line Numbers]

▪ '01.주피터 노트북 사용법.ipynb' 파일을 생성한다.

```
print('Hello World')
print('환영합니다.')
name = '연탄이'
print(name)
print('Ctrl + Enter : 현재 Cell 실행')
print('Shift + Enter : 현재 Cell 실행 후 다음 Cell 선택')
print('Alt + Enter : 현재 Cell 실행 후 다음 위치에 Cell 삽입')
```

```
Hello World
환영합니다.
연탄이
Ctrl + Enter : 현재 Cell 실행
Shift + Enter : 현재 Cell 실행 후 다음 Cell 선택
Alt + Enter : 현재 Cell 실행 후 다음 위치에 Cell 삽입
```

▪ 클래스 + 함수 + '.' 입력한 후 'Tab' 키를 누르면 사용 가능한 명령어 목록이 나온다.

```
import random
print(random.randint(1, 45))
```

▪ 함수 + '?' 입력한 후 'Shit' + 'Enter' 키를 누르면 설명이 출력된다.

random.randint?

Signature: random.randint(a, b)
Docstring:
Return random integer in range [a, b], including both end points.
File: /opt/conda/envs/anaconda-panel-2023.05-py310/lib/python3.11/random.py
Type: method

- 함수 + '??' 입력한 후 'Shift' + 'Enter' 키를 누르면 source code와 설명이 출력된다.

```
random.randint??
```

Signature: random.randint(a, b)

Docstring:

Return random integer in range [a, b], including both end points.

Source:

```
def randint(self, a, b):
    """Return random integer in range [a, b], including both end points.
    """
    return self.randrange(a, b+1)
```

File: /opt/conda/envs/anaconda-panel-2023.05-py310/lib/python3.11/random.py

Type: method

- 문장 앞에 '#'을 입력하거나 'Ctrl' + '/' 키를 누르면 주석처리를 할 수 있다.

```
# random.randint??
```

- 실행중인 경우 Number 대신 '*' 표시되고 [중지] 아이콘으로 실행중지가 가능하다.

```
import time
for i in range(10):
    print(i)
    time.sleep(1)
```

• 마크다운 사용법

마크다운이란 간단한 서식을 이용하여 일반 텍스트 문서의 양식을 편집하는 문법으로 HTML의 형태로 변환이 가능하다.

- '1라인 이동' + '명령모드' + 'A' 키를 입력하여 한 행을 추가한 후 'M' 키를 입력하여 Mark Down을 변경한다.

Jupyter Notebook

주피터 노트북은 웹 브라우저 상에서 개발할 수 있는 도구이며, 코드를 Cell 단위로 묶어서 실행하고 그래프나 표, 그리고 이미지나 영상 등을 쉽게 볼 수 있어서 특히 데이터 관련 작업을 할 때 많이 활용됩니다.

> 교육을 위한 강의 노트로도 아주 훌륭해요!

Jupyter Notebook

주피터 노트북은 웹 브라우저 상에서 개발할 수 있는 도구이며, 코드를 Cell 단위로 묶어서 실행하고 그래프나 표, 그리고 이미지나 영상 등을 쉽게 볼 수 있어서 특히 데이터 관련 작업을 할 때 많이 활용됩니다.

교육을 위한 강의 노트로도 아주 훌륭해요!

- 마크다운으로 이미지 입력하기 1 : [구글]-[Jupyter Notebook] 홈페이지를 화면 캡처하여 붙여넣기 한다.

```
![image.png](attachment:image.png)
```

- 마크다운으로 이미지 입력하기2 : [Edit]-[Insert Image] 메뉴에서 입력할 이미지를 선택한다.

```
![a.png](attachment:a.png)
```

- '#'의 개수에 따라 글자 크기가 결정되고 목록 작성을 작성하려면 숫자 '1' 키를 입력한 후 문장을 작성한다.

특징

1. 코드를 Cell 단위로 작성 및 실행
1. 마크다운을 통한 문서화
1. 그래프나 표 등을 실시간으로 확인
1. html, pdf 등 파일로 저장

특징

1. 코드를 Cell 단위로 작성 및 실행
2. 마크다운을 통한 문서화
3. 그래프나 표 등을 실시간으로 확인
4. html, pdf 등 파일로 저장

- '---' 키를 누르면 수평선이 출력되며 '-' 키를 입력하면 목록을 작성 한다.

시각화 예제 1: 그래프

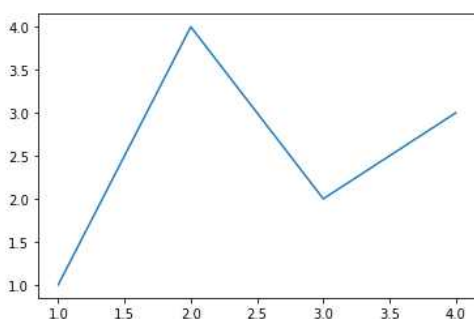
- matplotlib를 이용하면 다양한 그래프를 그릴 수 있으며 수행 결과를 바로 확인 할 수 있다.

시각화 예제 1: 그래프

- matplotlib를 이용하면 다양한 그래프를 그릴 수 있으며 수행 결과를 바로 확인 할 수 있다.

- [구글]-[matplotlib 검색]-[Tutorials]-[Basic Usage] 기본 그래프 코드를 복사하여 그래프를 출력해 본다.

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
fig, ax = plt.subplots() # Create a figure containing a single axes.
ax.plot([1, 2, 3, 4], [1, 4, 2, 3]); # Plot some data on the axes.
```



- [구글]-[pandas 검색]-[User Guide]-[Table Visualization] 테이블 작성 코드를 복사하여 테이블을 출력해 본다.

```
import pandas as pd
import numpy as np
import matplotlib as mpl

df = pd.DataFrame([[38.0, 2.0, 18.0, 22.0, 21, np.nan],[19, 439, 6, 452, 226,232]],
                  index=pd.Index(['Tumour (Positive)', 'Non-Tumour (Negative)'], name='Actual Label:'),
                  columns=pd.MultiIndex.from_product(
                      [['Decision Tree', 'Regression', 'Random'], ['Tumour', 'Non-Tumour']], names=['Model:', 'Predicted:']))
df.style
```

Actual Label:	Model:		Decision Tree		Regression		Random	
	Predicted:		Tumour	Non-Tumour	Tumour	Non-Tumour	Tumour	Non-Tumour
Tumour (Positive)			38.000000	2.000000	18.000000	22.000000	21	nan
Non-Tumour (Negative)			19.000000	439.000000	6.000000	452.000000	226	232.000000

- 하이퍼 링크 텍스트를 이용하려면 '[링크단어]' + '(링크주소)'를 입력한다.

사전 학습 자료
 다음 링크에서 파이썬 기본편에 대한 학습을 하실 수 있습니다. space 두 칸 입력
 - [나도 코딩 블로그](https://nadocoding.tistory.com)

사전 학습 자료

다음 링크에서 파이썬 기본편에 대한 학습을 하실 수 있습니다.
[나도 코딩 블로그](https://nadocoding.tistory.com)

- [유튜브]-[나도코딩 python 기본편 검색]-[오른쪽 마우스]-[소스코드복사] 메뉴를 선택하여 복사하여 'Code' 상태로 변경(Markdown인 경우 명령모드에서 후 'Y')한 후 상단에 '%%HTML' 입력 후 실행해 본다.

```
%%HTML
<iframe width="640" height="360"
src="https://www.youtube.com/embed/kWiCuklohdY"
title="YouTube video player" frameborder="0"
allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture" allowfullscreen>
</iframe>
```

- Line Number 재 정렬 : [Kernel]-[Restart & Run All]-[Restart and Run All Cells] 버튼 선택
- HTML 저장 : [File]-[Download as] - [HTML(*.htm)] 다운로드 폴더에 저장된다.
- Jupyter 파일 저장 : [c:]-[사용자]-[사용자 계정]에 확장자 IPYNB(Interactive PYthon NoteBook)로 저장된다.
- Jupyter 파일 열기 : [File]-[Open] 메뉴에서 파일을 선택한다.
- Jupyter 종료 : [Ctrl + S]로 저장-[File]-[Close and Halt]-[명령창]-[Ctrl + C, C]

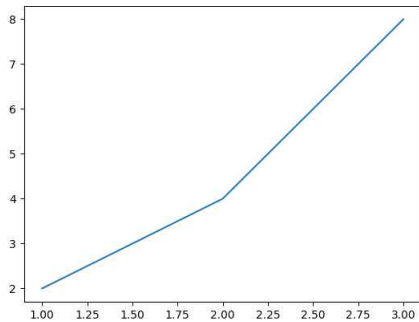
• Matplotlib

맷플롯립(Matplotlib)은 다양한 형태의 그래프를 통해서 데이터 시각화를 할 수 있는 라이브러리이다.

01. 그래프 기본

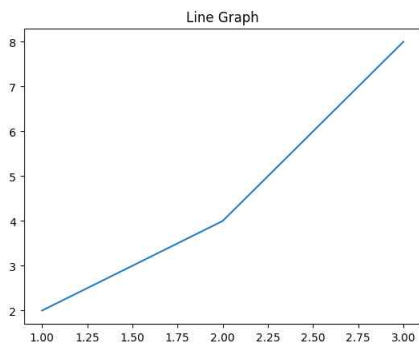
- 새로운 파일 '01.그래프기본.ipynb'를 생성한다.

```
import matplotlib.pyplot as plt
x = [1, 2, 3]
y = [2, 4, 8]
plt.plot(x, y)
```



- 직선 그래프에 제목(title)을 설정한다.

```
plt.plot(x, y)
plt.title('Line Graph')
plt.show()
```



- 코랩을 사용할 경우 한글 폰트를 설치한 후 [런타임]-[세션 다시 시작] 메뉴를 클릭하여 다시 시작한다.

```
!sudo apt-get install -y fonts-nanum
!sudo fc-cache -fv
!rm ~/.cache/matplotlib -rf
```

- matplotlib 그래프에서 한글 폰트를 사용하기 위한 설정을 한다. (방법1)

```
import matplotlib.pyplot as plt
plt.rc('font', family='NanumBarunGothic')
plt.rc('font', size= 20)
plt.rc('axes', unicode_minus=False)
```

- matplotlib 그래프에서 한글 폰트를 사용하기 위한 설정을 한다. (방법2)

```
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic' #맑은 고딕
matplotlib.rcParams['font.size'] = 15 #글자 크기
matplotlib.rcParams['axes.unicode_minus'] = False #한글 폰트 사용 시 마이너스 글자가 깨지는 현상을 해결
```

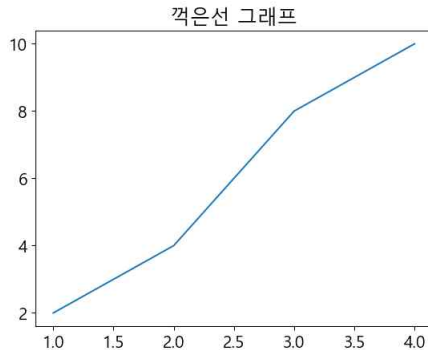
- 사용 가능한 한글 폰트 이름을 출력한다.

```
import matplotlib.font_manager as fm
fm.fontManager.ttflist
[f.name for f in fm.fontManager.ttflist]
```

```
['STIXGeneral',
 'STIXSizeThreeSym',
 'cmb10',
 'DejaVu Sans',
 'STIXSizeThreeSym',
 ...
 'HYbdaM',
 'Bookman Old Style']
```

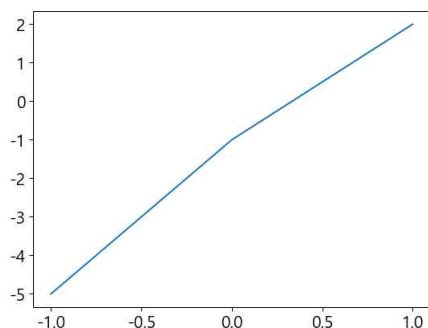
```
x = [1, 2, 3, 4]
y = [2, 4, 8, 10]
```

```
plt.plot(x, y)
plt.title('꺾은선 그래프')
plt.show()
```



- X, Y 값으로 리스트 타입의 데이터를 직접 입력할 수 있다.

```
plt.plot([-1, 0, 1], [-5, -1, 2])
plt.show()
```



02. 축(Axis)

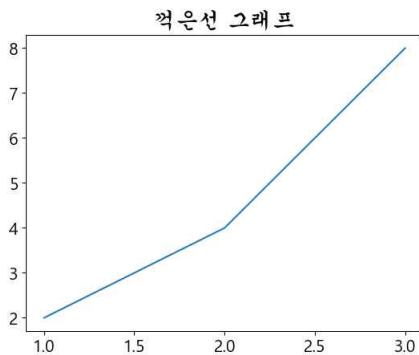
- 새로운 파일 '02.축.ipynb'를 생성한다.

```
import matplotlib.pyplot as plt
import matplotlib
```

```
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
x = [1, 2, 3]
y = [2, 4, 8]
```

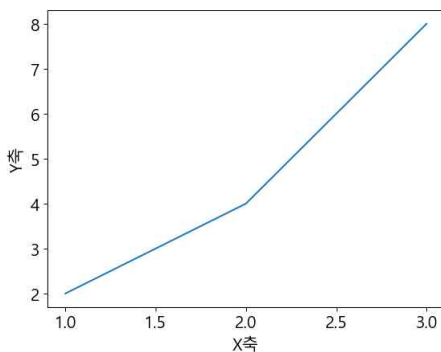
```
plt.plot(x, y)
plt.title('적은선 그래프', fontdict={'family':'HYGungSo-Bold', 'size':20})
plt.show
```



- X축, Y축 레이블을 지정한다.

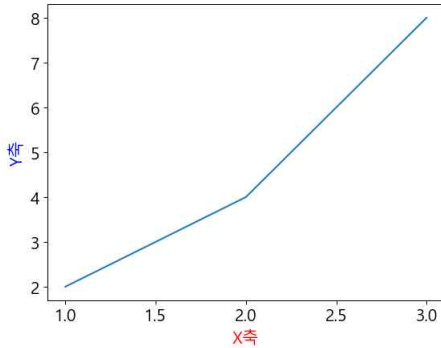
```
x = [1, 2, 3]
y = [2, 4, 8]
```

```
plt.plot(x, y)
plt.xlabel('X축')
plt.ylabel('Y축')
plt.show()
```



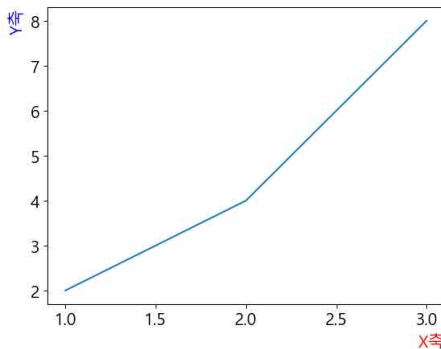
- X축, Y축 레이블 글자 색상을 지정한다.

```
plt.plot(x, y)
plt.xlabel('X축', color='red')
plt.ylabel('Y축', color='#0000FF')
plt.show()
```



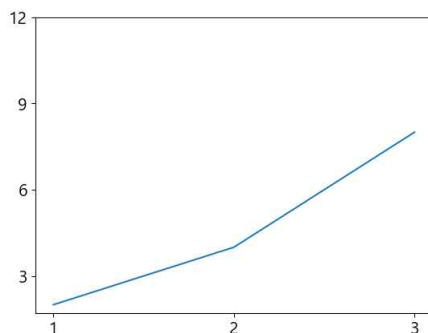
- X축 레이블의 위치를 오른쪽으로, Y축 레이블의 위치를 위쪽으로 지정한다.

```
plt.plot(x, y)
plt.xlabel('X축', color='red', loc='right')
plt.ylabel('Y축', color='#0000FF', loc='top')
plt.show()
```



- X축, Y축 간격지정을 위한 눈금들을 지정한다.

```
plt.plot(x, y)
plt.xticks([1, 2, 3])
plt.yticks([3, 6, 9, 12])
plt.show()
```



03. 범례(Legend)

- 새로운 파일 '03.범례.ipynb'를 생성한다.

```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

- year 변수에 년도 데이터를, height 변수에 list 타입으로 저장한다.

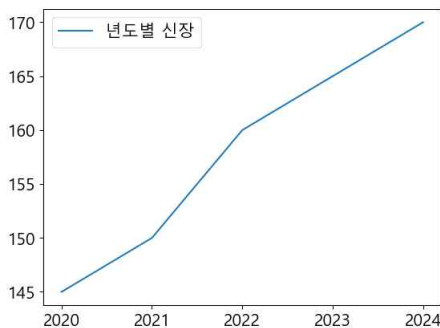
```
year = [2020, 2021, 2022, 2023, 2024]
height = [145, 150, 160, 165]
```

```
type(year), type(height)
```

```
(list, list)
```

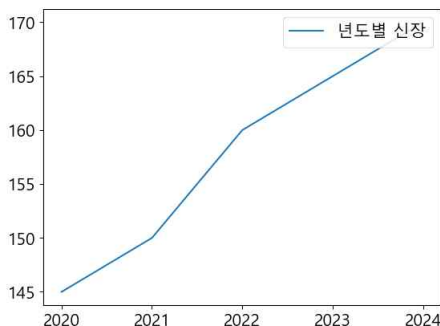
- 범례를 추가할 때는 legend() 함수를 사용하고 plot에서 지정한 label이 범례로 사용된다.

```
plt.plot(year, height, label='년도별 신장')
plt.legend()
plt.show()
```



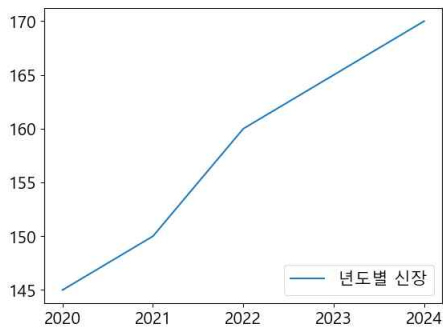
- 범례의 위치를 위쪽, 오른쪽에 출력한다.

```
plt.plot(year, height, label='년도별 신장')
plt.legend(loc='upper right')
plt.show()
```



- 범례의 위치를 아래쪽, 오른쪽으로 출력한다.

```
plt.plot(year, height, label='년도별 신장')
plt.legend(loc='lower right')
plt.show()
```

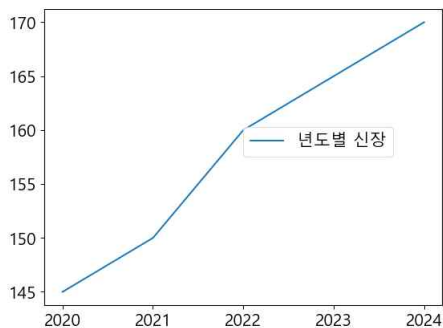


- 범례의 위치를 loc 속성을 사용하여 X축, Y축 중앙에 출력한다. (loc는 0~1 사용가능)

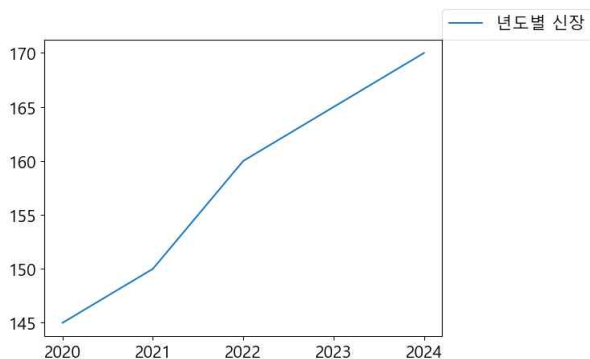
```
loc = (0.5, 0.5)
type(loc)
```

```
tuple
```

```
plt.plot(year, height, label='년도별 신장')
plt.legend(loc=loc)
plt.show()
```



```
plt.plot(year, height, label='년도별 신장')
plt.legend(loc=(1, 1))
plt.show()
```



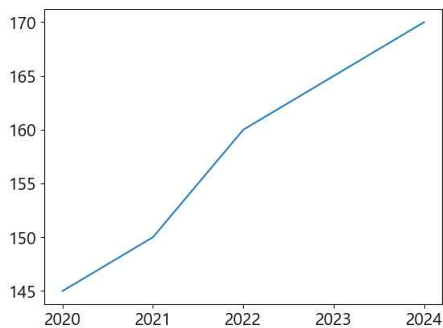
04. 스타일(Style)

- 새로운 파일 '04.스타일.ipnb'를 생성한다.

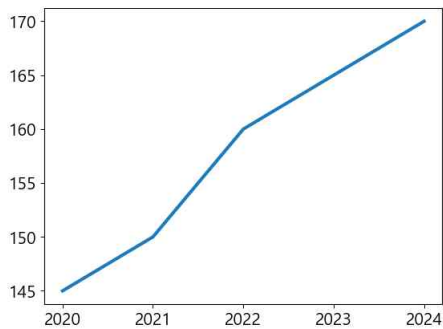
```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
year = [2020, 2021, 2022, 2023, 2024]
height = [145, 150, 160, 165, 170]
```

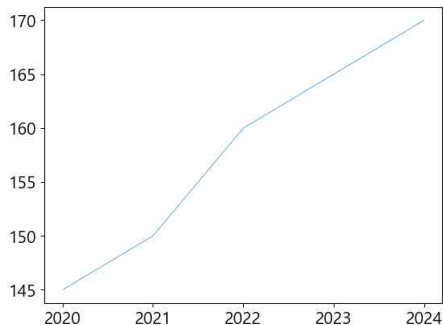
```
plt.plot(year, height)
plt.show()
```



```
plt.plot(year, height, linewidth=5)
plt.show()
```



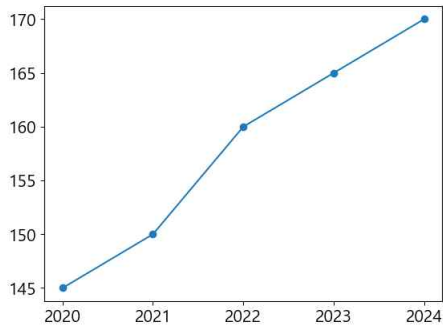
```
plt.plot(year, height, linewidth=0.5)
plt.show()
```



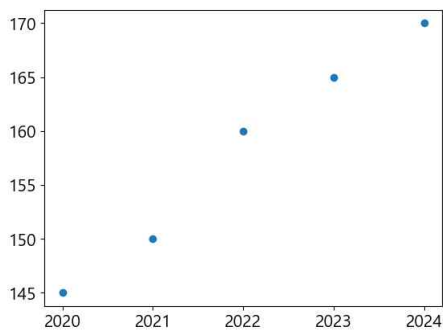
- 마커(Marker)

- 마커의 모양(marker), 크기(markersize), 면색(markeredgecolor), 섀드(markerfacecolor)을 지정한다.

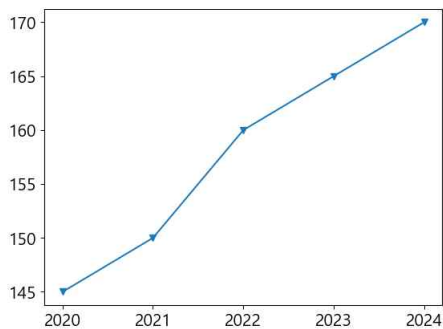
```
plt.plot(year, height, marker='o')
```



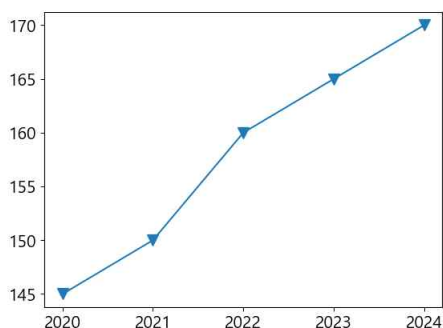
```
plt.plot(year, height, marker='o', linestyle='None')
```



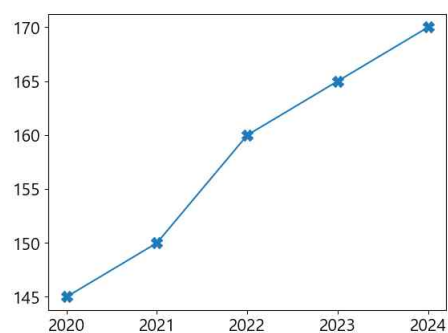
```
plt.plot(year, height, marker='v')
```



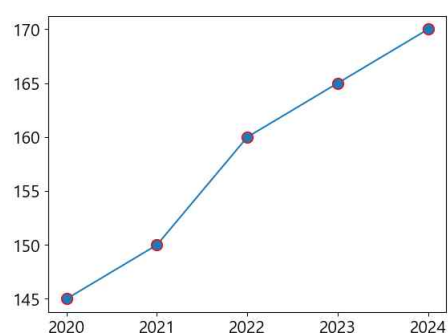
```
plt.plot(year, height, marker='v', markersize=10)
```



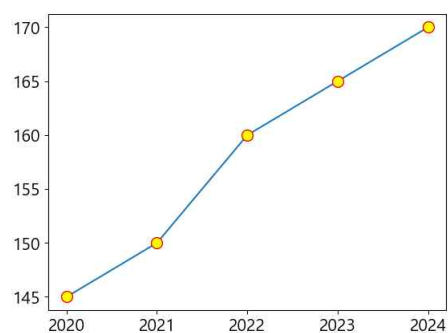
```
plt.plot(year, height, marker='X', markersize=10)
```



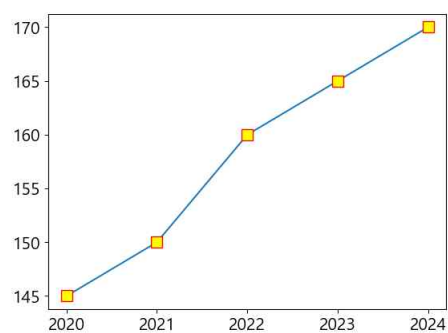
```
plt.plot(year, height, marker='o', markersize=10, markeredgcolor='red')
```



```
plt.plot(year, height, marker='o', markersize=10, markeredgcolor='red', markerfacecolor='yellow')
```

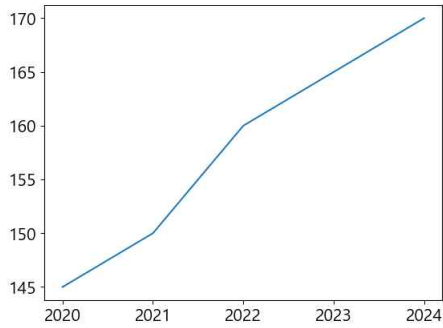


```
plt.plot(year, height, marker='o', markersize=10, markeredgcolor='red', markerfacecolor='yellow')
```

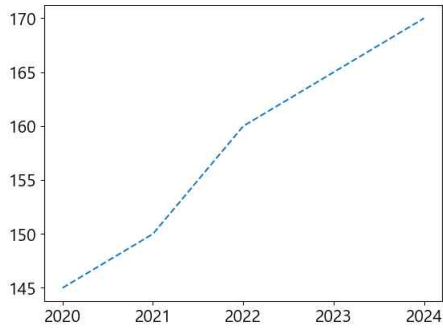


- 선 스타일(Line Style)

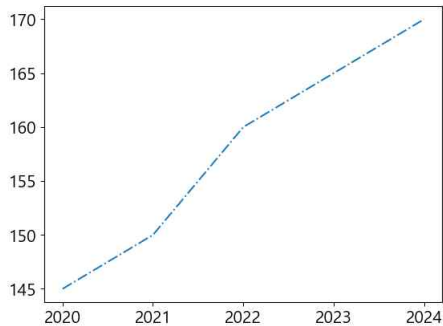
```
plt.plot(year, height, linestyle='-')
```



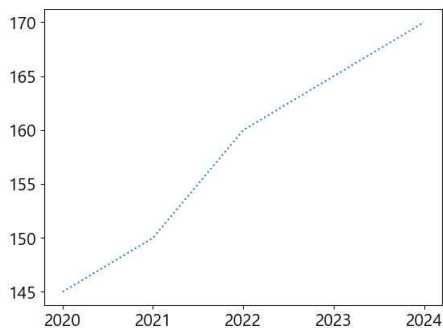
```
plt.plot(year, height, linestyle='--')
```



```
plt.plot(year, height, linestyle='-.')
```

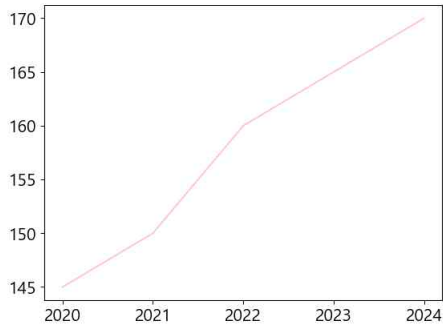


```
plt.plot(year, height, linestyle=':')
```

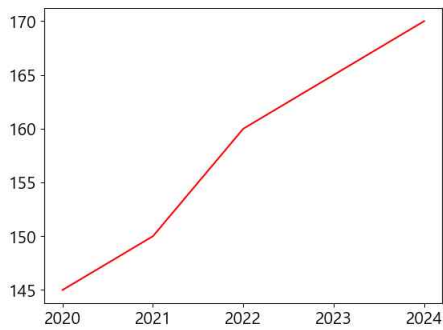


- 색깔(Color)

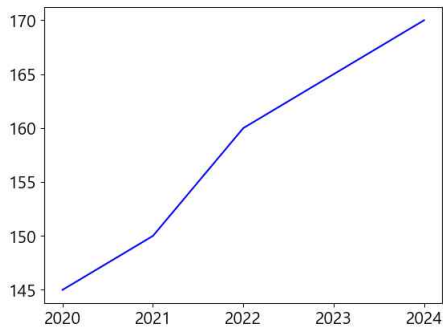
```
plt.plot(year, height, color='pink')
```



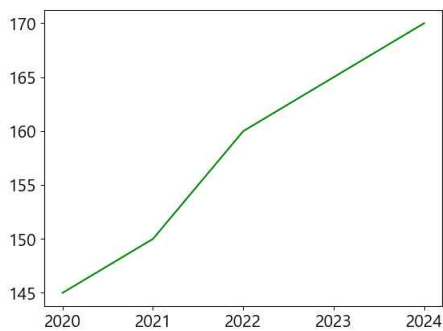
```
plt.plot(year, height, color='#FF0000')
```



```
plt.plot(year, height, color='b')
```

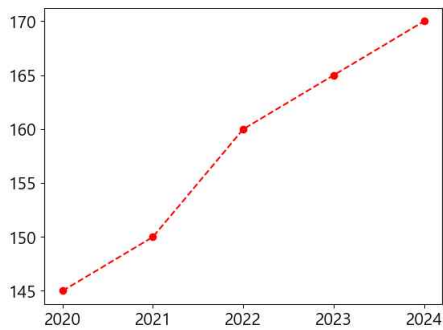


```
plt.plot(year, height, color='g')
```

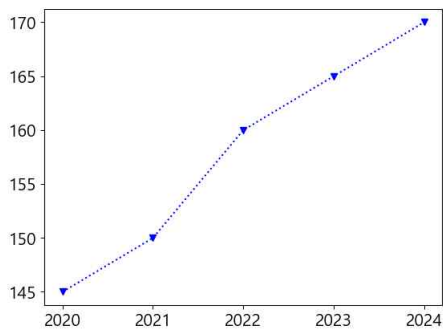


- 포맷(Format)

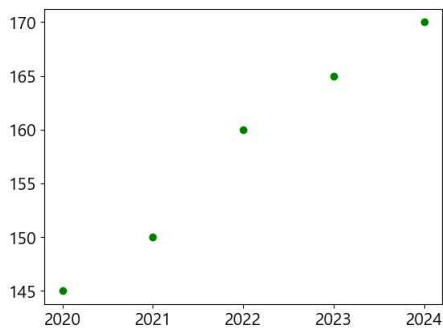
```
plt.plot(year, height, 'ro--') #color, marker, linestyle
```



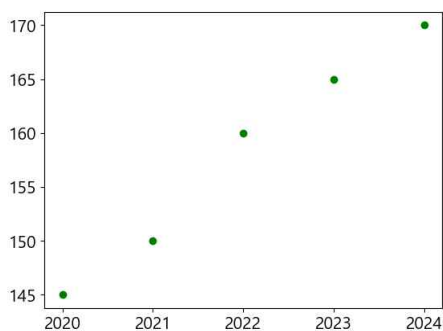
```
plt.plot(year, height, 'bv:') #파랑, 역삼각형, 점선
```



```
plt.plot(year, height, 'go') #초록색 원 선 없음
```

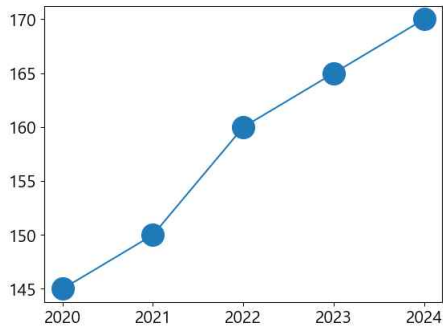


```
plt.plot(year, height, color='g', marker='o', linestyle='None')
```

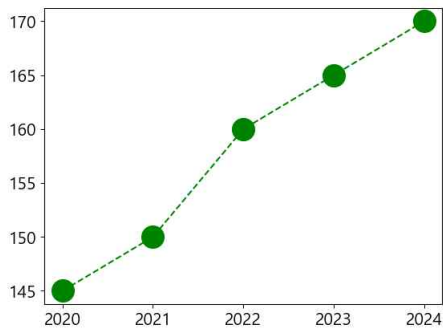


- 축약어

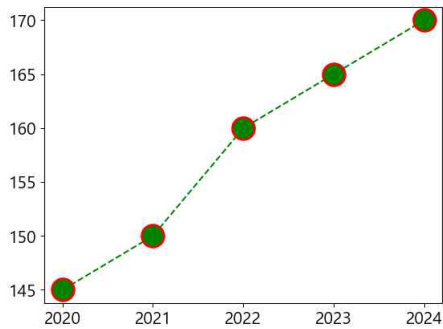
```
plt.plot(year, height, marker='o', ms=20)
```



```
plt.plot(year, height, marker='o', ms=20, c='g', ls='--')
```

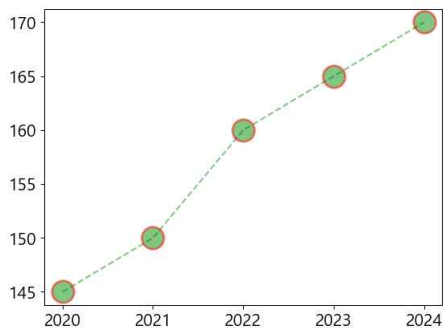


```
plt.plot(year, height, marker='o', ms=20, c='g', ls='--', mfc='g', mec='r', mew=2)
```



- 투명도

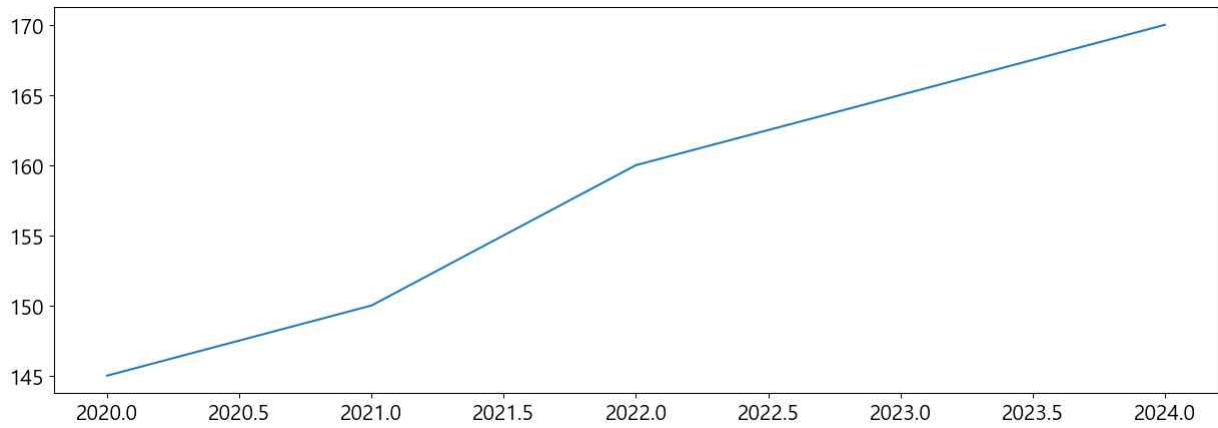
```
plt.plot(year, height, marker='o', ms=20, c='g', ls='--', mfc='g', mec='r', mew=2, alpha=0.5)
```



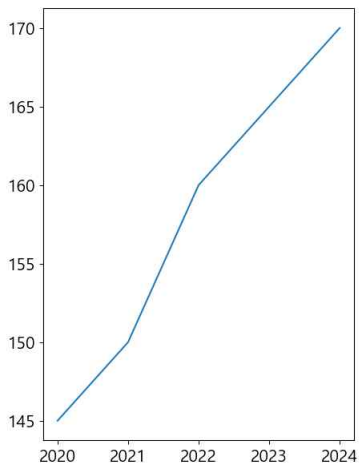
- 그래프 크기

그래프의 사이즈를 설정할 때는 `figsize(가로길이, 세로길이)` 옵션을 설정합니다. 단위는 inch이다.

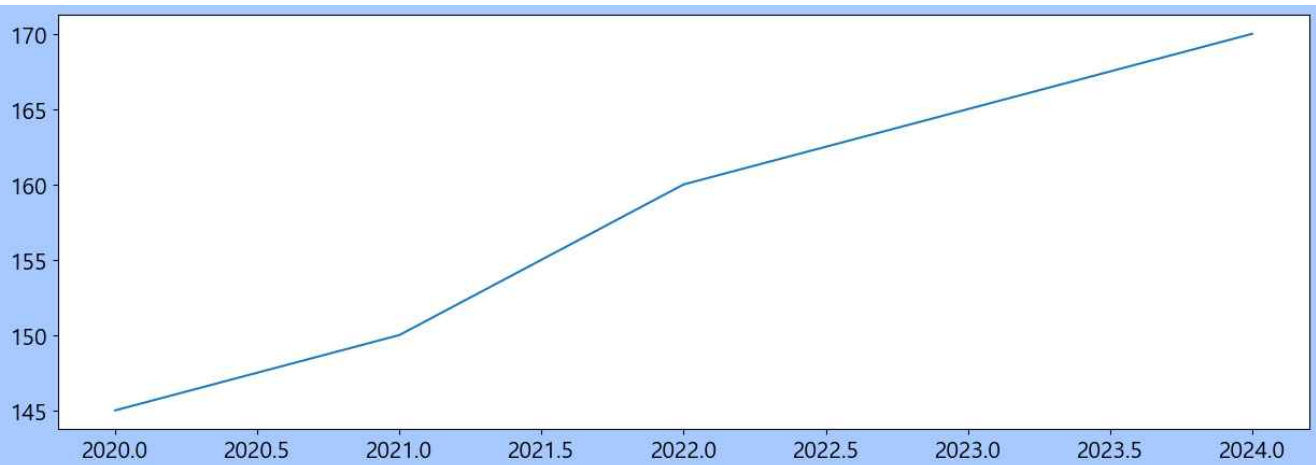
```
plt.figure(figsize=(15, 5))  
plt.plot(year, height)
```



```
plt.figure(figsize=(5, 7))  
plt.plot(year, height)
```

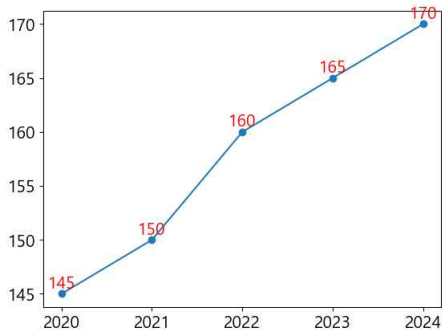


```
plt.figure(figsize=size, dpi=100, facecolor='#A1C3FF')  
plt.plot(year, height)
```



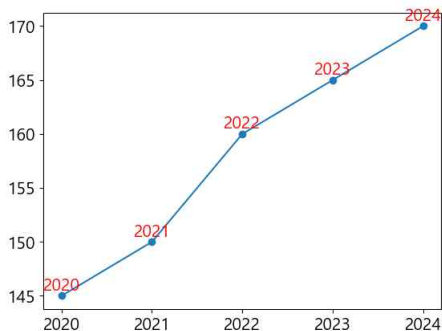
- 해당 년도 marker 위에 height(키) 값을 출력한다.

```
plt.plot(year, height, marker='o')
for idx, txt in enumerate(height):
    plt.text(year[idx], height[idx] + 0.5, txt, ha='center', color='red')
plt.show()
```



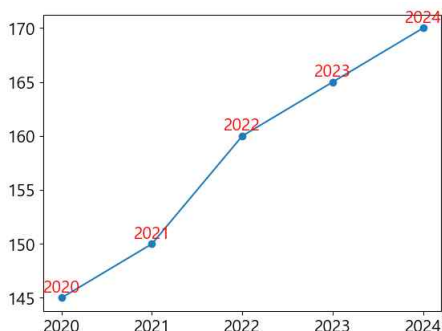
- 해당 년도 marker 위에 year(년도) 값을 출력한다.

```
plt.plot(year, height, marker='o')
for idx, txt in enumerate(year):
    plt.text(year[idx], height[idx] + 0.5, txt, ha='center', color='red')
plt.show()
```



- 위 그래프 내용을 'data/height.png'에 dpi=200으로 저장한다. (화면:100dpi)

```
plt.plot(year, height, marker='o')
for idx, txt in enumerate(year):
    plt.text(year[idx], height[idx] + 0.5, txt, ha='center', color='red')
plt.savefig('data/height.png', dpi=200)
```



05. 여러 데이터

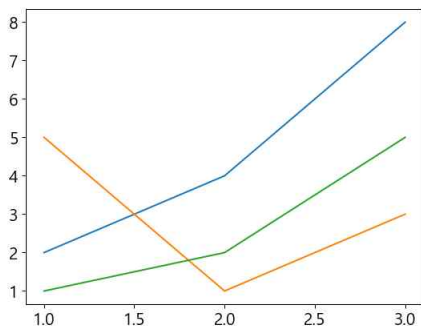
- 새로운 파일 '05.여러데이터.ipynb'를 생성한다.

```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

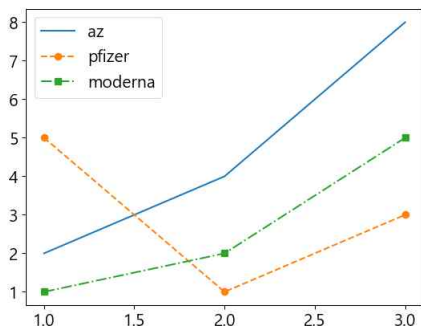
- 1일 ~ 3일까지 코로나 예방접종 인원수이다. 각 백신별, 날짜별 접종인원을 그래프로 작성한다.
(days : 1일, 2일, 3일, ax : 아스트라제네카, pfizer : 화이자, moderna : 모더나)

```
days = [1, 2, 3]
az = [2, 4, 8]
pfizer = [5, 1, 3]
moderna = [1, 2, 5]
```

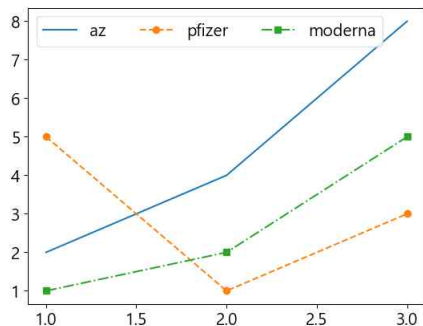
```
plt.plot(days, az)
plt.plot(days, pfizer)
plt.plot(days, moderna)
plt.show()
```



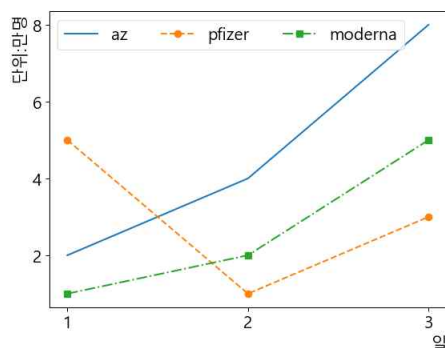
```
plt.plot(days, az, label='az')
plt.plot(days, pfizer, label='pfizer', marker='o', linestyle='--')
plt.plot(days, moderna, label='moderna', marker='s', ls='-.')
plt.legend()
plt.show()
```



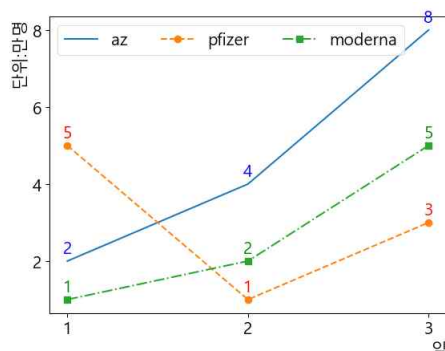
```
plt.plot(days, az, label='az')
plt.plot(days, pfizer, label='pfizer', marker='o', linestyle='--')
plt.plot(days, moderna, label='moderna', marker='s', ls='-.')
plt.legend(ncol=3)
```



```
plt.plot(days, az, label='az')
plt.plot(days, pfizer, label='pfizer', marker='o', linestyle='--')
plt.plot(days, moderna, label='moderna', marker='s', ls='-.')
plt.xticks([1, 2, 3])
plt.yticks([2, 4, 6, 8])
plt.xlabel('일', loc='right')
plt.ylabel('단위:만명', loc='top')
plt.legend(ncol=3)
```



```
...
for idx, txt in enumerate(moderna):
    plt.text(days[idx], moderna[idx]+0.2, txt, ha='center', color='green')
plt.show()
```



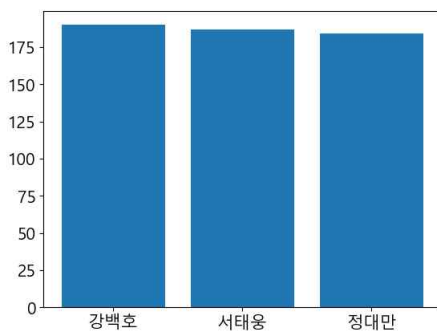
06. 막대그래프(기본)

- 새로운 파일 '06.막대그래프(기본).ipynb'를 생성한다.

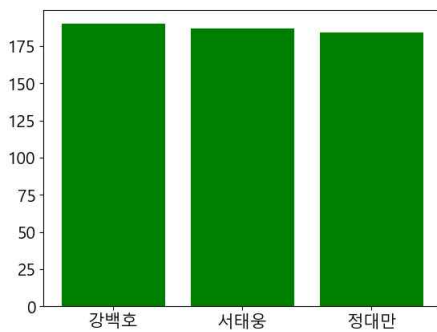
```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
labels = ['강백호', '서태웅', '정대만']
values = [190, 187, 184]
```

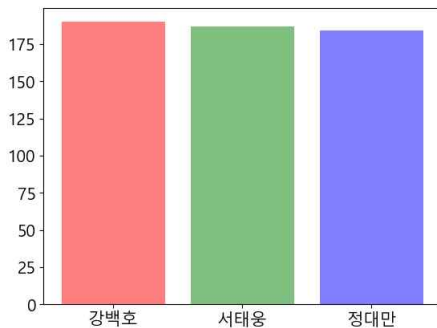
```
plt.bar(labels, values)
```



```
plt.bar(labels, values, color='g')
```

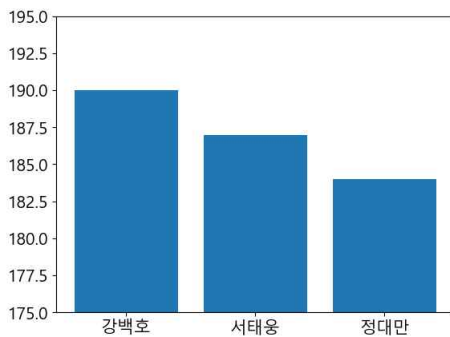


```
colors = ['r', 'g', 'b']
plt.bar(labels, values, color=colors, alpha=0.5)
plt.show()
```



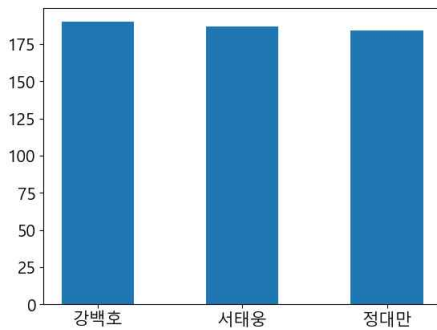
- Y축 범위를 `ylim(175, 195)` 함수를 이용해 175~195로 설정한다.

```
plt.bar(labels, values)
plt.ylim(175, 195)
plt.show()
```



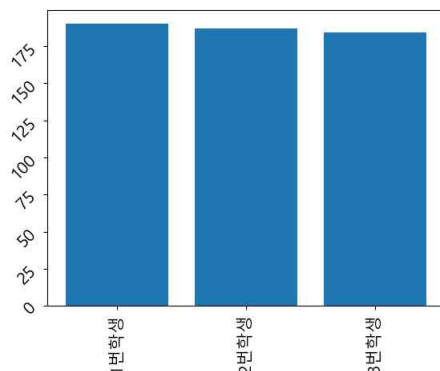
- 막대그래프의 막대넓이를 `width=0.5` 속성을 이용해 변경한다.

```
plt.bar(labels, values, width=0.5)
plt.show()
```



- 눈금의 이름과 출력 각도(rotation)를 변경한다.

```
plt.bar(labels, values)
ticks = ['1번학생', '2번학생', '3번학생']
plt.xticks(labels, ticks, rotation=90)
plt.yticks(rotation=45)
plt.show()
```



07. 막대그래프(심화)

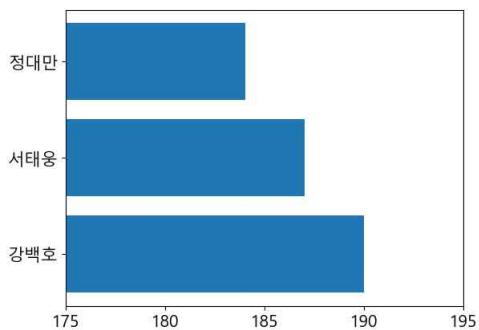
- 새로운 파일 '07.막대그래프(심화).ipynb'를 생성한다.

```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
labels = ['강백호', '서태웅', '정대만']
values = [190, 187, 184]
```

- 막대그래프를 `plt.barh`를 이용해 수평으로 출력한다.

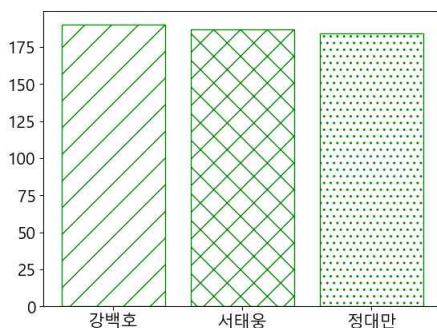
```
plt.barh(labels, values)
plt.xlim(175, 195)
plt.show()
```



```
names = [x + '님' for x in labels]
plt.barh(names, values)
plt.xlim(175, 195)
```

- 막대그래프의 `set_hatch()` 함수를 이용해 다양한 문양을 삽입할 수 있다.

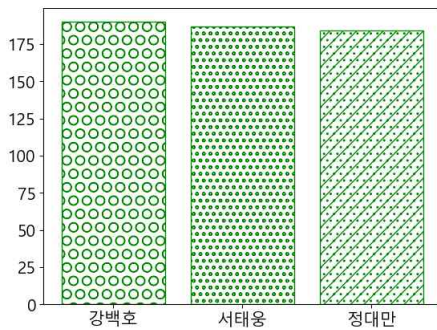
```
bar = plt.bar(labels, values, color='none', ec='g')
bar[0].set_hatch('/')
bar[1].set_hatch('x')
bar[2].set_hatch('..')
```



```

bar = plt.bar(labels, values, color='none', ec='g')
bar[0].set_hatch('O')
bar[1].set_hatch('oo')
bar[2].set_hatch('///..')

```



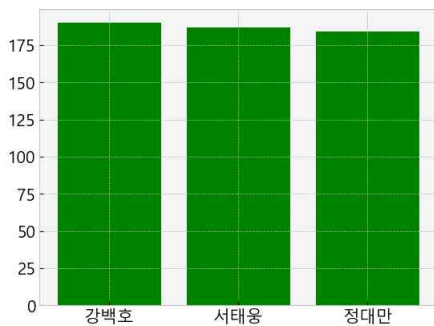
- 그래프에서 사용할 수 있는 다양한 스타일 목록을 출력한다.

```
plt.style.available
```

```

with plt.style.context('bmh'):
    plt.bar(labels, values, color='g')
    plt.show()

```



- 막대그래프의 각 bar위에 plt.text() 함수를 이용하여 점수를 출력한다.

```

bar = plt.bar(labels, values)
plt.ylim(175, 195)
for idx, rect in enumerate(bar):
    plt.text(idx, rect.get_height() + 0.5, values[idx], ha='center', color='blue')

```



08. 데이터프레임 활용

- 새로운 파일 '08.데이터프레임활용.ipynb'를 생성한다.

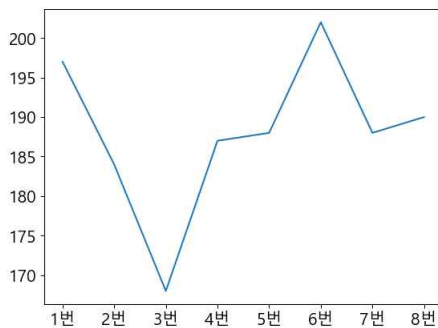
```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
df = pd.read_excel('../데이터분석/data/score.xlsx')
```

	지원번호	이름	학교	키	국어	영어	수학	과학	사회	SW특기
0	1번	채치수	북산고	197	90	85	100	95	85	Python
1	2번	정대만	북산고	184	40	35	50	55	25	Java
2	3번	송태섭	북산고	168	80	75	70	80	75	Javascript
3	4번	서태웅	북산고	187	40	60	70	75	80	NaN
4	5번	강백호	북산고	188	15	20	10	35	10	NaN
5	6번	변덕규	능남고	202	80	100	95	85	80	C
6	7번	황태산	능남고	188	55	65	45	40	35	PYTHON
7	8번	윤대협	능남고	190	100	85	90	95	95	C#

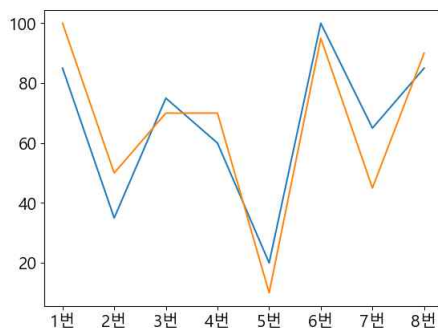
- 지원자별 키를 꺾은선 그래프로 출력한다.

```
plt.plot(df['지원번호'], df['키'])
```



- 지원자별 영어, 수학 점수를 꺾은선 그래프로 출력한다.

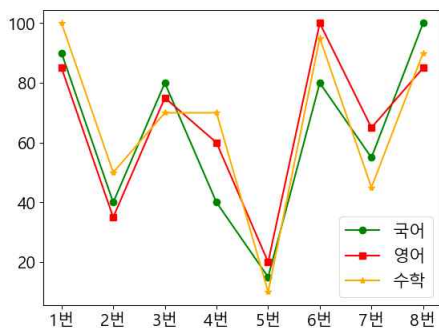
```
plt.plot(df['지원번호'], df['영어'])
plt.plot(df['지원번호'], df['수학'])
```



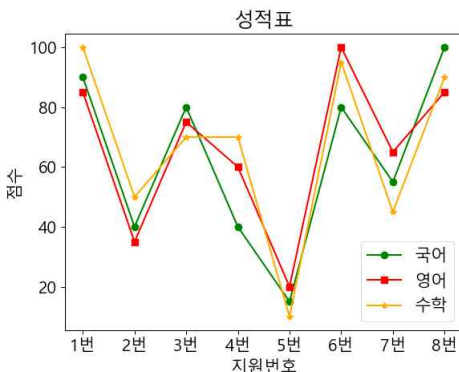
```
plt.plot(df['지원번호'], df['영어'], label='영어')
plt.plot(df['지원번호'], df['수학'])
plt.grid(axis='y', color='purple', alpha=0.5, linestyle='--', linewidth=2)
plt.show()
```



```
plt.plot(df['지원번호'], df['국어'], label='국어', color='green', marker='o')
plt.plot(df['지원번호'], df['영어'], label='영어', color='r', marker='s')
plt.plot(df['지원번호'], df['수학'], label='수학', color='orange', marker='*')
plt.legend()
plt.show()
```



```
plt.plot(df['지원번호'], df['국어'], label='국어', color='green', marker='o')
plt.plot(df['지원번호'], df['영어'], label='영어', color='r', marker='s')
plt.plot(df['지원번호'], df['수학'], label='수학', color='orange', marker='*')
plt.xlabel('지원번호')
plt.ylabel('점수')
plt.title('성적표')
plt.legend()
plt.show()
```



09. 누적 막대그래프

- 새로운 파일 '09.누적막대그래프.ipynb'를 생성한다.

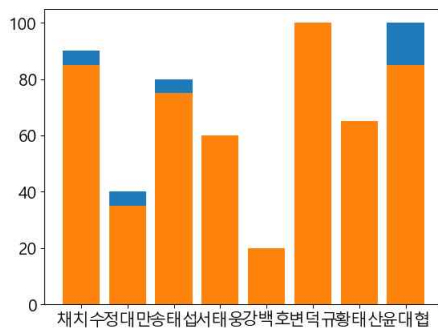
```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
df = pd.read_excel('../데이터분석/data/score.xlsx')
df.head()
```

	지원번호	이름	학교	키	국어	영어	수학	과학	사회	SW특기
0	1번	채치수	북산고	197	90	85	100	95	85	Python
1	2번	정대만	북산고	184	40	35	50	55	25	Java
2	3번	송태섭	북산고	168	80	75	70	80	75	Javascript
3	4번	서태웅	북산고	187	40	60	70	75	80	NaN
4	5번	강백호	북산고	188	15	20	10	35	10	NaN

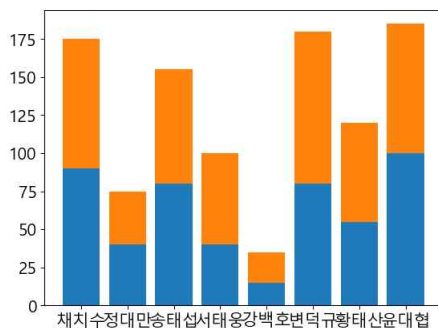
- 이름별 국어, 영어 점수를 막대그래프로 겹쳐서 출력한다.

```
plt.bar(df['이름'], df['국어'])
plt.bar(df['이름'], df['영어'])
```

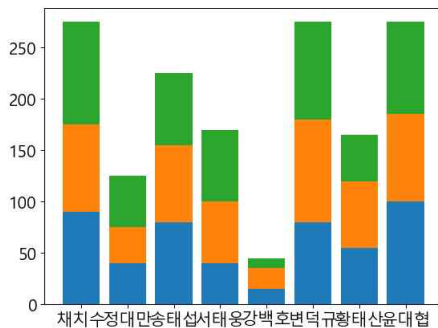


- 위 그래프를 아래에 국어, 위에 영어 점수를 누적하여 출력한다.

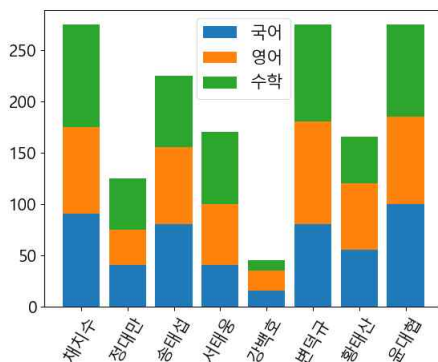
```
plt.bar(df['이름'], df['국어'])
plt.bar(df['이름'], df['영어'], bottom=df['국어'])
```



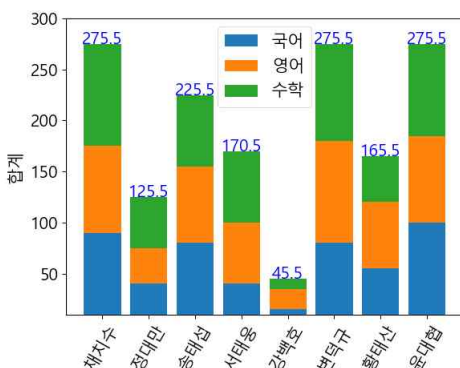
```
plt.bar(df['이름'], df['국어'])
plt.bar(df['이름'], df['영어'], bottom=df['국어'])
plt.bar(df['이름'], df['수학'], bottom=df['국어'] + df['영어'])
```



```
plt.bar(df['이름'], df['국어'], label='국어')
plt.bar(df['이름'], df['영어'], bottom=df['국어'], label='영어')
plt.bar(df['이름'], df['수학'], bottom=df['국어'] + df['영어'], label='수학')
plt.xticks(rotation=60)
plt.legend()
```



```
plt.bar(df['이름'], df['국어'], label='국어')
plt.bar(df['이름'], df['영어'], bottom=df['국어'], label='영어')
plt.bar(df['이름'], df['수학'], bottom=df['국어'] + df['영어'], label='수학')
plt.xticks(rotation=60)
plt.ylim(10, 300)
plt.ylabel('합계')
for idx, name in enumerate(df['이름']):
    sum = df['국어'][idx]+df['영어'][idx]+df['수학'][idx]
    plt.text(idx, sum, sum, ha='center', color='blue')
plt.legend()
```



10. 다중 막대그래프

- 새로운 파일 '10.다중막대그래프.ipynb'를 생성한다.

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
df = pd.read_excel('../데이터분석/data/score.xlsx')
df
```

	지원번호	이름	학교	키	국어	영어	수학	과학	사회	SW특기
0	1번	채치수	북산고	197	90	85	100	95	85	Python
1	2번	정대만	북산고	184	40	35	50	55	25	Java
2	3번	송태섭	북산고	168	80	75	70	80	75	Javascript
3	4번	서태웅	북산고	187	40	60	70	75	80	NaN
4	5번	강백호	북산고	188	15	20	10	35	10	NaN
5	6번	변덕규	능남고	202	80	100	95	85	80	C
6	7번	황태산	능남고	188	55	65	45	40	35	PYTHON
7	8번	윤대협	능남고	190	100	85	90	95	95	C#

- numpy 라이브러리의 `arrange()` 함수를 이용하여 데이터프레임(df) 행의 수만큼 정수를 생성한다.

```
import numpy as np
index = np.arange(df.shape[0])
index
```

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

- 이름, 국어, 영어, 수학, 인덱스 값들을 각각 변수에 저장한다.

```
kor = df['국어']
eng = df['영어']
mat = df['수학']
names = df['이름']
index = df.index
index
```

```
RangeIndex(start=0, stop=8, step=1)
```

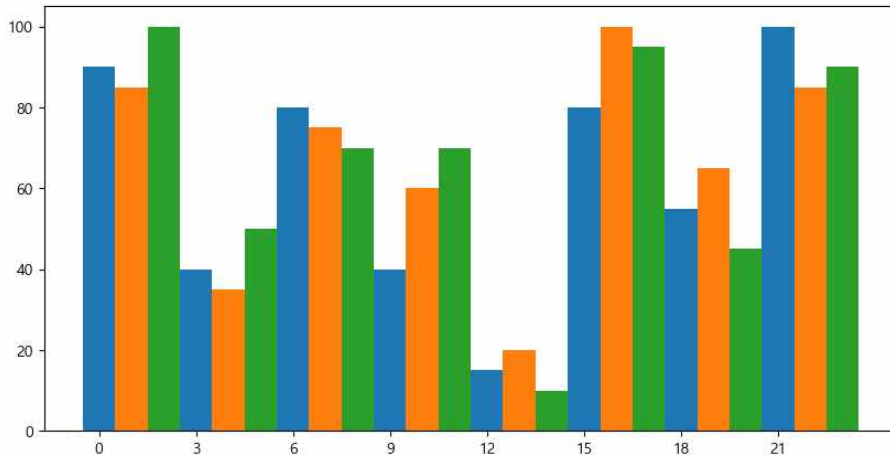
- 학생별 3과목(국어, 영어, 수학)이므로 x축의 값을 3씩 증가시킨다.

```
x = index * 3
```

```
RangeIndex(start=0, stop=24, step=3)
```

- 학생별 3과목(국어, 영어, 수학)에 대한 다중 막대그래프를 출력한다.

```
plt.figure(figsize=(10, 5))
plt.bar(x+0, kor, width=1)
plt.bar(x+1, eng, width=1)
plt.bar(x+2, mat, width=1)
plt.xticks(x)
plt.show()
```

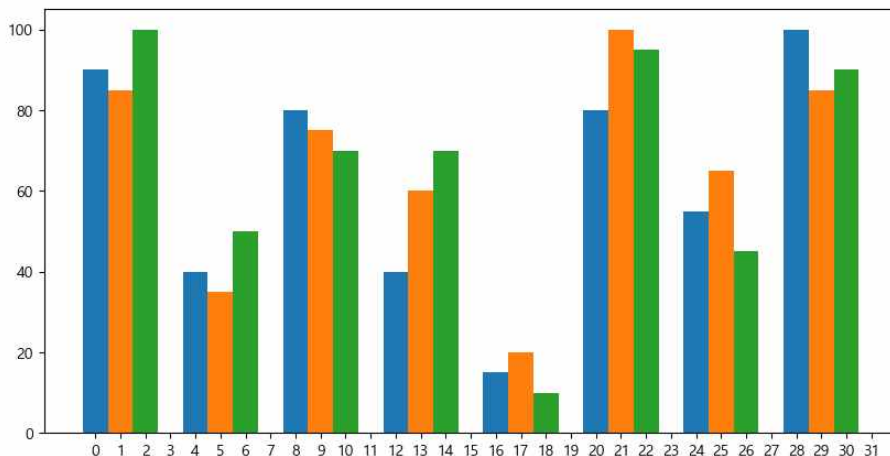


- 학생별 간격을 1만큼 띄어서 출력하기 위해 각 index값을 4씩 증가시킨다.

```
x = index * 4
RangeIndex(start=0, stop=32, step=4)
```

- 학생별 3과목(국어, 영어, 수학)에 대한 다중 막대그래프를 출력한다.

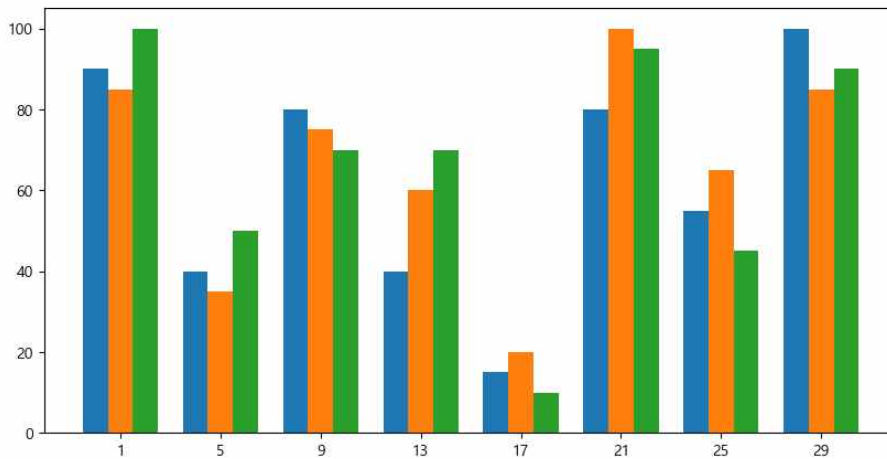
```
plt.figure(figsize=(10, 5))
plt.bar(x+0, kor, width=1)
plt.bar(x+1, eng, width=1)
plt.bar(x+2, mat, width=1)
plt.xticks([x for x in range(0, len(x)*4, 1)]) #plt.xticks(x)
plt.show()
```



- **xticks** 값을 가운데로 조정하기 위해 1을 더해준다.

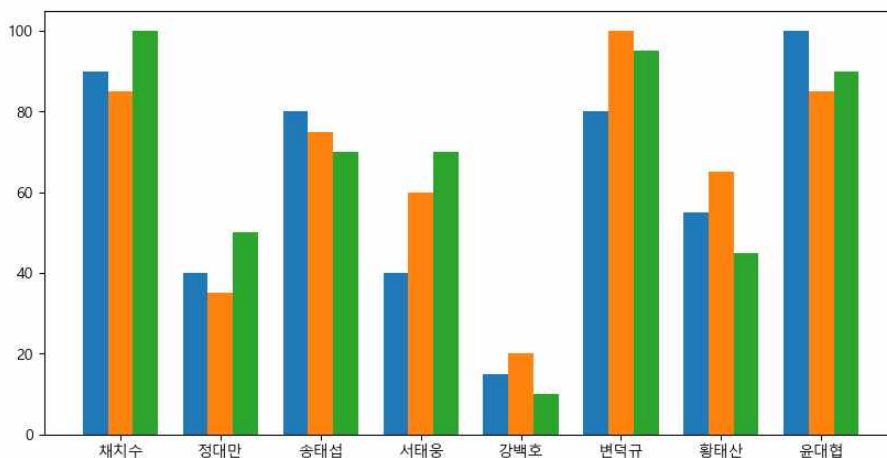
bar 수	1	2	3	4	5	n
중앙위치	0	start + 0.5	start + 1	start + 1.5	start + 2	start+ (n-1)*0.5

```
plt.figure(figsize=(10, 5))
plt.bar(x+0, kor, width=1)
plt.bar(x+1, eng, width=1)
plt.bar(x+2, mat, width=1)
plt.xticks([ start+1 for start in x ])
plt.show()
```



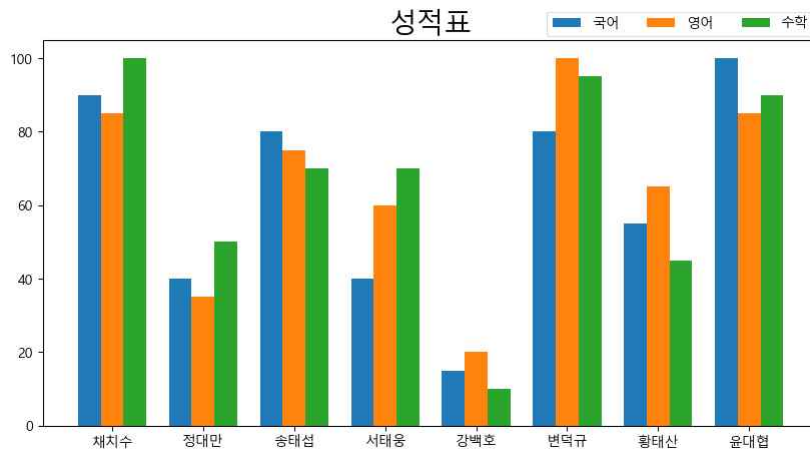
- **xticks** 레이블을 **names**값으로 변경한다.

```
plt.figure(figsize=(10, 5))
plt.bar(x+0, kor, width=1)
plt.bar(x+1, eng, width=1)
plt.bar(x+2, mat, width=1)
plt.xticks([start+1 for start in x], names)
plt.show()
```



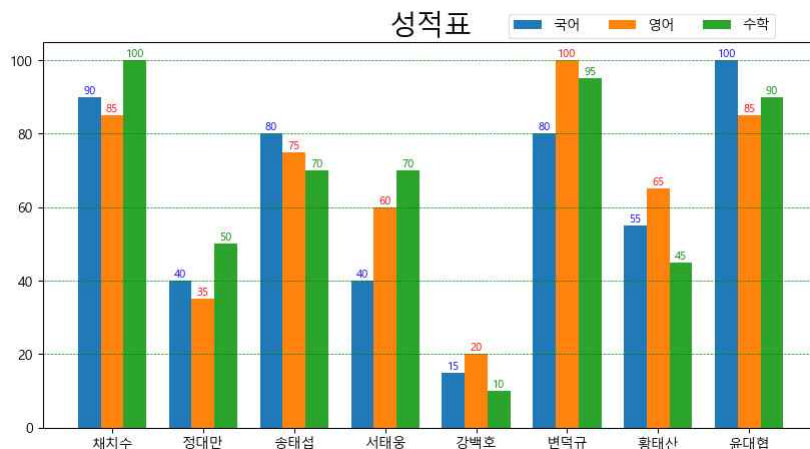
- 제목과 범례를 출력한다.

```
plt.figure(figsize=(10, 5))
plt.bar(x+0, kor, width=1, label='국어')
plt.bar(x+1, eng, width=1, label='영어')
plt.bar(x+2, mat, width=1, label='수학')
plt.xticks([start+1 for start in x], names)
plt.legend(ncol=3, loc=(0.65, 1.01))
plt.title('성적표', size=20)
```



- 사회, 과학 과목을 추가하여 다중 막대 그래프를 출력한다.

```
plt.figure(figsize=(10, 5))
plt.bar(x+0, kor, width=1, label='국어')
plt.bar(x+1, eng, width=1, label='영어')
plt.bar(x+2, mat, width=1, label='수학')
plt.xticks([start+1 for start in x], names)
plt.legend(ncol=5, loc=(0.6, 1.01))
plt.title('성적표', size=20)
for idx, name in enumerate(names):
    plt.text(x[idx]+0, kor[idx]+1, kor[idx], size=8, ha='center', color='b')
    plt.text(x[idx]+1, eng[idx]+1, eng[idx], size=8, ha='center', color='r')
    plt.text(x[idx]+2, mat[idx]+1, mat[idx], size=8, ha='center', color='g')
plt.grid(axis='y', ls='--', lw=0.5, color='g')
```



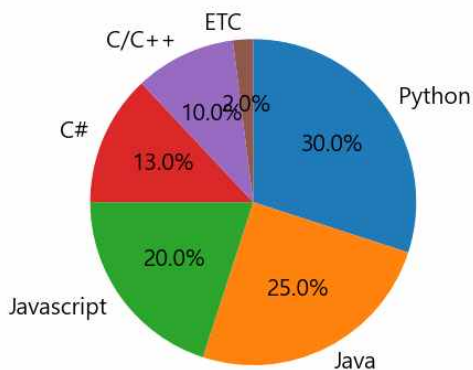
11. 원 그래프(기본)

- 새로운 파일 '11.원그래프(기본).ipynb'를 생성한다.

```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

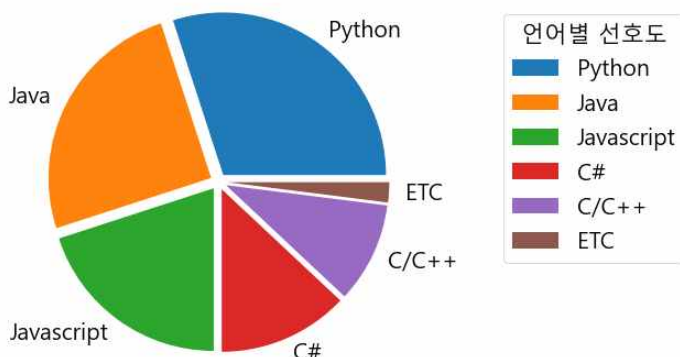
- 프로그램 언어별 사용한 가능한 인원수에 대한 원그래프를 출력한다. (%.1f%% : 소수점 첫째자리까지 %포함하여 출력)

```
values = [30, 25, 20, 13, 10, 2]
labels = ['Python', 'Java', 'Javascript', 'C#', 'C/C++', 'ETC']
plt.pie(values, labels = labels, autopct='%.1f%%', startangle=90, counter-clock=False)
plt.show()
```



- explode 속성을 이용해 프로그램 언어별 0.05만큼 간격으로 출력한다.

```
values = [30, 25, 20, 13, 10, 2]
labels = ['Python', 'Java', 'Javascript', 'C#', 'C/C++', 'ETC']
explode = [0.05] * 6
plt.pie(values, labels=labels, explode=explode)
plt.legend(loc=(1.2, 0.3), title='언어별 선호도')
plt.show()
```

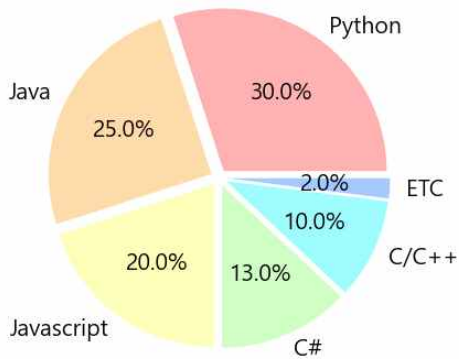


14. 원 그래프(심화)

- 새로운 파일 '14.원그래프(심화).ipynb'를 생성한다.

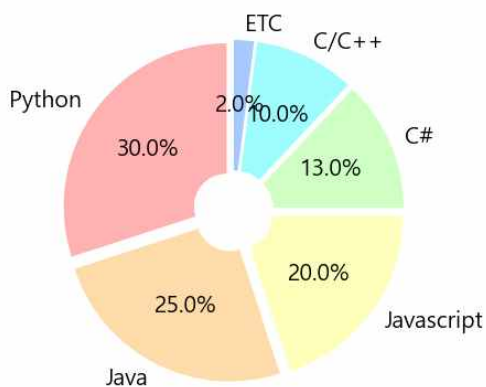
```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
values = [30, 25, 20, 13, 10, 2]
labels = ['Python', 'Java', 'Javascript', 'C#', 'C/C++', 'ETC']
colors = ['#FFADAD', '#FFD6A5', '#FDFB6', '#CAFFBF', '#9BF6FF', '#A0C4FF']
explode = [0.05] * 6
plt.pie(values, labels=labels, colors=colors, explode=explode,
        autopct='%1f%%')
plt.show()
```



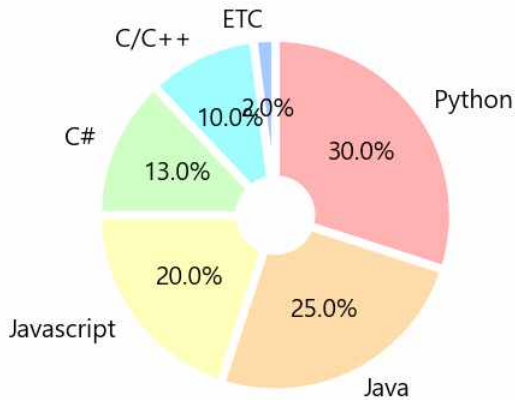
- wedge의 넓이를 0.8, 회전을 시계반대방향으로 90도 회전한 원그래프를 출력한다.

```
wedgeprops={'width':0.8}
plt.pie(values, labels=labels, colors=colors, explode=explode,
        autopct='%1f%%',
        wedgeprops=wedgeprops,
        counterclock=True, startangle=90)
plt.show()
```



- edge 속성 넓이를 0.8, 색상을 흰색, 선 넓이를 5로 지정하고 시계방향으로 90도 회전한 원그래프를 출력한다.

```
wedgeprops={'width':0.8, 'edgecolor':'w', 'linewidth':5}
plt.pie(values, labels=labels, colors=colors,
        autopct='%.1f%%',
        wedgeprops=wedgeprops,
        counterclock=False, startangle=90)
plt.show()
```

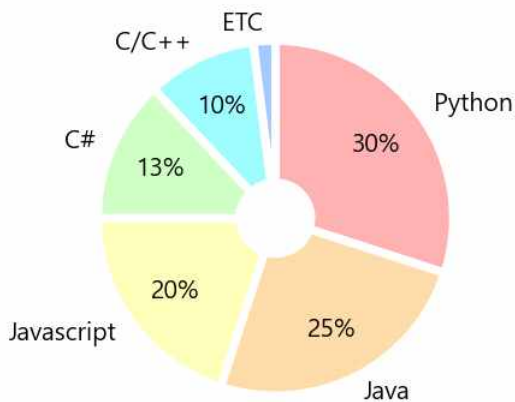


- 퍼센트 값이 10이상인 경우에는 값을 출력하고 아니면 공백을 리턴 하는 함수를 작성한다.

```
def custom_autopct(pct):
    #return '{:.0f}%'.format(pct) if pct >=10 else ''
    return f'{pct:.0f}%' if pct >= 10 else ''
```

- 퍼센트 출력을 위 함수(custom_autopce)를 이용하여 구하고 퍼센트 출력거리를 0.7로 지정한다.

```
plt.pie(values, labels=labels, colors=colors,
        autopct=custom_autopct,
        wedgeprops=wedgeprops,
        startangle=90, counterclock=False,
        pctdistance=0.7)
plt.show()
```



- 데이터프레임 활용한 원그래프

```
import pandas as pd
df = pd.read_excel('../데이터분석/data/score.xlsx')
df
```

	지원번호	이름	학교	키	국어	영어	수학	과학	사회	SW특기
0	1번	채치수	북산고	197	90	85	100	95	85	Python
1	2번	정대만	북산고	184	40	35	50	55	25	Java
2	3번	송태섭	북산고	168	80	75	70	80	75	Javascript
3	4번	서태웅	북산고	187	40	60	70	75	80	NaN
4	5번	강백호	북산고	188	15	20	10	35	10	NaN
5	6번	변덕규	능남고	202	80	100	95	85	80	C
6	7번	황태산	능남고	188	55	65	45	40	35	PYTHON
7	8번	윤대협	능남고	190	100	85	90	95	95	C#

- 학교별 그룹화한 후 학생 수와 학교명을 각각 변수에 저장한다.

```
df_group = df.groupby('학교').size()
df_group
```

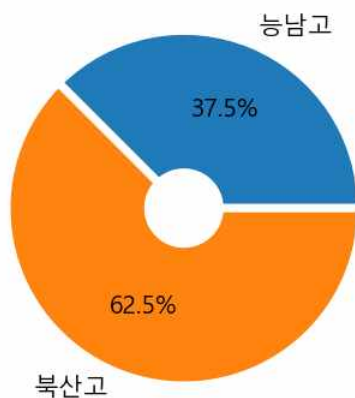
```
학교
능남고    3
북산고    5
dtype: int64
```

```
values=list(df_group.values)
labels=list(df_group.index)
print(values, labels)
```

```
[np.int64(3), np.int64(5)] ['능남고', '북산고']
```

- 학교별 학생수 의 퍼센트를 원그래프로 출력한다.

```
wedgeprops={'width':0.8, 'edgecolor':'w', 'linewidth':5}
plt.pie(values, labels=labels, autopct='%1f%%', wedgeprops=wedgeprops)
plt.show()
```



df.head()

	지원번호	이름	학교	키	국어	영어	수학	과학	사회	SW특기
0	1번	채치수	북산고	197	90	85	100	95	85	Python
1	2번	정대만	북산고	184	40	35	50	55	25	Java
2	3번	송태섭	북산고	168	80	75	70	80	75	Javascript
3	4번	서태웅	북산고	187	40	60	70	75	80	NaN
4	5번	강백호	북산고	188	15	20	10	35	10	NaN

- 'SW특기' 첫 글자를 대문자로 바꾸고 null값에 '없음'이라고 저장한다.

```
df['SW특기'] = df['SW특기'].str.capitalize()
df['SW특기'] = df['SW특기'].fillna('없음')
df.head()
```

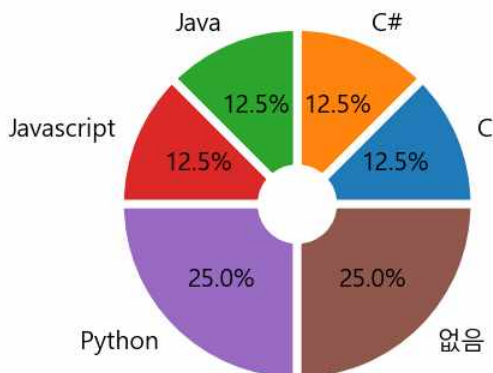
	지원번호	이름	학교	키	국어	영어	수학	과학	사회	SW특기
0	1번	채치수	북산고	197	90	85	100	95	85	Python
1	2번	정대만	북산고	184	40	35	50	55	25	Java
2	3번	송태섭	북산고	168	80	75	70	80	75	Javascript
3	4번	서태웅	북산고	187	40	60	70	75	80	없음
4	5번	강백호	북산고	188	15	20	10	35	10	없음

- 'SW특기'별로 학생 수를 구한 후 list로 변환하여 변수에 저장한다.

```
df_group = df.groupby('SW특기').size()
labels = list(df_group.index)
values = list(df_group.values)
print(labels, values)
```

- 'SW특기'별 학생 수를 퍼센트로 변환하여 원그래프를 출력한다.

```
wedgeprops={'width':0.8, 'edgecolor':'w', 'linewidth':5}
plt.pie(values, labels=labels, autopct='%1f%%', wedgeprops=wedgeprops)
plt.show()
```



13. 산점도 그래프

- 새로운 파일 '13.산점도그래프.ipynb'를 생성한다.

```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
import pandas as pd
df = pd.read_excel('../데이터분석/data/score.xlsx')
```

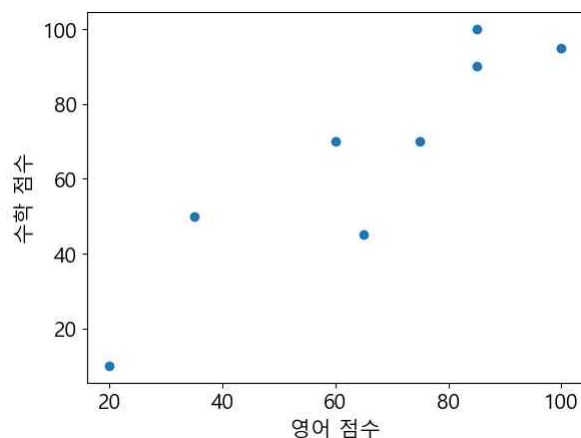
- 데이터프레임(df) 새로운 '학년'열에 아래 학년 데이터를 입력한다.

```
df['학년'] = [3, 3, 2, 1, 1, 3, 2, 2]
df
```

	이름	학교	키	국어	영어	수학	과학	사회	SW특기	학년
지원번호										
1번	채치수	북산고	197	90	85	100	95	85	Python	3
2번	정대만	북산고	184	40	35	50	55	25	Java	3
3번	송태섭	북산고	168	80	75	70	80	75	Javascript	2
4번	서태웅	북산고	187	40	60	70	75	80	NaN	1
5번	강백호	북산고	188	15	20	10	35	10	NaN	1
6번	변덕규	능남고	202	80	100	95	85	80	C	3
7번	황태산	능남고	188	55	65	45	40	35	PYTHON	2
8번	윤대협	능남고	190	100	85	90	95	95	C#	2

- X축을 영어점수, Y축을 수학점수로 산점도 그래프를 출력한다.

```
plt.scatter(df['영어'], df['수학'])
plt.xlabel('영어 점수')
plt.ylabel('수학 점수')
```



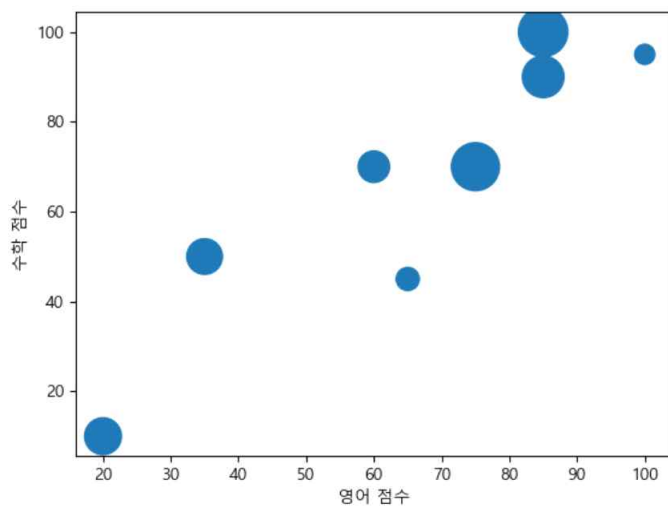
- sizes 변수에 numpy의 random 값을 8개 생성하여 저장한다.

```
import numpy as np
sizes = np.random.rand(8) * 1000 #np.random.rand(8) 0~1사이 random값 8개 발생
sizes

array([725.59542822, 751.12690514, 124.56294605, 724.72915655,
       918.44665112, 716.67121237, 586.01135598, 927.21700885])
```

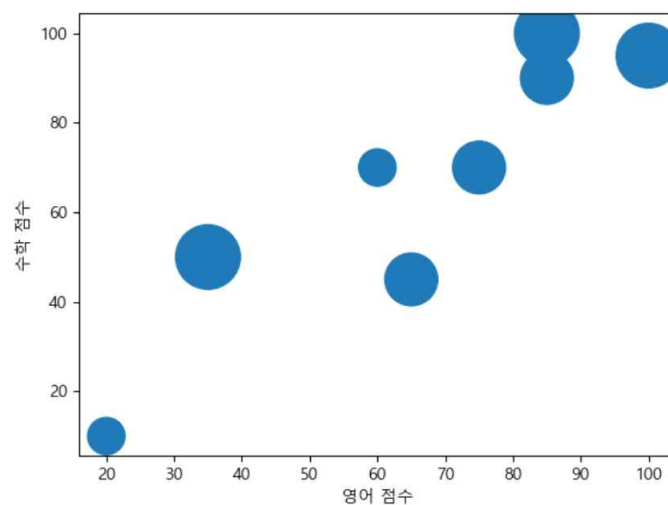
- 산점도 그래프 도트 사이즈일 1학년(500), 2학년(1000), 3학년(1500)으로 sizes변수에 저장한다.

```
plt.scatter(df['영어'], df['수학'], s=sizes)
plt.xlabel('영어 점수')
plt.ylabel('수학 점수')
plt.show()
```



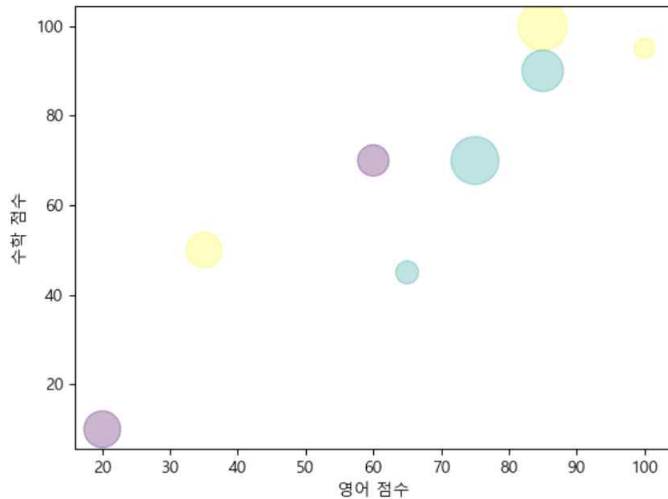
- 산점도 그래프 도트 사이즈일 1학년(500), 2학년(1000), 3학년(1500)으로 sizes변수에 저장한다.

```
sizes = df['학년'] * 500
plt.scatter(df['영어'], df['수학'], s=sizes)
plt.xlabel('영어 점수')
plt.ylabel('수학 점수')
plt.show()
```



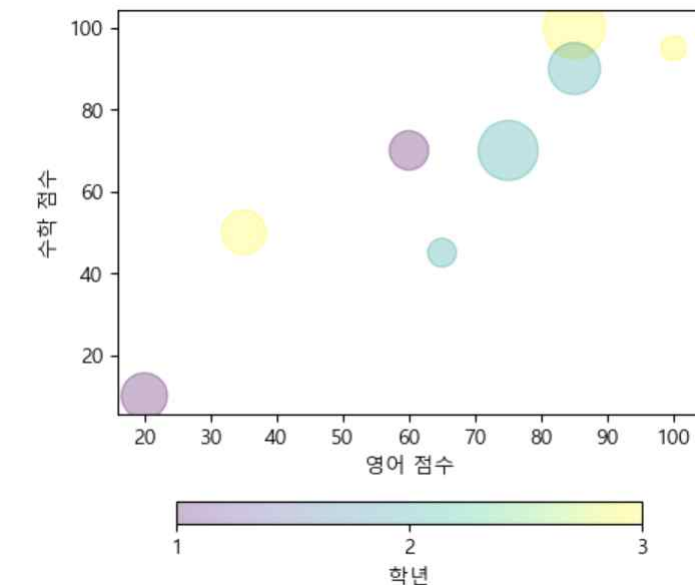
- 산점도 그래프의 도트 색상을 학년별로 다르게 출력하고 cmapd를 이용해 다양한 색상목록을 이용한다.
(cmap='plasma', cmap='inferno', cmap='magma', cmap='cividis', spring, summer, winter)

```
plt.scatter(df['영어'], df['수학'], s=sizes, c=df['학년'], cmap='viridis', alpha=0.3)
plt.xlabel('영어 점수')
plt.ylabel('수학 점수')
plt.show()
```



- 컬러 바(colorbar)를 가로로(horizontal) 축소(shrink)를 0.5로 출력한다.

```
plt.figure(figsize=(5, 5))
plt.scatter(df['영어'], df['수학'], s=sizes, c=df['학년'], cmap='viridis', alpha=0.3)
plt.xlabel('영어 점수')
plt.ylabel('수학 점수')
plt.colorbar(ticks=[1, 2, 3], label='학년', shrink=0.5, orientation='horizontal')
plt.show()
```



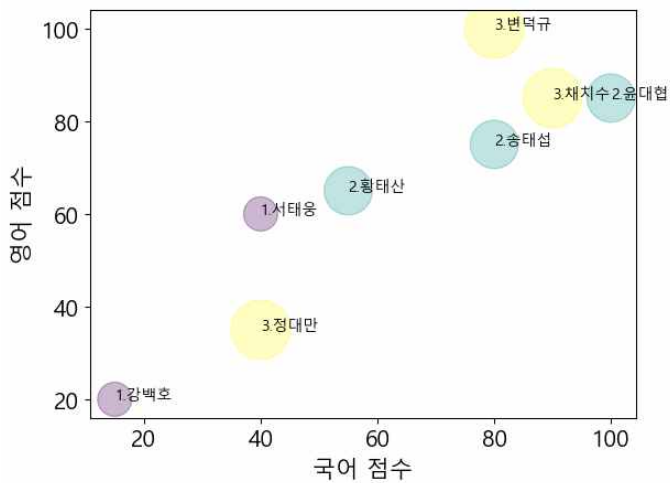
```
ticks = [x for x in range(min(df['학년']), max(df['학년'])+ 1, 1)]
ticks
```

```
[1, 2, 3]
```

- 국어, 영어 점수별 학년과 이름을 출력하는 산점도 그래프를 출력한다.

```
names = list(df['이름'])
year = list(df['학년'])
x = list(df['국어'])
y = list(df['영어'])
```

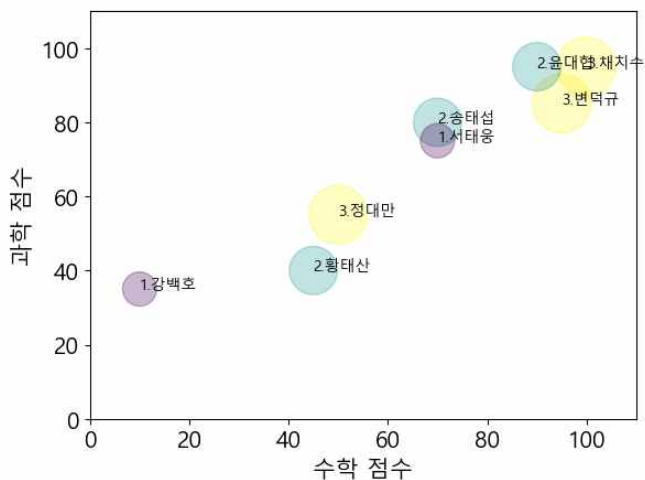
```
plt.scatter(x, y, s=sizes, c=year, cmap='viridis', alpha=0.3)
plt.xlabel('영어 점수')
plt.ylabel('수학 점수')
for idx, name in enumerate(names):
    plt.text(x[idx], y[idx], f'{year[idx]}.{name}', size=10)
```



- 수학, 과학 점수별 학년과 이름을 출력하는 산점도 그래프를 출력한다.

```
x = list(df['수학'])
y = list(df['과학'])
```

```
plt.scatter(x, y, s=sizes, c=df['학년'], cmap='viridis', alpha=0.3)
plt.xlabel('수학 점수')
plt.ylabel('과학 점수')
for idx, name in enumerate(names):
    plt.text(x[idx], y[idx], str(year[idx]) + '.' + name, size=10)
```



14. 여러 그래프

- 새로운 파일 '14.여러그래프.ipynb'를 생성한다.

```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
import pandas as pd
df = pd.read_excel('../데이터분석/data/score.xlsx')
df.head(3)
```

	지원번호	이름	학교	키	국어	영어	수학	과학	사회	SW특기
0	1번	채치수	북산고	197	90	85	100	95	85	Python
1	2번	정대만	북산고	184	40	35	50	55	25	Java
2	3번	송태섭	북산고	168	80	75	70	80	75	Javascript

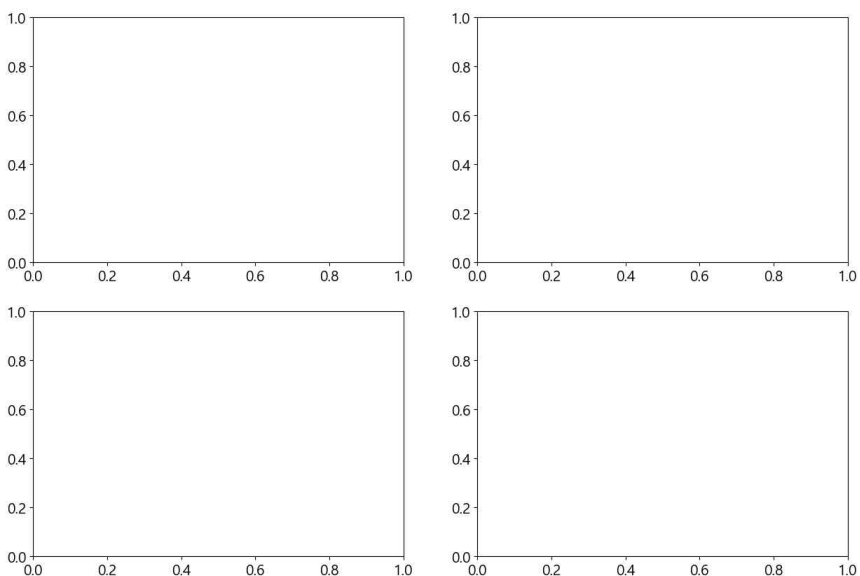
```
df['학년'] = [3, 3, 2, 1, 1, 3, 2, 2]
df.head(3)
```

	지원번호	이름	학교	키	국어	영어	수학	과학	사회	SW특기	학년
0	1번	채치수	북산고	197	90	85	100	95	85	Python	3
1	2번	정대만	북산고	184	40	35	50	55	25	Java	3
2	3번	송태섭	북산고	168	80	75	70	80	75	Javascript	2

- plt.subplots() 함수를 이용하여 2행, 2열의 그래프들을 생성한다. (fig:그래프화면, axs:2행 2열의 그래프)

```
fig, axs = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle('여러 그래프 넣기')
plt.show()
```

여러 그래프 넣기



- 모양이 다른 네 개의 그래프들(막대, 선, 원, 산점도)을 출력한다.

```
fig, axs = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle('여러 그래프 넣기')

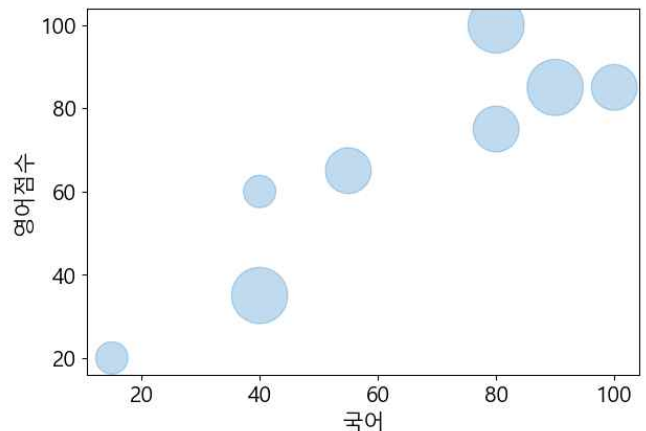
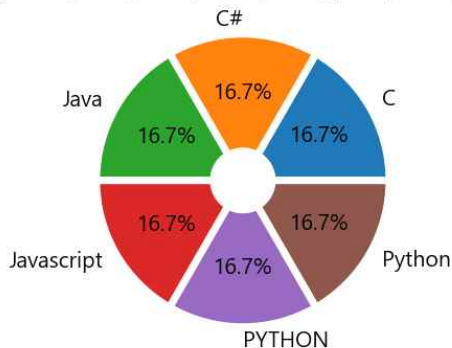
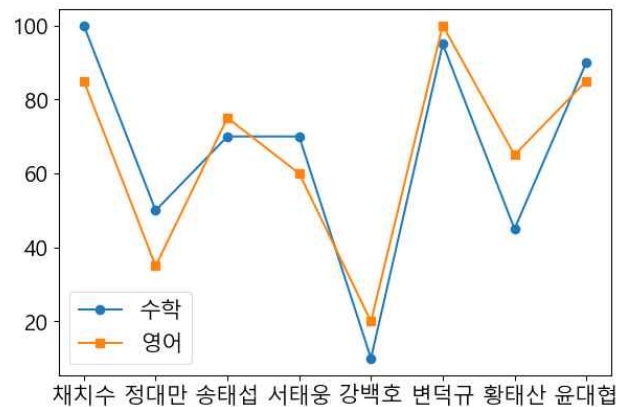
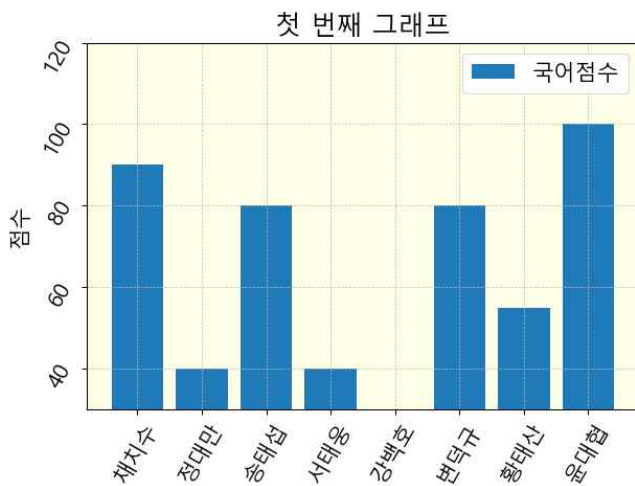
axs[0, 0].bar(df['이름'], df['국어'], label='국어점수')
axs[0, 0].set_title('첫 번째 그래프')
axs[0, 0].legend() #plt.xticks(rotation=60)
axs[0, 0].tick_params(rotation=60)
axs[0, 0].set_ylabel='점수', ylim=(30,120))
axs[0, 0].set_facecolor('lightyellow')
axs[0, 0].grid(linestyle='--', linewidth=0.5)

axs[0, 1].plot(df['이름'], df['수학'], label='수학', marker='o')
axs[0, 1].plot(df['이름'], df['영어'], label='영어', marker='s')
axs[0, 1].legend()

group = df.groupby('SW특기').size()
values = list(group.values)
labels = list(group.index)
wedgeprops={'width':0.8, 'edgecolor':'w', 'linewidth':5}
axs[1, 0].pie(values, labels=labels, autopct='%.1f%%', wedgeprops=wedgeprops)

axs[1, 1].scatter(df['국어'], df['영어'], alpha=0.3, s=df['학년']*500)
axs[1, 1].set(xlabel='국어', ylabel='영어점수')
plt.show()
```

여러 그래프 넣기



15. 퀴즈

- 새로운 파일 '15.퀴즈.ipynb'를 생성한다.
- 다음은 대한민국 영화 중에서 관객 수가 가장 많은 상위 8개의 데이터이다. 주어진 코드를 이용하여 퀴즈를 풀어보시오.

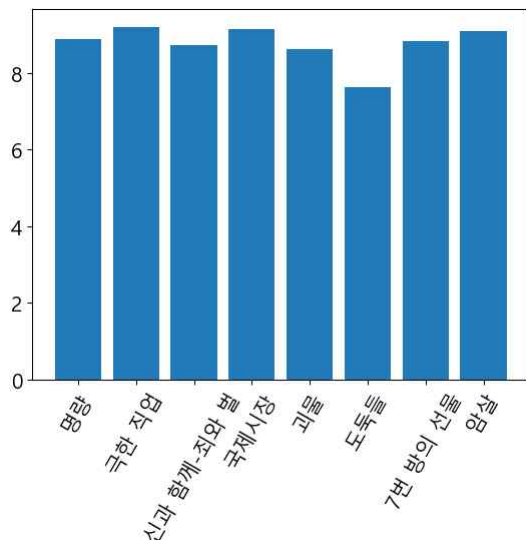
```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
import pandas as pd
data = {
    '영화' : ['명량', '극한 직업', '신과 함께-죄와 벌', '국제시장', '괴물', '도둑들', '7번 방의 선물', '암살'],
    '개봉 연도' : [2014, 2019, 2017, 2014, 2006, 2012, 2013, 2015],
    '관객 수' : [1761, 1626, 1441, 1426, 1301, 1298, 1281, 1270],
    '평점' : [8.88, 9.20, 8.73, 9.16, 8.62, 7.64, 8.83, 9.10]
}
df = pd.DataFrame(data)
df.head()
```

	영화	개봉 연도	관객 수	평점
0	명량	2014	1761	8.88
1	극한 직업	2019	1626	9.20
2	신과 함께-죄와 벌	2017	1441	8.73
3	국제시장	2014	1426	9.16
4	괴물	2006	1301	8.62

- 1) 영화 데이터를 활용하여 x 축은 영화, y 축은 평점인 막대그래프를 만드시오.

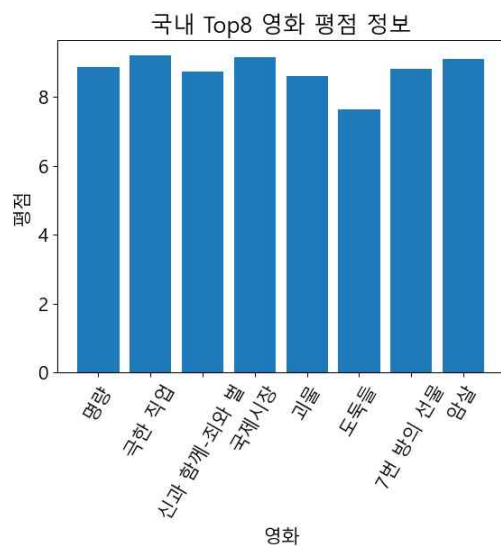
```
plt.bar(df['영화'], df['평점'])
plt.xticks(rotation=60)
plt.show()
```



2) 앞에서 만든 막대그래프에 제시된 세부 사항을 적용하시오.

- 제목: 국내 Top8 영화 평점 정보
- x축 label : 영화
- y축 label : 평점

```
plt.bar(df['영화'], df['평점'])
plt.title('국내 Top8 영화 평점 정보')
plt.xticks(rotation=60)
plt.xlabel('영화')
plt.ylabel('평점')
plt.show()
```

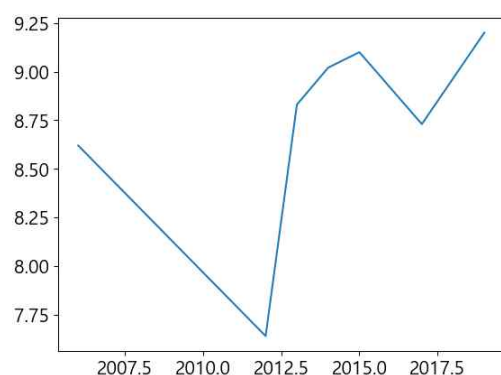


3) 개봉 연도별 평점 변화 추이를 꺾은선 그래프로 그리시오.

```
df_group = df.groupby('개봉 연도')[['관객 수', '평점']].mean()
df_group.head()
```

	영화	개봉 연도	관객 수	평점
0	명량	2014	1761	8.88
1	극한 직업	2019	1626	9.20
2	신과 함께-죄와 벌	2017	1441	8.73
3	국제시장	2014	1426	9.16
4	괴물	2006	1301	8.62

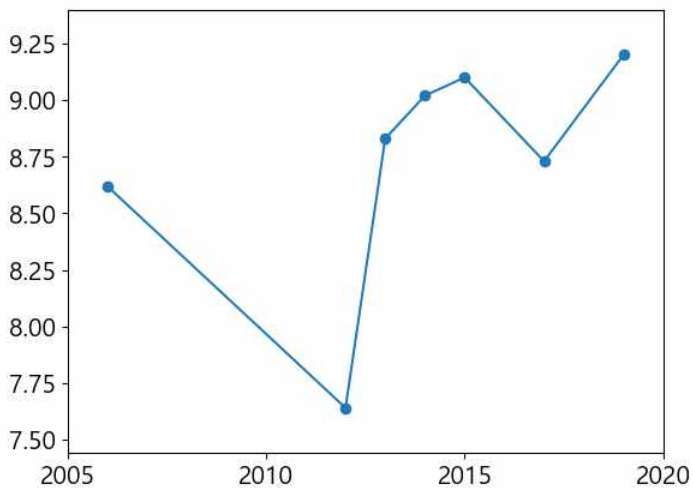
```
plt.plot(df_group.index, df_group['평점'])
```



4) 앞에서 만든 그래프에 제시된 세부 사항을 적용하시오.

- marker : 'o'
- x축 눈금 : 5년 단위(2005, 2010, 2015, 2020)
- y축 범위 : 최소 7, 최대 10

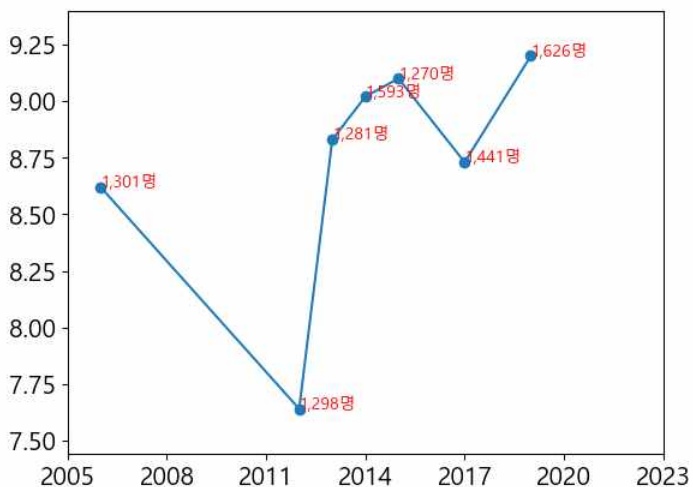
```
max = df['평점'].max()
min = df['평점'].min()
plt.plot(df_group.index, df_group['평점'], marker='o')
plt.xticks([i for i in range(2005, 2025, 5)])
plt.ylim(df['평점'].min()-0.2, df['평점'].max()+0.2)
```



5) 앞에서 만든 그래프에 plt.text() 함수를 이용해 관객 수를 출력한다.

```
max = df['평점'].max()
min = df['평점'].min()
plt.plot(df_group.index, df_group['평점'], marker='o')
plt.xticks([i for i in range(2005, 2025, 3)])
plt.ylim(df['평점'].min()-0.2, df['평점'].max()+0.2)

year = list(df_group.index)
rating = list(df_group['평점'])
for idx, people in enumerate(df_group['관객 수']):
    print(idx, year[idx], rating[idx], people)
    plt.text(year[idx], rating[idx], str(f'{int(people):,}명'), size=10)
```



6) 평점이 9점 이상인 영화의 비율을 확인할 수 있는 원그래프를 제시된 세부 사항을 적용하여 그리시오.

- label : 9점 이상 / 9점 미만
- 퍼센트 : 소수점 첫째 자리까지 표시
- 범례 : 그래프 우측에 표시

```
filt = df['평점'] >= 9.0
df[filt]
df[~filt]
values = [len(df[filt]), len(df[~filt])]
labels = ['9점 이상', '9점 미만']

wedgeprops={'width':0.8, 'edgecolor':'w', 'linewidth':5}
plt.pie(values, labels=labels, autopct='%1f%%', wedgeprops=wedgeprops)
plt.legend(loc=(1, 0.3))
```



7) 영화 데이터를 활용하여 x 축은 영화, y 축은 관객 수인 수평 막대그래프를 만드시오.

```
df.sort_values('개봉 연도', inplace=True)
```

```
plt.figure(figsize=(10, 5))
plt.barh(df['영화'], df['관객 수'], color='orange', alpha=0.9)
plt.xlim(0, 2200)
movies = list(df['영화'])
for idx, people in enumerate(df['관객 수']):
    plt.text(250, movies[idx], str(f'{people:,}명({year[idx]}년)'), size=12, color='yellow')
```



16. 인구 피라미드

2011년 8월 남녀, 나이별 전국 인구수와 2021년 8월 남녀, 나이별 전국 인구수를 시각화하여 10년 후 인구수를 비교한다.

- `pip install openpyxl`
- 새로운 파일 '16.인구피라미드.ipynb'를 생성한다.
- [구글]-[연령별 인구현황]을 검색하여 행정안전부에서 제공되는 아래 조건의 데이터(엑셀)를 이용한다.
- 아래 조건의 검색 결과를 엑셀 파일로 저장한다.

행정구역: 전국 시·군·구

등록구분: 전체

조회기간: 월간 2011년 08월 ~ 2011년 08월

※ 매월 말일 작성 / 공표일시: 매월 1일 12시 이후(공표일이 주말, 공휴일인 경우에는 다음 평일에 공표)

구분: ☐ 계 ☒ 남녀 구분

연령 구분 단위: 5세

만 연령구분: 0 100이상

- 남자 데이터를 데이터프레임(df_m)에 저장한다. (행정기관별 0세~100세까지 전국 시도의 남자 데이터만 저장한다.)

```
import pandas as pd
df_m=pd.read_excel('data/201108.xlsx', skiprows=3, index_col='행정기관', usecols='B, E:Y')
df_m.head(1)
```

	0~4세	5~9세	10~14세	15~19세	20~24세	25~29세	30~34세	35~39세	40~44세	45~49세	...
행정기관											
전국	1,195,951	1,233,465	1,663,534	1,890,498	1,667,705	1,844,651	2,048,038	2,178,901	2,335,456	2,165,327	...

1 rows × 21 columns

- 데이터프레임(df_m) index 0행(전국) 모든 열의 데이터에 콤마를 제거한 후 정수로 다시 저장한다.

```
df_m.iloc[0]=df_m.iloc[0].str.replace(',', '').astype(int)
df_m.head(1)
```

	0~4세	5~9세	10~14세	15~19세	20~24세	25~29세	30~34세	35~39세	40~44세	45~49세	...
행정기관											
전국	1195951	1233465	1663534	1890498	1667705	1844651	2048038	2178901	2335456	2165327	...

1 rows × 21 columns

- 여자 데이터를 데이터프레임(df_w)에 저장한다. (행정기관별 0세~100세까지 전국 시도의 여자 데이터만 저장한다.)

```
df_w=pd.read_excel('data/201108.xlsx', skiprows=3, index_col='행정기관', usecols='B, AB:AV')
df_w.head(1)
```

	0~4세.1	5~9세.1	10~14세.1	15~19세.1	20~24세.1	25~29세.1	30~34세.1	35~39세.1	40~44세.1	45~49세.1	...
행정기관											
전국	1,125,896	1,141,102	1,524,143	1,668,412	1,506,676	1,737,973	1,963,094	2,092,175	2,257,824	2,052,553	...

1 rows × 21 columns

- 데이터프레임(df_w) index 0행(전국) 모든 열의 데이터에 콤마를 제거한 후 정수로 다시 저장한다.

```
df_w.iloc[0]=df_w.iloc[0].str.replace(',','').astype(int)
df_w
```

	0~4세.1	5~9세.1	10~14세.1	15~19세.1	20~24세.1	25~29세.1	30~34세.1	35~39세.1	40~44세.1	45~49세.1	...
행정기관											
전국	1125896	1141102	1524143	1668412	1506676	1737973	1963094	2092175	2257824	2052553	...

1 rows × 21 columns

- 데이터프레임(df_m)의 모든 열 목록을 출력한다.

```
df_m.columns
```

```
Index(['0~4세', '5~9세', '10~14세', '15~19세', '20~24세', '25~29세', '30~34세', ...
      '70~74세', '75~79세', '80~84세', '85~89세', '90~94세', '95~99세', '100세 이상'], dtype='object')
```

- 데이터프레임(df_w)의 모든 열 목록을 출력한다.

```
df_w.columns
```

```
Index(['0~4세.1', '5~9세.1', '10~14세.1', '15~19세.1', '20~24세.1', '25~29세.1', ...
      '90~94세.1', '95~99세.1', '100세 이상.1'], dtype='object')
```

- 데이터프레임(df_w)의 열 이름들을 데이터프레임(df_m)의 열 이름들로 같게 변경한다.

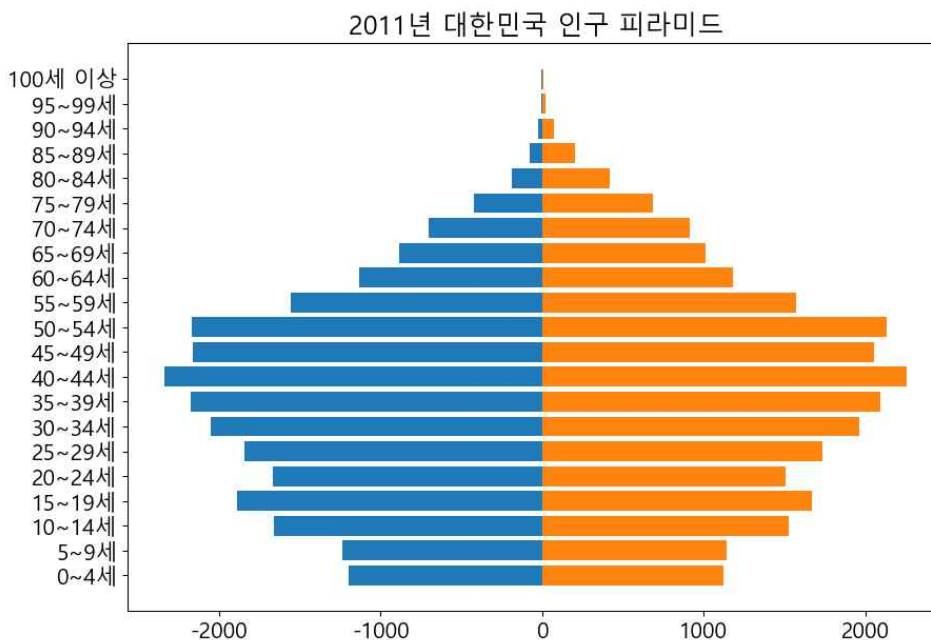
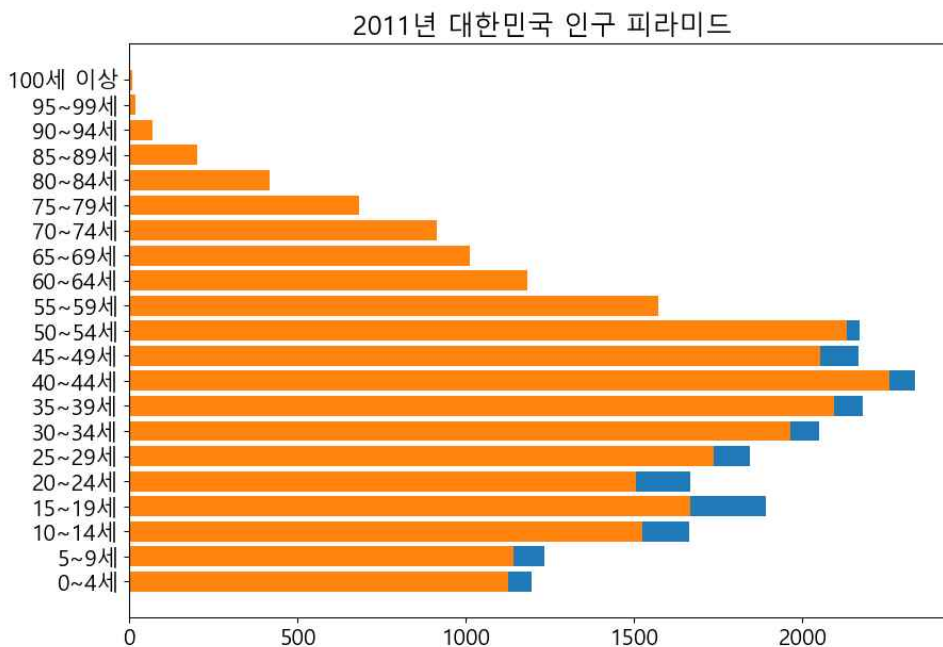
```
df_w.columns = df_m.columns
df_w.columns
```

```
Index(['0~4세', '5~9세', '10~14세', '15~19세', '20~24세', '25~29세', '30~34세', ...
      '35~39세', '40~44세', '45~49세', '50~54세', '55~59세', '60~64세', '65~69세', ...
      '70~74세', '75~79세', '80~84세', '85~89세', '90~94세', '95~99세', '100세 이상'], dtype='object')
```

- 맷플롯립(matplotlib) 라이브러리를 import하고 한글 폰트와 폰트크기를 지정한 후 인구수를 시각화한다. (단위:천명)

```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
plt.figure(figsize=(10,7))
plt.barh(df_m.columns, -df_m.iloc[0]//1000) #몹스(//)
plt.barh(df_w.columns, df_w.iloc[0]//1000)
plt.title('2011년 대한민국 인구 피라미드')
plt.savefig('data/2011_인구피라미드.png', dpi=100)
plt.show()
```



- 여성 데이터프레임(w_df)에 대한 시각화 그래프를 출력한다.

행정구역: 전국 시·군·구

등록구분: 전체

조회기간: 월간 연간 2021년 08월 ~ 2021년 08월

※ 매월 말일 작성 / 공표일시: 매월 1일 12시 이후(공표일이 주말, 공휴일인 경우에는 다음 평일에 공표)

구분: ☐ 계 ☒ 남·여 구분

연령 구분 단위: 5세

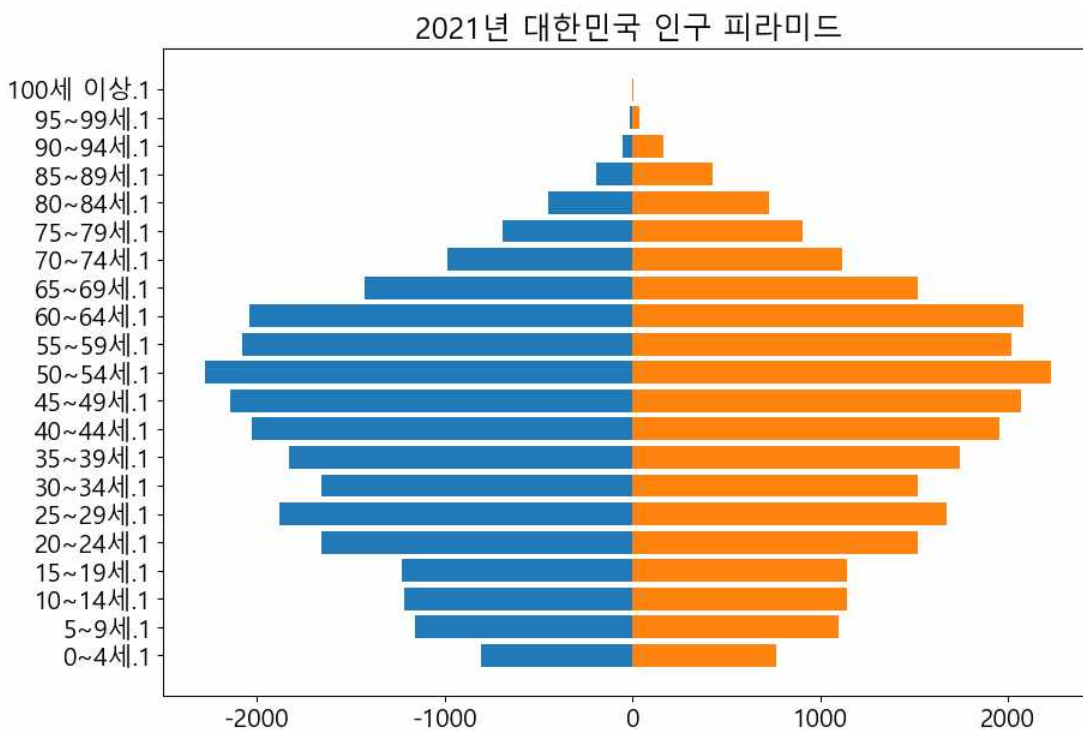
만 연령구분: 0 100이상

```
df_m=pd.read_excel('202108.xlsx', skiprows=3, index_col='행정기관', usecols='B, E:Y')
df_m.iloc[0]=df_m.iloc[0].str.replace(',','').astype(int)
```

```
df_w=pd.read_excel('202108.xlsx', skiprows=3, index_col='행정기관', usecols='B, AB:AV')
df_w.iloc[0]=df_w.iloc[0].str.replace(',','').astype(int)
```

```
df_m.columns = df_w.columns
```

```
plt.figure(figsize=(10, 7))
plt.barh(df_m.columns, -df_m.iloc[0] // 1000)
plt.title('2021년 대한민국 인구 피라미드')
plt.barh(df_w.columns, df_w.iloc[0] // 1000)
plt.savefig('2021_인구피라미드.png', dpi=100)
plt.show()
```



17. 출생아 수 및 합계출산율

- 새로운 파일 '17.출생아수및합계출산율.ipynb'를 생성한다.
- [구글]-[출생아 수 합계출산율]-[합계출산율]을 검색하여 에서 제공되는 데이터(엑셀)를 이용한다.

통계표
합계 출산율
기간선택
연도
2012
~
2021
조회

행렬전환
조화설정
데이터찾기
heatMap
증감비교
전체 증감비교
초기화
통계표다운로드
틀고정
출처영역고정

- 2행을 skip하고 추출하고 0~2번째 행을 추출해 0열을 index 열로 지정해 데이터프레임(df)에 저장한다.

```
import pandas as pd
df=pd.read_excel('data/합계출산율.xlsx', skiprows=2, nrows=2, index_col=0)
df
```

	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021
출생아 수	484.600	436.500	435.400	438.400	406.200	357.800	326.800	302.700	272.300	260.600
합계 출산율	1.297	1.187	1.205	1.239	1.172	1.052	0.977	0.918	0.837	0.808

- 데이터프레임(df)의 index 이름 공백에 들어간 이상한 문자를 제거해 새로운 index 이름으로 적용한다.

df.index

Index(['출생아 수', '합계 출산율'], dtype='object')

df.index.values

array(['출생아\xa0수', '합계\xa0출산율'], dtype=object)

df.rename(index={'출생아\xa0수':'출생아 수', '합계\xa0출산율':'합계 출산율'}, inplace=True)

df.index.value

array(['출생아 수', '합계 출산율'], dtype=object)

- 데이터프레임(df)에서 '출생아 수'를 출력하려면 아래 두 가지(loc, iloc) 방법을 이용할 수 있다.

df.loc[['출생아 수']]

	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021
출생아 수	484.6	436.5	435.4	438.4	406.2	357.8	326.8	302.7	272.3	260.6

df.iloc[[0]]

	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021
출생아 수	484.6	436.5	435.4	438.4	406.2	357.8	326.8	302.7	272.3	260.6

- 데이터프레임(df)에서 T 속성을 이용하여 행과 열을 바꾼 후 데이터프레임(df)에 새로 저장한다.

df

	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021
출생아 수	484.600	436.500	435.400	438.400	406.200	357.800	326.800	302.700	272.300	260.600
합계 출산율	1.297	1.187	1.205	1.239	1.172	1.052	0.977	0.918	0.837	0.808

df = df.T

df

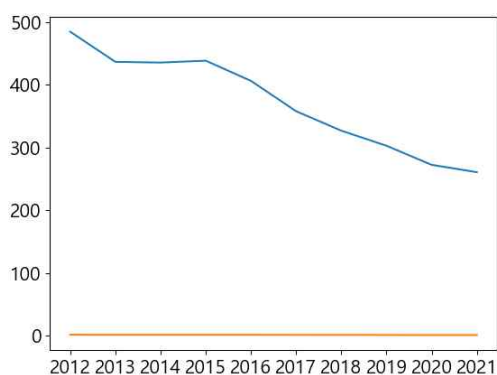
	출생아 수	합계 출산율
2012	484.6	1.297
2013	436.5	1.187
2014	435.4	1.205
2015	438.4	1.239
2016	406.2	1.172
2017	357.8	1.052
2018	326.8	0.977
2019	302.7	0.918
2020	272.3	0.837
2021	260.6	0.808

- 데이터 시각화를 위하여 맷플롯립(matplotlib) 라이브러리를 import하고 한글 폰트와 폰트크기를 설정한다.

```
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 15
matplotlib.rcParams['axes.unicode_minus'] = False
```

- X축을 index, Y축은 '출생아 수', '합계 출산율'로 선 그래프로 시각화 작업을 한다.

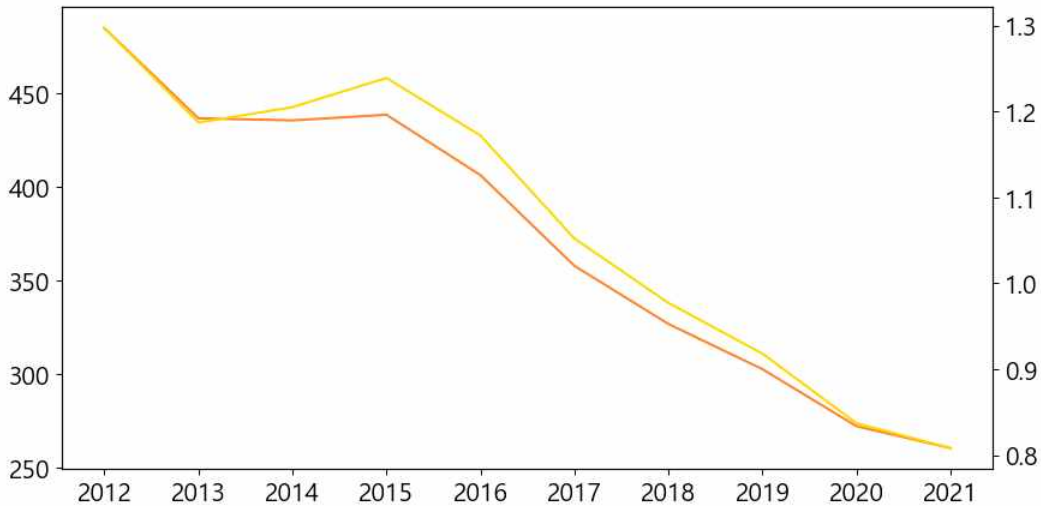
```
plt.plot(df.index, df['출생아 수'])
plt.plot(df.index, df['합계 출산율'])
plt.show()
```



- '합계 출산율'의 값이 너무 작으므로 X축은 공유하고 '합계 출산율'의 Y축은 오른쪽으로 지정한다. `subplots()`는 하나의 그림(`figure`) 안에 여러 개의 그래프(`subplot`)를 배치할 때 사용하는 함수이다.

```
fig, ax1 = plt.subplots(figsize=(10, 5))
ax1.plot(df.index, df['출생아 수'], color='ff812d')

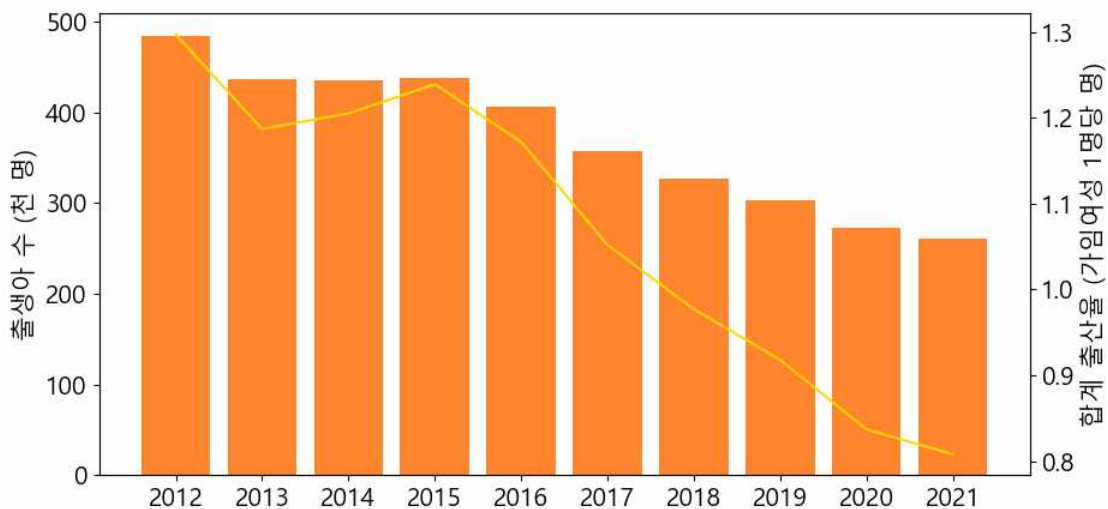
ax2 = ax1.twinx()
ax2.plot(df.index, df['합계 출산율'], color='ffd100')
```



- '출생아 수'를 막대그래프로 변경하고 Y축의 레이블을 '출생아 수 (천 명)'로 변경한다.

```
fig, ax1 = plt.subplots(figsize=(10, 5))
ax1.bar(df.index, df['출생아 수'], color='ff812d')
ax1.set_ylabel('출생아 수 (천 명)')

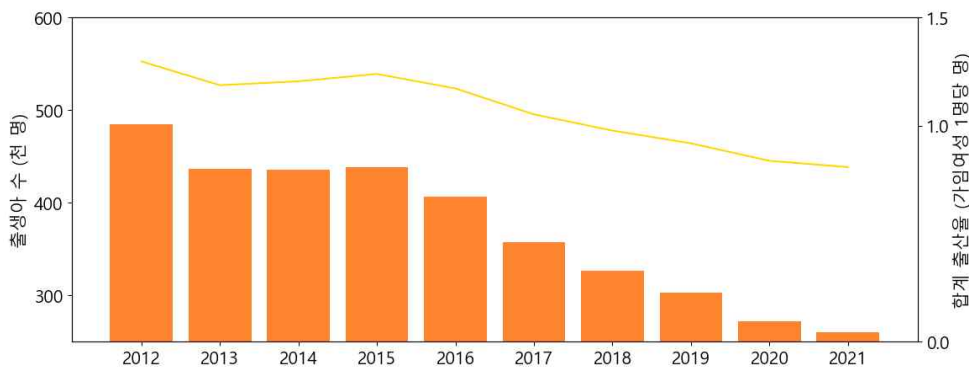
ax2 = ax1.twinx()
ax2.plot(df.index, df['합계 출산율'], color='ffd100')
ax2.set_ylabel('합계 출산율 (가임여성 1명당 명)')
```



- 그래프의 가로, 세로 비율을 변경하고 두 그래프의 Y축 범위, 눈금의 간격을 아래와 같이 변경한다.

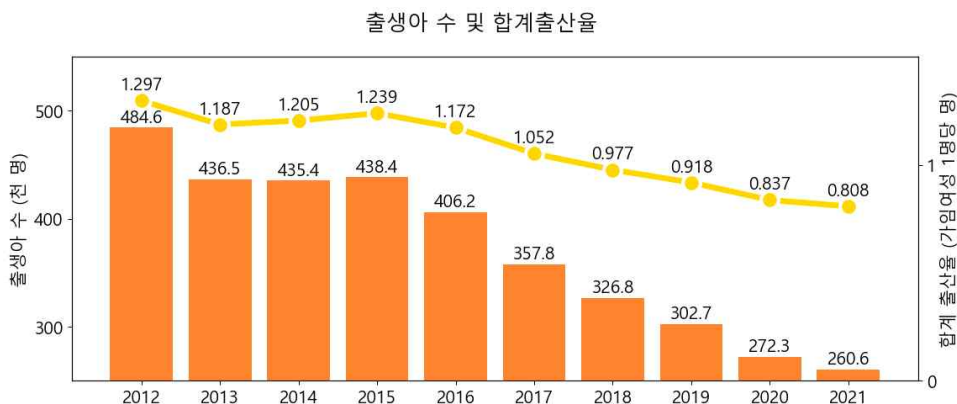
```
fig, ax1 = plt.subplots(figsize=(13, 5))
ax1.bar(df.index, df['출생아 수'], color='#ff812d')
ax1.set_ylabel('출생아 수 (천 명)')
ax1.set_ylim(250, 550)
ax1.set_yticks([300, 400, 500, 600])

ax2 = ax1.twinx()
ax2.plot(df.index, df['합계 출산율'], color='#ffd100')
ax2.set_ylabel('합계 출산율 (가임여성 1명당 명)')
ax2.set_ylim(0, 1.5)
ax2.set_yticks([0, 1, 1.5])
```



```
fig, ax1 = plt.subplots(figsize=(13, 5))
fig.suptitle('출생아 수 및 합계출산율')
ax1.bar(df.index, df['출생아 수'], color='#ff812d')
ax1.set_ylabel('출생아 수 (천 명)')
ax1.set_ylim(250, 550)
ax1.set_yticks([300, 400, 500])
for idx, val in enumerate(df['출생아 수']):
    ax1.text(idx, val+5, val, ha='center')

ax2 = ax1.twinx()
ax2.plot(df.index, df['합계 출산율'], color='#ffd100', marker='o', ms=15, lw=5, mec='w', mew=3)
ax2.set_ylabel('합계 출산율 (가임여성 1명당 명)')
ax2.set_ylim(0, 1.5)
ax2.set_yticks([0, 1])
for idx, val in enumerate(df['합계 출산율']):
    ax2.text(idx, val+0.05, val, ha='center')
```



18. 워드클라우드

워드클라우드(wordCloud)는 각 단어를 발생 빈도에 비례하는 크기로 나타낸다. 소셜미디어의 게시물, 뉴스기사, 고객리뷰와 같은 대량의 텍스트 데이터를 탐색하여 시각화하는 데 많이 사용된다. 워드클라우드는 텍스트 데이터에서 자주 나타나거나 중요한 주제를 빠르고 직관적으로 확인하도록 도와준다.

- 워드클라우드를 생성하는 과정은 다음과 같다.
 - 1) 텍스트 데이터를 사전 처리하여 영어의 the, end, of와 같은 일반 적인 단어를 제거한다.
 - 2) 남은 단어들의 빈도수를 계산합니다.
 - 3) 빈도수에 비례하는 크기로 단어를 표시하여 구름 모양으로 나타낸다.
 - 4) 워드클라우드의 마스크, 단어 크기와 글꼴, 색을 설정하여 시각화 효과를 높인다.

- 새로운 파일 '18.워드클라우드.ipynb'를 생성한다.

- WordCloud, matplotlib 라이브러리를 import 한다.

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
```

- 영어 노래가사를 텍스트파일로 저장 후 text 변수에 저장한다.

```
with open('노래(가사).txt', 'r') as f:
    text = f.read()
f.close()
print(text)

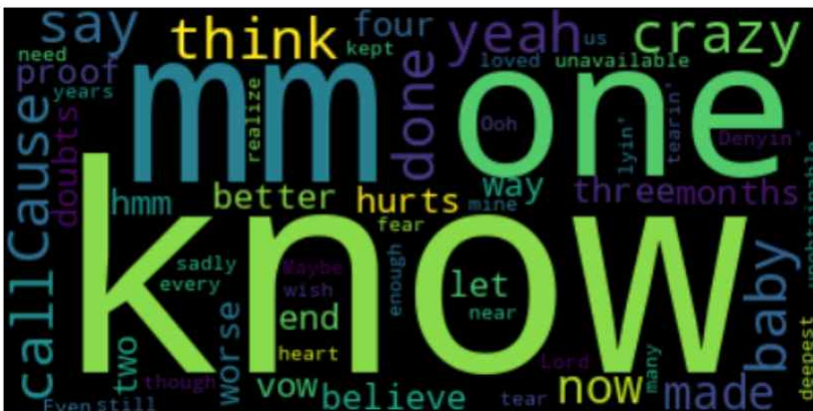
...
And I know, and I know, and I know, and I know, and I know, and I know
I know I'm not the only one
```

- WordCloud 객체를 생성한다.

```
wordcloud = WordCloud().generate(text)
```

- 워드클라우드로 시각화한다. (interpolation='bilinear'은 이미지를 부드럽게 출력한다.)

```
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.show()
```



- 한국어 자연어 처리 패키지(Okt)와 단어수를 구하는 패키지(Counter)를 import 한다.

```
from konlpy.tag import Okt
from collections import Counter
```

```
with open('애국가(가사).txt', 'r', encoding='utf-8') as f:
    text = f.read()
f.close()
```

- Okt() 함수로 명사만 추출해 글자길이가 1보다 큰 명사들만 words 변수에 저장한다.

```
okt = Okt()
nouns = okt.nouns(text)
words = [n for n in nouns if len(n) > 1]
```

Counter() 함수로 words를 처리하여 단어가 키이고 빈도수가 값인 딕셔너리 c를 생성한다.

```
c = Counter(words)
```

- 워드클라우드 객체를 생성한다.

```
wc = WordCloud(font_path="/usr/share/fonts/truetype/nanum/NanumBarunGothic.ttf",
               width=400, height=400,
               scale=2.0, max_font_size=250)
```

- generate_from_frequencies() 함수로 단어의 빈도수가 많은 것을 크게 표시하는 워드클라우드를 생성한다.

```
gen = wc.generate_from_frequencies(c)
```

```
plt.figure()
plt.imshow(gen)
plt.axis("off")
plt.show()
wc.to_file('애국가.png')
```



19. 타이타닉(EDA:Exploratory Data Analysis)

프로그램 언어를 통한 데이터 분석을 할 때 아래와 같은 이유로 타이타닉 탑승자 데이터를 많이 사용된다.

- 1) 테스트를 위해 만든 임의의 데이터가 아닌 실제 데이터이다.
- 2) 다양한 분석 기법을 사용해서 정확도를 올릴 수 있을 만한 적당한 변수가 있고 데이터 포맷도 꽤 깔끔한 편이다.
- 3) 너무 많지도 적지도 않은 데이터이다.

- 새로운 파일 '19.타이타닉.ipynb'를 생성한다.
- 아래 kaggle 사이트에서 데이터셋을 다운로드 받는다.

```
https://www.kaggle.com/c/titanic/data
```

- train 데이터셋을 읽어 train 데이터 프레임 변수에 읽어 들인다.

```
import pandas as pd
train = pd.read_csv('data/train.csv')
```

- train 데이터를 확인해 본다.

칼럼 이름	칼럼 설명
Survived	생존유무(0 = 사망, 1 = 생존)
Name	탑승객 성명
Pclass	티켓 클래스 (1 = 1st, 2 = 2nd, 3 = 3rd)
Sex	성별
Age	나이(세)
SibSp	함께 탑승한 형제자매, 배우자 수 총합
Parch	함께 탑승한 부모, 자녀 수 총합
Embarked	탑승 항구
Fare	탑승 요금
Ticket	티켓 넘버
Cabin	객실 넘버

train.head()

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

- 891개의 행과 12 칼럼이 존재한다.

train.shape

(891, 12)

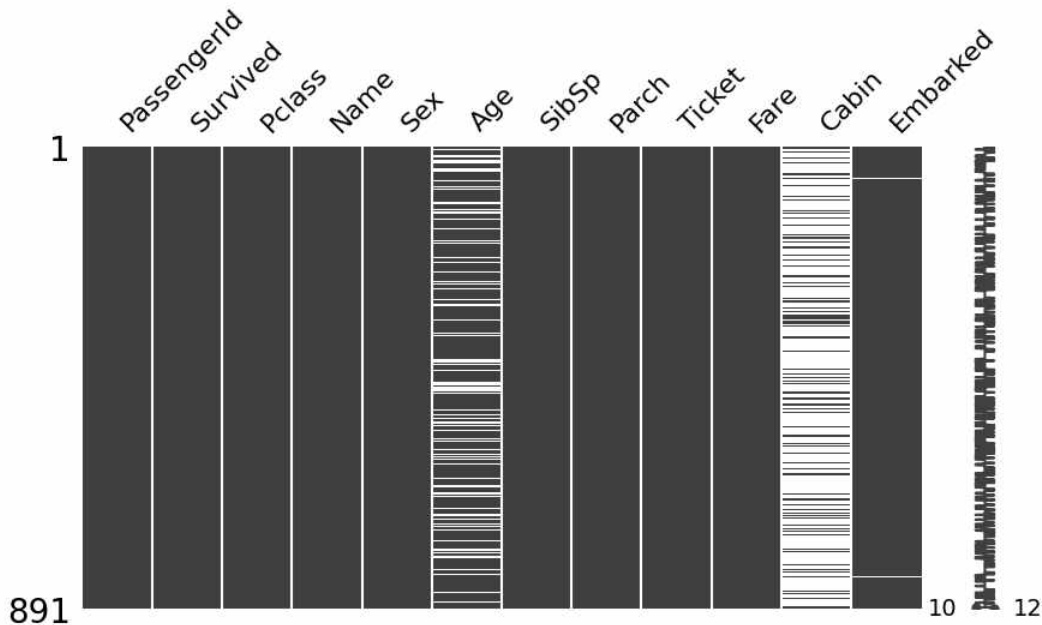
- train data의 결측치를 확인한다. train data에는 Age, Cabin, Embarked 칼럼에 결측치가 존재 한다

```
train.isnull().sum()
```

```
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age           177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin         687
Embarked       2
dtype: int64
```

- train data의 결측치를 missingno를 이용하면 시각화해볼 수 있다

```
import missingno as msno
msno.matrix(train, figsize=(10, 5))
```



- 사망, 생존 비율을 원 그래프로 출력한다.

```
survived = train.value_counts('Survived')
survived
```

```
Survived
0    549
1    342
Name: count, dtype: int64
```

```
import matplotlib.pyplot as plt
plt.rc('font', family='Malgun Gothic')
plt.rc('font', size=10)
plt.rc('axes', unicode_minus=False)
```

- 사망 비율은 61.6%, 생존 비율은 38.4%로 사망한 탑승객의 수가 더 많은 것이 확인 된다.

```
plt.figure(figsize=(8, 5))
lables = ['사망', '생존']
plt.pie(survived.values, labels=lables, explode=[0, 0.08], shadow=True, autopct='%0.1f%%',
        textprops={'color':'cyan', 'fontsize':'15'})
plt.title('생존, 사망 비율', size=20)
plt.show()
```



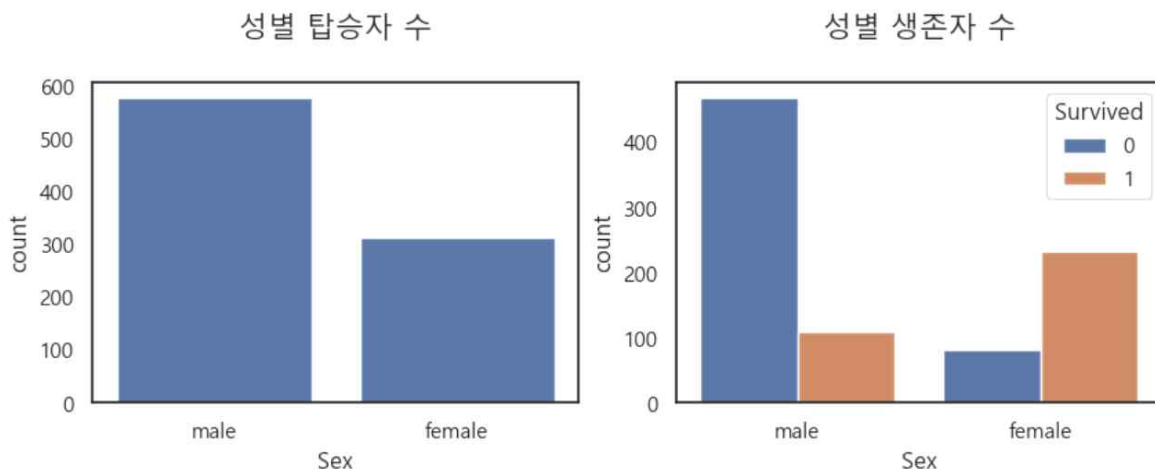
- seaborn 라이브러리를 import 한다.

Seaborn은 파이썬 데이터 시각화 라이브러리로, Matplotlib을 기반으로 하며 통계적인 그래프를 쉽게 만들 수 있다.

```
import seaborn as sns
sns.set_theme(font='Malgun Gothic', style='white') #syle=white, whitegrid, dark, darkgrid, ticks
```

- seaborn 라이브러리 countplot 함수는 각 카테고리별 데이터 개수를 막대 그래프 형태로 시각화하는 데 사용된다.
- 성별 탑승자수와 성별 생존자 수를 출력 결과 남성 탑승자 수가 더 많지만 여성 생존자 수가 더 많다.

```
fig, axes = plt.subplots(1, 2, figsize=(12,5))
axes[0].set_title('성별 탑승자 수\n', size=15)
sns.countplot(x='Sex', data=train, ax=axes[0])
axes[1].set_title('성별 생존자 수\n', size=15)
sns.countplot(x='Sex', hue='Survived', data=train, ax=axes[1])
plt.show()
```



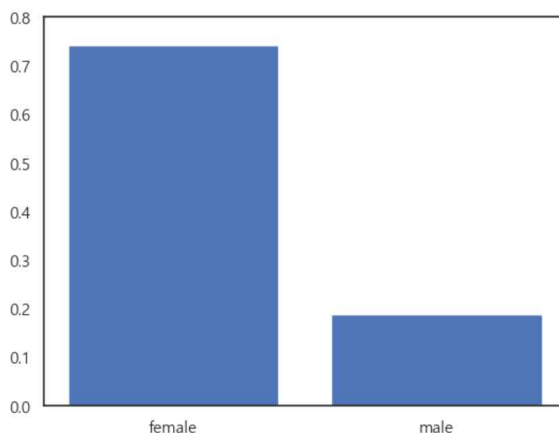
- 성별 생존율을 출력한다.

```
sex = train.groupby('Sex').size().to_frame('전체인원')
filt = train['Survived'] == 1
sex_survived = train[filt].groupby('Sex').size().to_frame('생존인원')
merge = sex.merge(sex_survived, on='Sex')
merge['생존율'] = merge['생존인원']/merge['전체인원']
```

	전체인원	생존인원	생존율
Sex			
female	314	233	0.742038
male	577	109	0.188908

- 여성의 생존율은 약 74.2%, 남성의 생존율은 약 18.9%로 여성의 생존율이 압도적으로 높은 것이 확인 된다. 남성보다 여성을 먼저 구조한 것으로 보인다.

```
plt.bar(merge.index, merge['생존율'])
plt.ylim(0, 0.8)
plt.show()
```



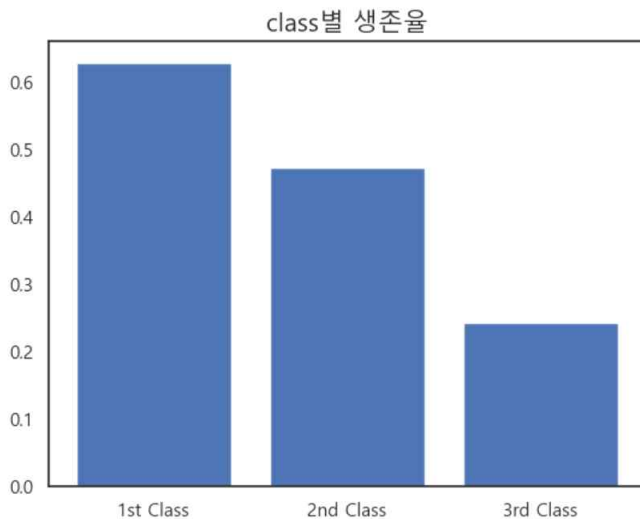
- 티켓 클래스별로 어떤 생존율을 보이고 있는지 확인해 본다. 1등석의 생존율은 약 63%, 2등석의 생존율은 약 47.3%, 3등석의 생존율은 약 24.2%이다. 3등석 탑승객들이 많이 사망한 것으로 보인다.

```
pclass = train.groupby('Pclass').size().to_frame('전체인원')
filt = train['Survived'] == 1
pclass_survived = train[filt].groupby('Pclass').size().to_frame('생존인원')
merge = pclass.merge(pclass_survived, on='Pclass')
merge['생존율'] = merge['생존인원'] / merge['전체인원']
merge
```

	전체인원	생존인원	생존율
Pclass			
1	216	136	0.629630
2	184	87	0.472826
3	491	119	0.242363

- 클래스별 생존율을 시각해 보면 1등석이 가장 높은 생존율을 보이고 3등석이 가장 낮은 생존율을 보인다. 티켓 클래스는 생존 여부에 영향을 끼치는 것으로 보인다.

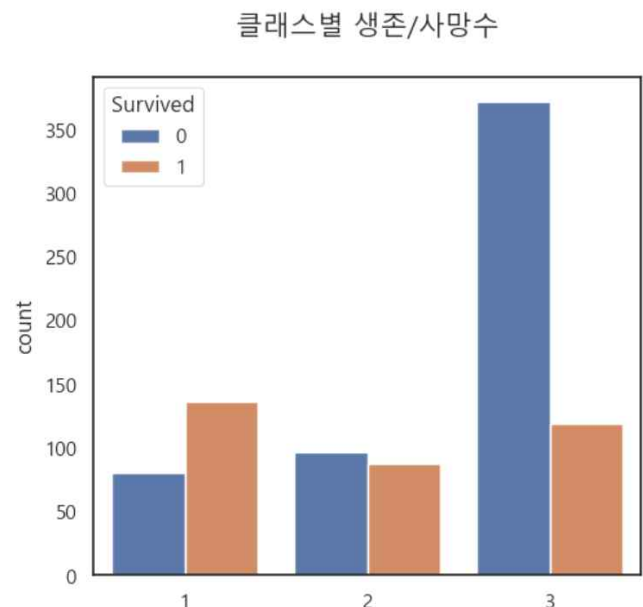
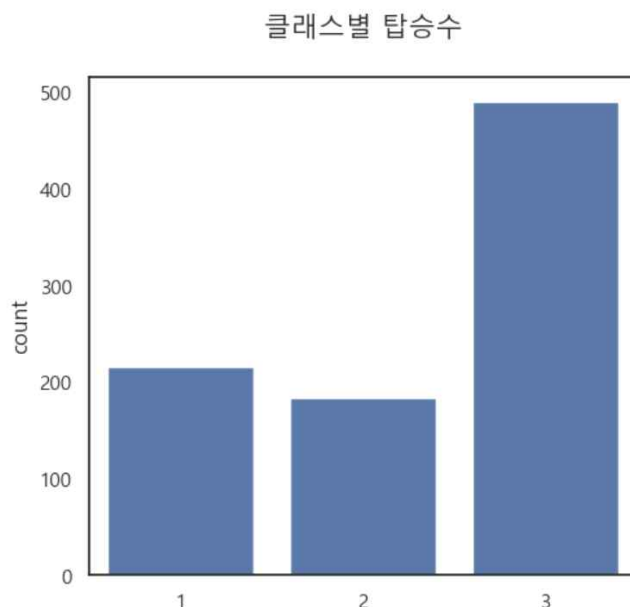
```
merge.index=['1st Class', '2nd Class', '3rd Class']
plt.bar(merge.index, merge['생존율'])
plt.title('class별 생존율', size=15)
plt.show()
```



- 티켓 클래스별로 탑승자수와 클래스별 생존/사망자 수를 시각화 하면 3등석 탑승객이 가장 많고, 3등석 탑승객의 사망 인원수 또한 가장 많다. 상대적으로 1등석 탑승객들은 많이 살아남은 것으로 보인다.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 3))
axes[0].set_title('클래스별 탑승수\n', size=15)
sns.countplot(x='Pclass', data=train, ax=axes[0])

axes[1].set_title('클래스별 생존/사망수\n', size=15)
sns.countplot(x='Pclass', hue='Survived', data=train, ax=axes[1])
plt.show()
```



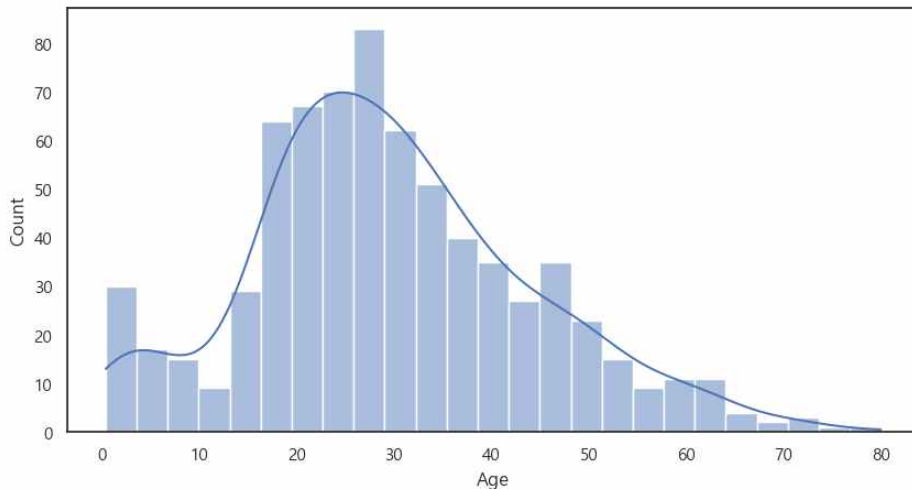
- 나이의 분포를 살펴보면 탑승객들의 평균 나이는 30세이다.

```
train['Age'].describe()
```

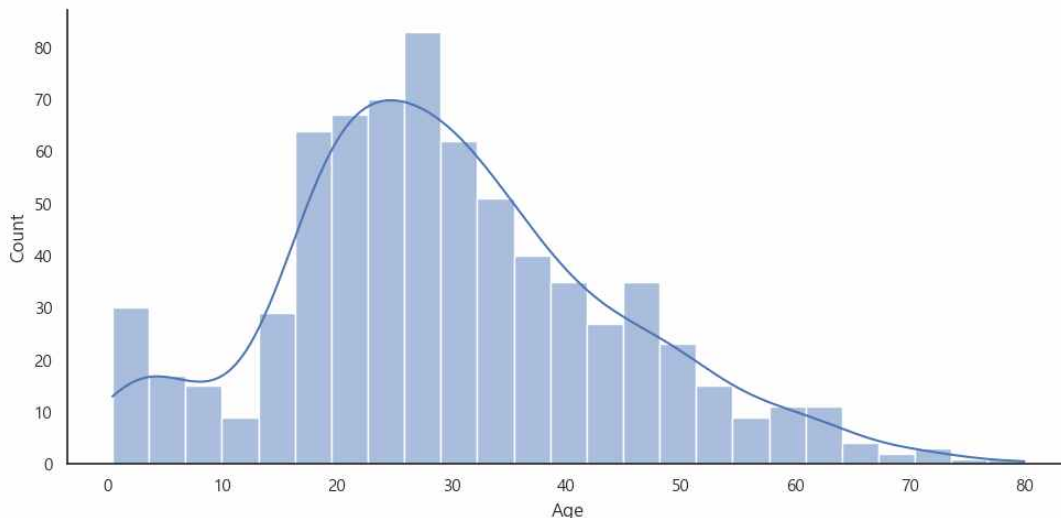
```
count      714.000000
mean       29.699118
std        14.526497
min         0.420000
25%        20.125000
50%        28.000000
75%        38.000000
max        80.000000
Name: Age, dtype: float64
```

- Seaborn의 `displot()` 및 `histplot()` 함수는 분포 플롯과 히스토그램을 만드는데 필수적인 도구이다. 히스토그램은 데이터의 분포를 시각적으로 나타내므로 데이터가 어떻게 분포되어 있는지 쉽게 파악할 수 있다. 아래 그래프는 탑승객들의 나이가 주로 20~40세에 분포되어 있음을 보여준다. KDE(커널밀도추정, kernel density estimate)

```
plt.figure(figsize=(10, 5))
sns.histplot(data=train, x='Age', bins=25, kde=True)
plt.show()
```

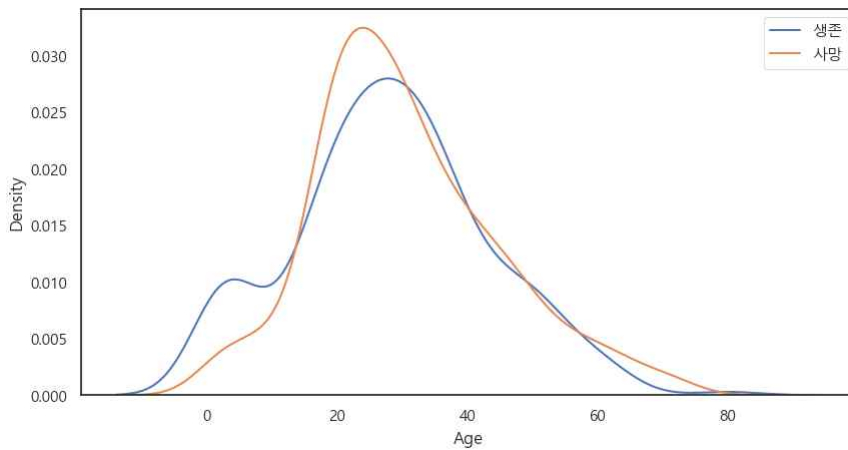


```
sns.displot(data=train, x='Age', bins=25, height=5, aspect=2, kde=True)
plt.show()
```



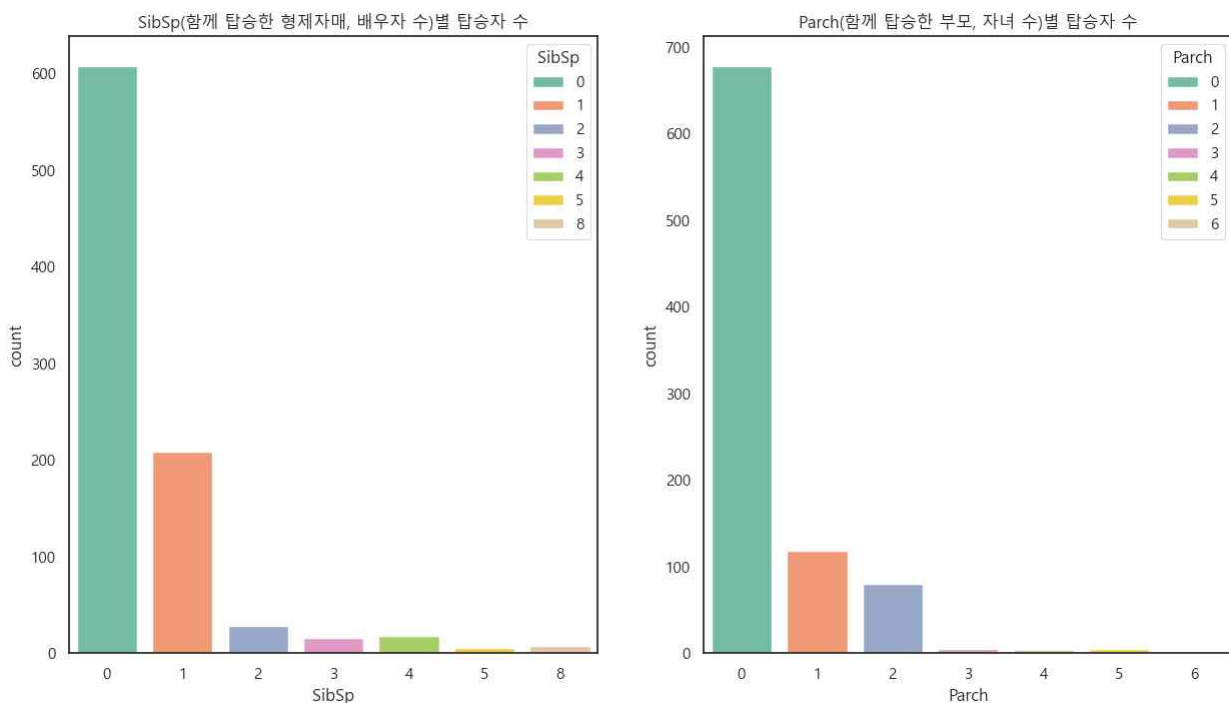
- `kdeplot`은 `Seaborn` 라이브러리의 함수로 커널 밀도 추정(KDE)을 통해 데이터 분포를 시각화한다. 나이가 생존여부에 영향을 끼치는지 확인하기 위해 생존/사망 탑승객을 나누어 비교해봤다. 비슷하긴 하지만 어린 나이의 탑승객들의 생존율이 비교적 높고 20-30세의 생존율은 비교적 낮은 것으로 보인다.

```
survived = train[train['Survived'] == 1]
dead = train[train['Survived'] == 0]
plt.figure(figsize=(10, 3))
sns.kdeplot(survived['Age'], label='생존')
sns.kdeplot(dead['Age'], label='사망')
plt.legend()
```



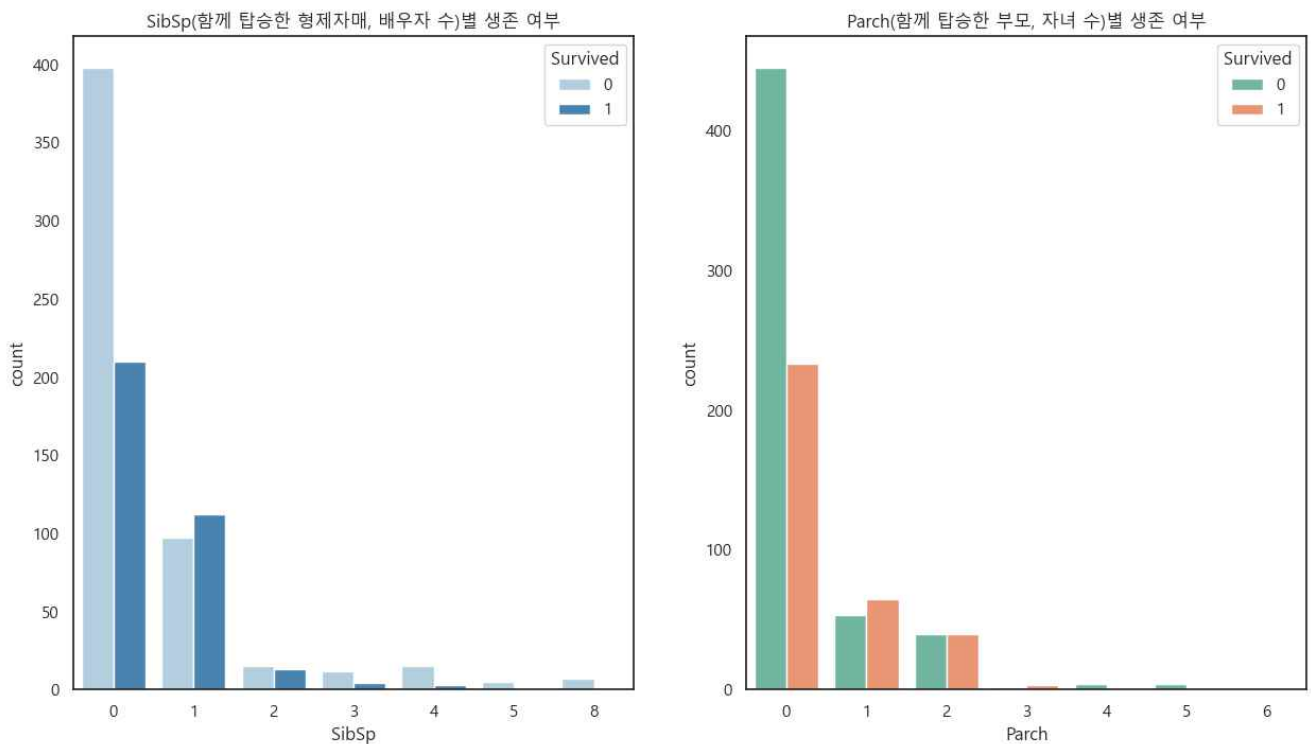
- 함께 탑승한 가족의 수별 탑승자 수를 시각화한다. 혼자 탑승한 탑승객이 가장 많다.

```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
axes[0].set_title('SibSp(함께 탑승한 형제자매, 배우자 수)별 탑승자 수')
sns.countplot(x='SibSp', data=train, ax=axes[0], hue='SibSp', palette='Set2')
axes[1].set_title('Parch(함께 탑승한 부모, 자녀 수)별 탑승자 수')
sns.countplot(x='Parch', data=train, ax=axes[1], hue='Parch', palette='Set2')
```



- 함께 탑승한 가족의 수별 생존자와 사망자수를 시각화 한다. 생존 여부를 확인해보면 혼자 탑승한 사람들이 많이 사망한 것을 확인할 수 있다.

```
fig, axes = plt.subplots(1, 2, figsize=(15, 8))
axes[0].set_title('SibSp(함께 탑승한 형제자매, 배우자 수)별 생존 여부')
sns.countplot(x='SibSp', hue='Survived', data=train, ax=axes[0], palette='Blues')
axes[1].set_title('Parch(함께 탑승한 부모, 자녀 수)별 생존 여부')
sns.countplot(x='Parch', hue='Survived', data=train, ax=axes[1], palette='Set2')
plt.show()
```



- SibSp(함께 탑승한 형제자매, 배우자 수)별 생존율을 구한다. 혼자 탑승한 사람보다 1명, 2명 SibSp와 탑승한 탑승객들의 사망률이 가장 높다.

```
sibsp = train.groupby('SibSp').size().to_frame('전체인원')
filt = train['Survived'] == 1
sibsp_survived = train[filt].groupby('SibSp').size().to_frame('생존인원')
merge = sibsp.merge(sibsp_survived, on='SibSp')
merge['생존율'] = merge['생존인원']/merge['전체인원']
```

	전체인원	생존인원	생존율
SibSp			
0	608	210	0.345395
1	209	112	0.535885
2	28	13	0.464286
3	16	4	0.250000
4	18	3	0.166667

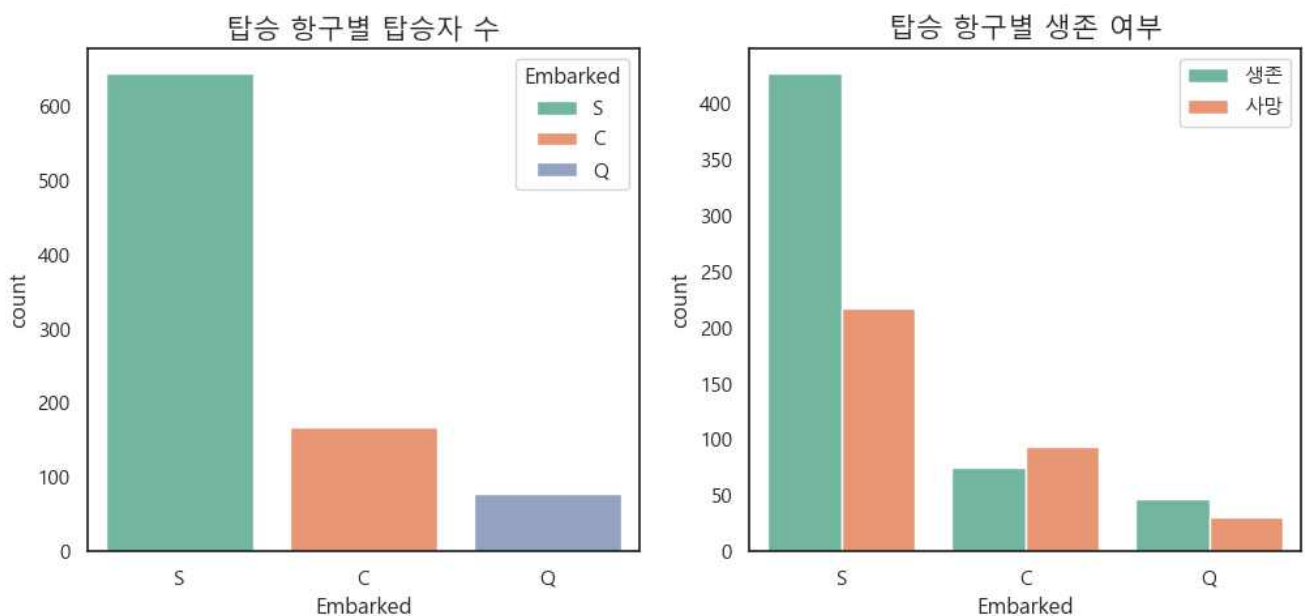
- Parch(함께 탑승한 부모, 자녀 수)별 생존율을 구한다. 생존율을 구한 결과 부모, 자녀 없이 혼자 탑승한 사람의 사망률이 가장 높다.

```
parch = train.value_counts('Parch').to_frame('전체인원')
filt = train['Survived'] == 1
parch_survived = train[filt].value_counts('Parch').to_frame('생존인원')
merge = parch.merge(parch_survived, on='Parch')
merge['생존율'] = merge['생존인원'] / merge['전체인원']
merge
```

	전체인원	생존인원	생존율
Parch			
0	678	233	0.343658
1	118	65	0.550847
2	80	40	0.500000
3	5	3	0.600000
5	5	1	0.200000

- 탑승 항구(Embarked)별 탑승자 수와 생존 여부를 시각화한다. S항구 탑승객이 가장 많았으며 사망한 인원 역시 S항구 탑승객이 가장 많다.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].set_title('탑승 항구별 탑승자 수', size=15)
sns.countplot(x='Embarked', data=train, ax=axes[0], hue='Embarked', palette='Set2')
axes[1].set_title('탑승 항구별 생존 여부', size=15)
sns.countplot(x='Embarked', hue='Survived', data=train, ax=axes[1], palette='Set2')
axes[1].legend(labels=['생존', '사망'])
plt.show()
```



- 탑승 항구별 사망 비율을 확인한다. S항구 탑승객의 사망률이 가장 높다

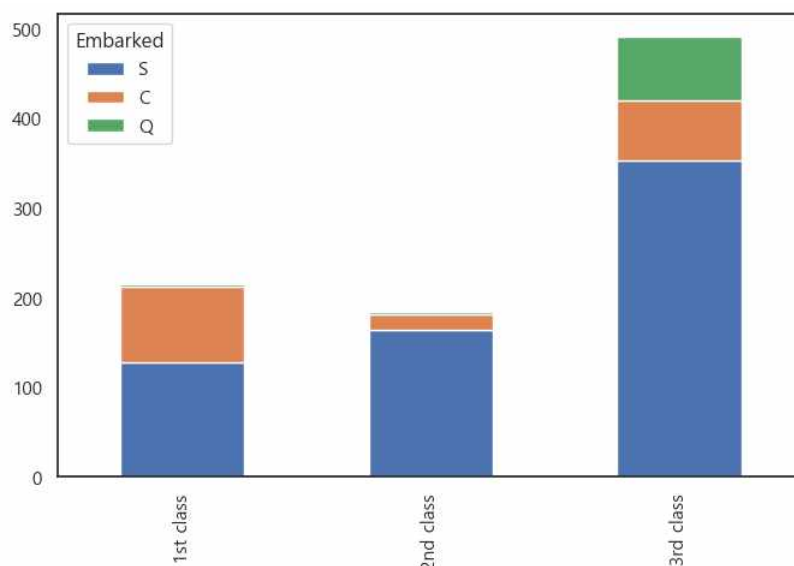
```
embarked = train.groupby('Embarked').size().to_frame('전체인원')
filt = train['Survived'] == 1
embarked_survived = train[filt].groupby('Embarked').size().to_frame('생존인원')
merge = embarked.merge(embarked_survived, on='Embarked')
merge['생존율'] = merge['생존인원'] / merge['전체인원']
merge
```

	전체인원	생존인원	생존율
Embarked			
C	168	93	0.553571
Q	77	30	0.389610
S	644	217	0.336957

- 항구별 탑승객들의 티켓 클래스를 시각화하면 3등석에 S항구 탑승객들이 많은 것을 확인할 수 있다. 위에서 3 등석의 사망률이 가장 높았듯이 3등석에 S항구 탑승객들이 많이 때문에 S항구 탑승객들의 사망률이 높은 것으로 보인다.

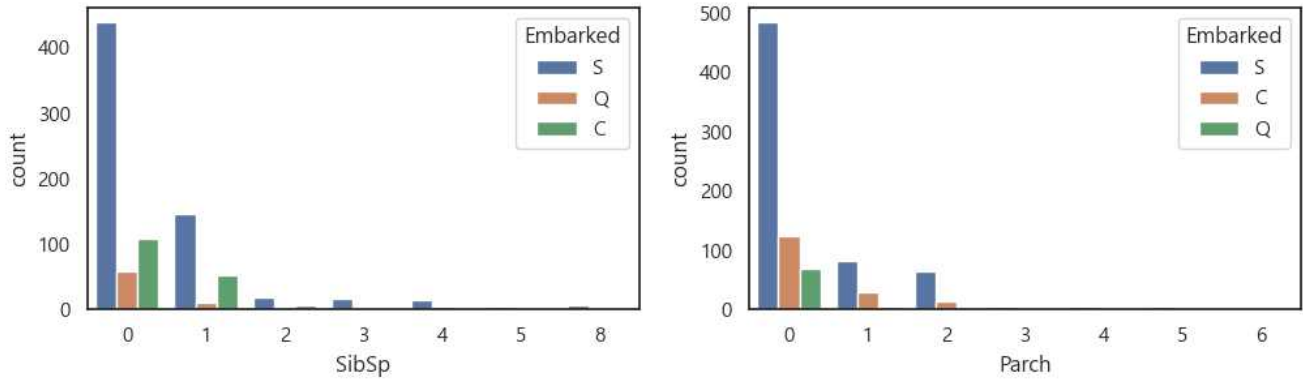
```
pc1 = train[train['Pclass'] == 1].value_counts('Embarked')
pc2 = train[train['Pclass'] == 2].value_counts('Embarked')
pc3 = train[train['Pclass'] == 3].value_counts('Embarked')
pc = pd.DataFrame([pc1, pc2, pc3])
pc.index = ['1st class', '2nd class', '3rd class']
```

```
# plt.bar(pc.index, pc['S'])
# plt.bar(pc.index, pc['C'], bottom=pc['S'])
# plt.bar(pc.index, pc['Q'], bottom=pc['S'] + pc['C'])
fig, ax = plt.subplots(figsize=(8, 5))
pc.plot(kind='bar', stacked=True, ax=ax)
plt.show()
```



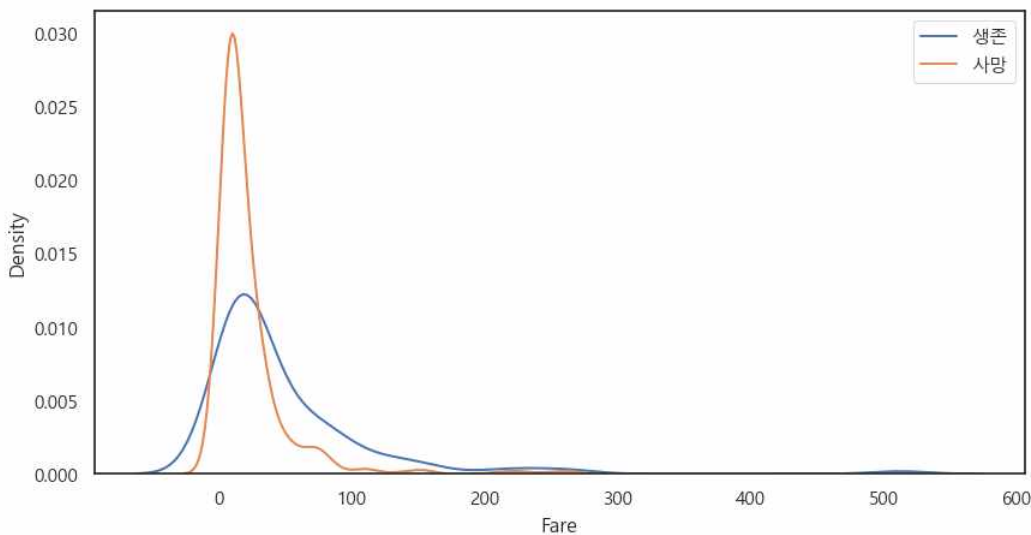
- 항구별 탑승객들의 동승자수를 시각화한다. 혼자 탑승한 탑승객의 수는 S항구 탑승객이 압도적으로 많았다.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 3))
sns.countplot(data=train, x='SibSp', hue='Embarked', ax=axes[0])
sns.countplot(data=train, x='Parch', hue='Embarked', ax=axes[1])
plt.show()
```



- 탑승 금액의 분포를 확인해보면 탑승 금액이 적은 쪽은 사망률이 높은 것이 확인된다.

```
plt.figure(figsize=(10, 6))
filt = train['Survived'] == 1
sns.kdeplot(train[filt]['Fare'], label='생존')
sns.kdeplot(train[~filt]['Fare'], label='사망')
plt.xlim(0, train['Fare'].max())
plt.legend()
plt.show()
```



- Cabin은 결측치가 많고 PassengerID, Name, Ticket은 생존 여부와 연관이 없어 보인다.

```
train.columns
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
```

20. 공공의료기관 현황

행정구역별로 공공보건의료기관 수를 파악하고 행정구역별로 인구수 대비 공공보건의료기관 비율을 비교 분석한다.

■ 전국 공공보건의료기관 현황 데이터 수집하기

- 1) 공공데이터포털 사이트(www.data.go.kr)에서 '보건복지부_공공보건 의료기관 현황'을 클릭한다.
- 2) 데이터 상세 페이지가 나타나면 [다운로드] 버튼을 클릭한다.
- 3) 다운로드한 파일은 파일명을 '공공보건의료기관현황.csv'로 변경하여 저장한다.

■ 새로운 파일 '20.공공의료기관현황.ipynb'를 생성한 후 데이터프레임(data)에 저장한다.

```
import pandas as pd
data = pd.read_csv('data/공공보건의료기관현황.csv', encoding='CP949')
data.head()
```

	시도	시군구	기관유형	상위 보건기관명	보건기관명	주소	읍면동명	도서지역 여부	대표 전화번호
0	서울특별시	종로구	보건소	종로구보건소	종로구보건소	서울특별시 종로구 자하문로19길 36 (옥인동, 종로구보건소, 청운효자동자치회관) ...	청운효자동	아니오	02-2148-3514
1	서울특별시	중구	보건소	서울중구보건소	서울중구보건소	서울특별시 중구 다산로39길 16 (무학동, 중구보건소)	신당동	아니오	02-3396-6302
2	서울특별시	중구	일반보건지소	서울중구보건소	약수보건지소	서울특별시 중구 다산로 92 (신당동, 약수동주민센터) 약수동주민센터	신당동	아니오	02-3396-6964
3	서울특별시	중구	일반보건지소	서울중구보건소	황학보건지소	서울특별시 중구 난계로11길 52 (황학동, 황학동주민센터) 황학동주민센터	황학동	아니오	02-3396-6989
4	서울특별시	중구	일반보건지소	서울중구보건소	다산보건지소	서울특별시 중구 동호로15길 50 (신당동, 동사무소어린이집)	신당동	아니오	02-3396-6959

■ 데이터프레임(data)에서 시도, 군구 행정구역별 공공보건의료기관수를 구한다.

- 1) '주소' 컬럼 값을 blank로 분리하여 2차원 배열로 변경한 후 변수(addr)에 저장한다.

```
firstAddr = data.loc[0, '주소']
firstAddr
```

'서울특별시 종로구 자하문로19길 36 (옥인동, 종로구보건소, 청운효자동자치회관) 1~4층'

```
firstAddr.split()
```

['서울특별시', '종로구', '자하문로19길', '36', '(옥인동,', '종로구보건소,', '청운효자동자치회관)', '1~4층']

```
addr = data['주소'].apply(lambda x: x.split()[2]).tolist()
addr
```

['서울특별시', '종로구'],
 ['서울특별시', '중구'],
 ['서울특별시', '중구'],
 ['서울특별시', '중구'],
 ['서울특별시', '중구'],
 ['서울특별시', '용산구'],
 ['서울특별시', '성동구'],
 ['서울특별시', '성동구'],
 ...
 ['전라북도', '진안군'],
 ['전라북도', '무주군'],
 ['전라북도', '무주군'],
 ['전라북도', '무주군'],
 ...]

2) addr 2차원 배열을 데이터프레임(addr)로 변경한다.

```
addr = pd.DataFrame(addr, columns=['시도', '군구'])
addr.head()
```

	시도	군구
0	서울특별시	종로구
1	서울특별시	중구
2	서울특별시	중구
3	서울특별시	중구
4	서울특별시	중구

3) '시도' 칼럼의 고유값을 확인 후 아래 두 지역의 행정구역명을 최신 행정구역명으로 변경한다.

```
addr['시도'].unique()
```

```
array(['서울특별시', '충청북도', '충청남도', '전라북도', '전라남도', '경상북도', '경상남도', '제주특별자치도',
'세종특별자치시', '부산광역시', '대구광역시', '인천광역시', '광주광역시', '대전광역시', '울산광역시',
'경기도', '강원도', '강원특별자치도'], dtype=object)
```

```
address_aliases = {'강원도': '강원특별자치도', '전라북도': '전북특별자치도'}
addr['시도'] = addr['시도'].apply(lambda x: address_aliases.get(x, x))
addr['시도'].unique()
```

```
array(['서울특별시', '충청북도', '충청남도', '전북특별자치도', '전라남도', '경상북도', '경상남도',
'제주특별자치도', '세종특별자치시', '부산광역시', '대구광역시', '인천광역시', '광주광역시', '대전광역시',
'울산광역시', '경기도', '강원특별자치도'], dtype=object)
```

```
addr['군구'].unique()
```

```
array(['종로구', '중구', '용산구', '성동구', '광진구', '동대문구', '중랑구', '성북구', '강북구',
...,
'양양군'], dtype=object)
```

4) '시도', '군구' 칼럼을 연결하여 새로운 '시도군구' 칼럼을 추가한다.

```
addr['시도군구'] = addr.apply(lambda x: x['시도'] + ' ' + x['군구'], axis=1)
addr.head(2)
```

	시도	군구	시도군구
0	서울특별시	종로구	서울특별시 종로구
1	서울특별시	중구	서울특별시 중구

5) '시도', '군구', '시도군구' 별 공공보건의료기관 수를 구한다.

```
addr['count'] = 0
addr.head(2)
```

	시도	군구	시도군구	count
0	서울특별시	종로구	서울특별시 종로구	0
1	서울특별시	중구	서울특별시 중구	0

```
addr_group = addr.groupby(['시도', '군구', '시도군구'], as_index=False).count()
addr_group.head()
```

	시도	군구	시도군구	count
0	강원특별자치도	강릉시	강원특별자치도 강릉시	19
1	강원특별자치도	고성군	강원특별자치도 고성군	14
2	강원특별자치도	동해시	강원특별자치도 동해시	4
3	강원특별자치도	삼척시	강원특별자치도 삼척시	15
4	강원특별자치도	속초시	강원특별자치도 속초시	1

6) '시도군구' 컬럼을 index 컬럼으로 지정한다.

```
addr_group = addr_group.set_index('시도군구')
addr_group.head()
```

	시도	군구	count
시도군구			
강원특별자치도 강릉시	강원특별자치도	강릉시	19
강원특별자치도 고성군	강원특별자치도	고성군	14
강원특별자치도 동해시	강원특별자치도	동해시	4
강원특별자치도 삼척시	강원특별자치도	삼척시	15
강원특별자치도 속초시	강원특별자치도	속초시	1

▪ 행정구역별 인구수를 행정구역별 공공보건의료기관 수와 병합한다.

1) 데이터분석에서 작성한 '행정구역별인구수.csv' 파일을 읽어 데이터프레임(population)에 저장 후 컬럼 이름을 변경한다.

```
population = pd.read_csv('data/행정구역별인구수.csv', index_col=0)
population.head()
```

	행정구역1	행정구역2	총인구수	남자인구수	여자인구수
0	전국	합계	51313912	25558944	25754968
1	서울특별시	소계	9384325	4538354	4845971
2	서울특별시	종로구	139378	67240	72138
3	서울특별시	중구	121322	58651	62671
4	서울특별시	용산구	212175	101793	110382

```
population = population.rename(columns={'행정구역1':'시도', '행정구역2':'군구'})
population.head(3)
```

	시도	군구	총인구수	남자인구수	여자인구수
0	전국	합계	51313912	25558944	25754968
1	서울특별시	소계	9384325	4538354	4845971
2	서울특별시	종로구	139378	67240	72138

2) '군구' 컬럼에 blank을 strip() 함수를 이용해 제거한다.

```
population.loc[2, '군구'], len(population.loc[2, '군구'])
```

```
population.loc[2, '군구'], len(population.loc[2, '군구'])
```

```
population['군구'] = population['군구'].str.strip()
```

```
population.loc[2, '군구'], len(population.loc[2, '군구'])
```

```
('종로구', 3)
```

3) '시도' 컬럼과 '군구' 컬럼을 연결하여 새로운 '시도군구' 컬럼을 추가한다.

```
population['시도군구'] = population['시도'] + ' ' + population['군구']
population.head()
```

	시도	군구	총인구수	남자인구수	여자인구수	시도군구
0	전국	합계	51313912	25558944	25754968	전국 합계
1	서울특별시	소계	9384325	4538354	4845971	서울특별시 소계
2	서울특별시	종로구	139378	67240	72138	서울특별시 종로구
3	서울특별시	중구	121322	58651	62671	서울특별시 중구
4	서울특별시	용산구	212175	101793	110382	서울특별시 용산구

4) '군구' 컬럼 값에 '합계', '소계'인 행들을 삭제한다.

```
population = population[population.군구 != '합계']
population = population[population.군구 != '소계']
population.head(3)
```

	시도	군구	총인구수	남자인구수	여자인구수
시도군구					
서울특별시 종로구	서울특별시	종로구	139378	67240	72138
서울특별시 중구	서울특별시	중구	121322	58651	62671
서울특별시 용산구	서울특별시	용산구	212175	101793	110382

5) '시도군구' 컬럼을 index 컬럼으로 지정한다.

```
population.set_index('시도군구', inplace=True)
population.head(3)
```

	시도	군구	총인구수	남자인구수	여자인구수
시도군구					
서울특별시 종로구	서울특별시	종로구	139378	67240	72138
서울특별시 중구	서울특별시	중구	121322	58651	62671
서울특별시 용산구	서울특별시	용산구	212175	101793	110382

6) 두 개의 데이터프레임(addr_group, population)을 index 칼럼을 기준으로 같으면 병합한 후 데이터프레임에 저장한다.

```
addr_population_merge = pd.merge(addr_group, population,
                                  how='inner',
                                  left_index=True, right_index=True)
addr_population_merge.head()
```

	시도_x	군구_x	count	시도_y	군구_y	총인구수	남자인구수	여자인구수
시도군구								
강원특별자치도 강릉시	강원특별자치도	강릉시	19	강원특별자치도	강릉시	209174	103616	105558
강원특별자치도 고성군	강원특별자치도	고성군	14	강원특별자치도	고성군	27294	14008	13286
강원특별자치도 동해시	강원특별자치도	동해시	4	강원특별자치도	동해시	88591	45007	43584
강원특별자치도 삼척시	강원특별자치도	삼척시	15	강원특별자치도	삼척시	62309	31645	30664
강원특별자치도 속초시	강원특별자치도	속초시	1	강원특별자치도	속초시	82021	40606	41415

7) 필요한 칼럼들만 추려서 칼럼 이름을 변경한다.

```
local_population = addr_population_merge[['시도_x', '군구_x', 'count', '총인구수']]
local_population.head(2)
```

	시도_x	군구_x	count	총인구수
시도군구				
강원특별자치도 강릉시	강원특별자치도	강릉시	19	209174
강원특별자치도 고성군	강원특별자치도	고성군	14	27294

```
local_population = local_population.rename(columns={'시도_x':'시도', '군구_x':'군구', '총인구수':'인구수'})
local_population.head()
```

	시도	군구	count	인구수
시도군구				
강원특별자치도 강릉시	강원특별자치도	강릉시	19	209174
강원특별자치도 고성군	강원특별자치도	고성군	14	27294

8) 인구수별 공공보건의료기관수의 비율을 인구 100000명당 몇 개인지를 비율로 계산한다.

```
local_population['ratio'] = (local_population['count']/local_population['인구수']) * 100000
local_population.head()
```

	시도	군구	count	인구수	ratio
시도군구					
강원특별자치도 강릉시	강원특별자치도	강릉시	19	209174	9.083347
강원특별자치도 고성군	강원특별자치도	고성군	14	27294	51.293325

9) 전 처리한 데이터프레임(local_population)을 '인구수별공공의료기관수.csv' 파일로 저장한다.

```
local_population.to_csv('data/인구수별공공의료기관수.csv')
```

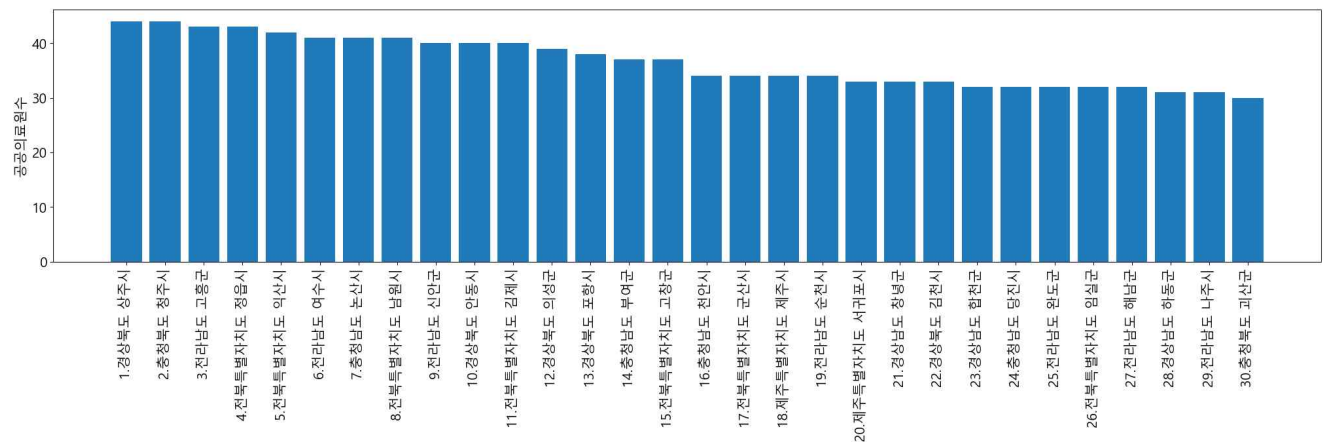
- 전 처리한 데이터프레임(local_population)을 이용해 시각화 그래프를 출력한다.

1) 시각화 작업에 필요한 matplotlib 패키지를 import하고 한글 설정을 한다.

```
import matplotlib.pyplot as plt
plt.rc('font', family='Malgun Gothic')
plt.rc('font', size= 15)
plt.rc('axes', unicode_minus=False)
```

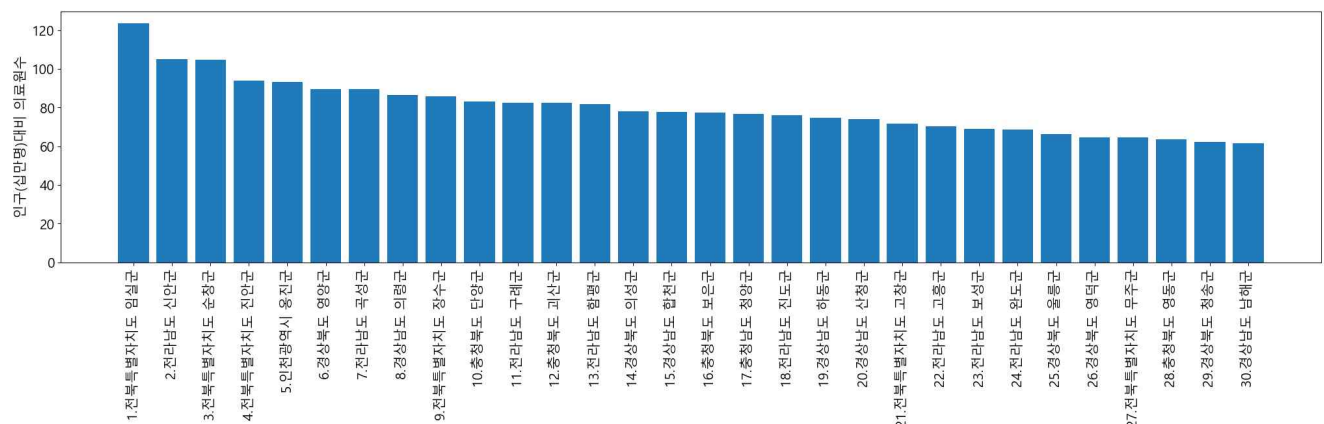
2) 행정구역별 공공보건의료기관 수를 막대그래프로 시각화 한다.

```
local = local_population.sort_values(['count'], ascending=False)
local = local[:30]
plt.figure(figsize=(25, 5))
plt.bar(local.index, local['count'])
plt.xticks(local.index, labels=[f'{i+1}. {x}' for i, x in enumerate(local.index)], rotation=90)
plt.ylabel('공공의료원수')
```



3) 행정구역별 인구대비 공공보건의료기관수 비율을 막대그래프로 시각화 한다.

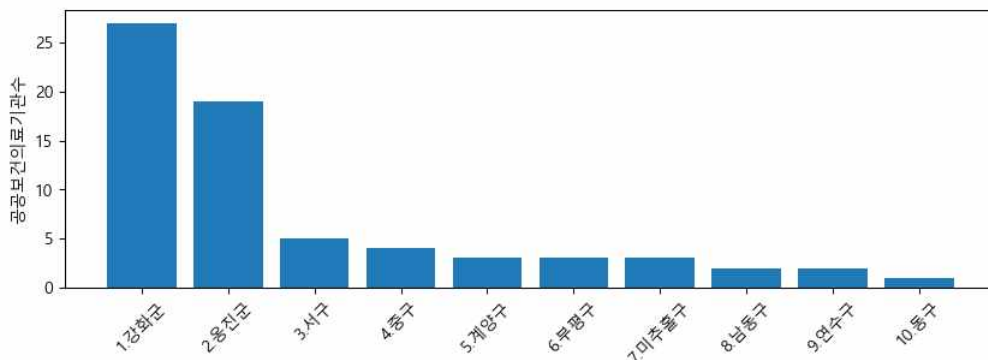
```
local = local_population.sort_values(['ratio'], ascending=False)
local = local[:30]
plt.figure(figsize=(25, 5))
plt.bar(local.index, local['ratio'])
plt.xticks(local.index, labels=[f'{i+1}. {x}' for i, x in enumerate(local.index)], rotation=90)
plt.ylabel('인구(십만명)대비 의료원수')
```



4) '인천광역시' 데이터만 필터링하여 '군구'별 공공보건의료기관수를 막대그래프로 인구대비 의료기관수 비율을 원그래프로 시각화한다.

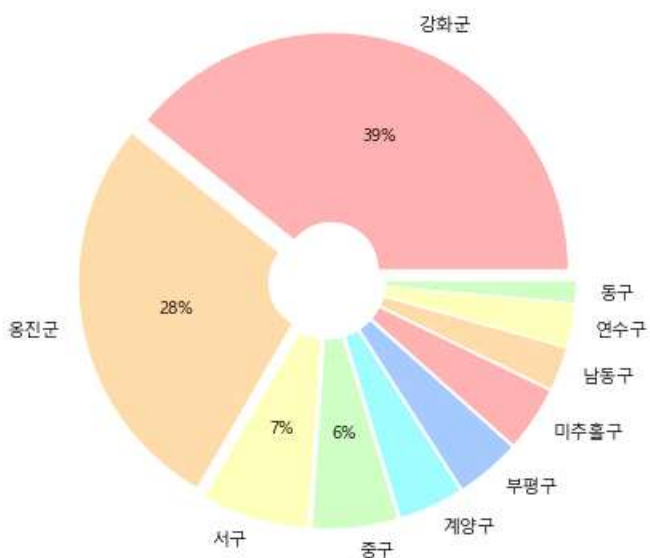
```
filt = local_population['시도'] == '인천광역시'
inchon = local_population[filt]
```

```
plt.rc('font', size= 10)
local = inchon.sort_values(['count'], ascending=False)
local = local[:30]
plt.figure(figsize=(10, 3))
plt.bar(local.index, local['count'])
plt.xticks(local.index, labels=[f'{i+1}. {x}' for i, x in enumerate(local['군구'])], rotation=45)
plt.ylabel('공공보건의료기관수')
```



```
def custom_autopct(pct):
    return f'{pct:.0f}%' if pct>=5 else ''
```

```
plt.rc('font', size= 8)
colors = ['#FFADAD', '#FFD6A5', '#FDFFB6', '#CAFFBF', '#9BF6FF', '#A0C4FF',
          '#FFADAD', '#FFD6A5', '#FDFFB6', '#CAFFBF', '#9BF6FF', '#A0C4FF']
plt.pie(local['count'], labels=local['군구'], colors=colors,
        explode=[0.05] * len(local.index),
        autopct=custom_autopct, wedgeprops={'width':0.8})
```



21. 공공의료기관 블록맵

행정구역별로 공공보건의료기관 수를 파악하고 행정구역별로 인구수 대비 공공보건의료기관 비율을 블록맵으로 시각화한다.

■ 데이터 준비하기

1) 블록맵을 그리기 위한 'data_draw_korea.csv' 파일을 구글링하여 다운로드한 후 준비한다.

2) 이전 장에서 만든 '인구수별공공의료기관수.csv' 파일을 준비한다.

■ 새로운 파일 '21.공공의료기관블록맵.ipynb'를 생성한 후 데이터프레임(data_draw_korea)에 저장한다. x와 y는 행정 구역을 나타내는 블록의 위치를 의미한다. 여기에도 인구수 칼럼이 있지만 이전 정에서 정리한 파일을 이용한다.

```
import pandas as pd
data_draw_korea = pd.read_csv('data/data_draw_korea.csv', index_col=0)
data_draw_korea.head(2)
```

	인구수	shortName	x	y	면적	광역시도	행정구역
0	202520	강릉	11	4	1040.07	강원도	강릉시
1	25589	고성(강원)	9	0	664.19	강원도	고성군
2	86747	동해	11	5	180.01	강원도	동해시
3	63986	삼척	11	8	1185.80	강원도	삼척시
4	76733	속초	9	1	105.25	강원도	속초시

■ 공공의료기관 블록맵 시각화를 위한 데이터프레임(data_draw_korea)을 전 처리 작업을 한다.

1) 행정구역명 중 변경된 시도명을 최근 시도명으로 변경한다.

```
data_draw_korea['광역시도'].unique()
```

```
array(['강원도', '경기도', '경상남도', '경상북도', '광주광역시', '대구광역시', '대전광역시', '부산광역시',  
      '서울특별시', '세종특별자치시', '울산광역시', '인천광역시', '전라남도', '전라북도',  
      '제주특별자치도', '충청남도', '충청북도'], dtype=object)
```

```
addr_aliases = {'강원도':'강원특별자치도', '전라북도':'전북특별자치도'}
```

```
data_draw_korea['광역시도'] = data_draw_korea['광역시도'].apply(lambda x:addr_aliases.get(x, x))
data_draw_korea['광역시도'].unique()
```

```
array(['강원특별자치도', '경기도', '경상남도', '경상북도', '광주광역시', '대구광역시', '대전광역시',  
      '부산광역시', '서울특별시', '세종특별자치시', '울산광역시', '인천광역시', '전라남도', '전북특별자치도',  
      '제주특별자치도', '충청남도', '충청북도'], dtype=object)
```

2) 인천광역시의 남구를 최근 행정구역명인 미추홀구로 변경한다.

```
filt = data_draw_korea['광역시도'] == '인천광역시'
data_draw_korea[filt].head()
```

	인구수	shortName	x	y	면적	광역시도	행정구역
155	58296	인천강화	1	3	411.20	인천광역시	강화군
156	317785	인천계양	2	5	45.58	인천광역시	계양구
157	388535	인천남구	2	7	24.47	인천광역시	남구
158	505791	인천남동	2	8	56.90	인천광역시	남동구
159	68950	인천동구	1	5	7.05	인천광역시	동구

```
data_draw_korea.loc[157, '행정구역'] = '미추홀구'
data_draw_korea[filter].head()
```

	인구수	shortName	x	y	면적	광역시도	행정구역
155	58296	인천강화	1	3	411.20	인천광역시	강화군
156	317785	인천계양	2	5	45.58	인천광역시	계양구
157	388535	인천남구	2	7	24.47	인천광역시	미추홀구
158	505791	인천남동	2	8	56.90	인천광역시	남동구
159	68950	인천동구	1	5	7.05	인천광역시	동구

3) '광역시도' 칼럼과 '행정구역' 칼럼을 연결하여 새로운 '시도군구' 칼럼을 추가한 뒤 '시도군구' 칼럼을 index 칼럼으로 변경한다.

```
data_draw_korea['시도군구'] = data_draw_korea['광역시도'] + ' ' + data_draw_korea['행정구역']
data_draw_korea.set_index('시도군구', inplace=True)
data_draw_korea.head()
```

	인구수	shortName	x	y	면적	광역시도	행정구역
시도군구							
강원특별자치도 강릉시	202520	강릉	11	4	1040.07	강원특별자치도	강릉시
강원특별자치도 고성군	25589	고성(강원)	9	0	664.19	강원특별자치도	고성군
강원특별자치도 동해시	86747	동해	11	5	180.01	강원특별자치도	동해시
강원특별자치도 삼척시	63986	삼척	11	8	1185.80	강원특별자치도	삼척시
강원특별자치도 속초시	76733	속초	9	1	105.25	강원특별자치도	속초시

■ 공공의료기관 블록맵 시각화를 위한 데이터프레임(local_population)을 전 처리 작업을 한다.

1) '인구수별공공의료기관수.csv' 파일을 읽어 데이터프레임(local_population)에 저장한다.

```
local_population = pd.read_csv('data/인구수별공공의료기관수.csv')
local_population.set_index('시도군구', inplace=True)
local_population.head()
```

	시도	군구	count	인구수	ratio
시도군구					
강원특별자치도 강릉시	강원특별자치도	강릉시	19	209174	9.083347
강원특별자치도 고성군	강원특별자치도	고성군	14	27294	51.293325
강원특별자치도 동해시	강원특별자치도	동해시	4	88591	4.515131
강원특별자치도 삼척시	강원특별자치도	삼척시	15	62309	24.073569
강원특별자치도 속초시	강원특별자치도	속초시	1	82021	1.219200

2) 두 개의 데이터프레임(data_draw_korea, local_population)을 index를 기준으로 외부 병합한다. 블록맵에 있는 '시도군구'이지만 공공의료기관이 없는 경우에는 ratio에 NaN으로 표시된다.

```
data_draw_korea_population_all = pd.merge(data_draw_korea, local_population,
                                           how='outer',
                                           left_index=True, right_index=True )
```



```
data_draw_korea_population_all.head(5)
```

	인구수_x	shortName	x	y	면적	광역시도	행정구역	시도	군구	count	인구수_y	ratio
시도군구												
강원특별자치도 강릉시	202520	강릉	11	4	1040.07	강원특별자치도	강릉시	강원특별자치도	강릉시	19.0	209174.0	9.083347
강원특별자치도 고성군	25589	고성(강원)	9	0	664.19	강원특별자치도	고성군	강원특별자치도	고성군	14.0	27294.0	51.293325
강원특별자치도 동해시	86747	동해	11	5	180.01	강원특별자치도	동해시	강원특별자치도	동해시	4.0	88591.0	4.515131
강원특별자치도 삼척시	63986	삼척	11	8	1185.80	강원특별자치도	삼척시	강원특별자치도	삼척시	15.0	62309.0	24.073569
강원특별자치도 속초시	76733	속초	9	1	105.25	강원특별자치도	속초시	강원특별자치도	속초시	1.0	82021.0	1.219200

3) 데이터프레임(data_draw_korea) 칼럼별 NaN 값이 몇 개인지 확인한다.

```
data_draw_korea_population_all.isna().sum()
```

```
인구수_x      0
shortName     0
x             0
y             0
면적          0
광역시도      0
행정구역      0
시도          2
군구          2
count         2
인구수_y      2
ratio         2
dtype: int64
```

```
print(data_draw_korea.shape, local_population.shape, data_draw_korea_population_all.shape)
```

```
(229, 7) (227, 5) (229, 12)
```

4) 'count' 칼럼 값이 NaN인 행을 확인한다.

```
filt = data_draw_korea_population_all['count'].isna()
data_draw_korea_population_all[filt]
```

	인구수_x	shortName	x	y	면적	광역시도	행정구역	시도	군구	count	인구수_y	ratio
시도군구												
경상북도 군위군	20596	군위	9	11	614.15	경상북도	군위군	NaN	NaN	NaN	NaN	NaN
세종특별자치시 세종시	191233	세종	4	11	465.23	세종특별자치시	세종시	NaN	NaN	NaN	NaN	NaN

▪ 행정구역별 공공보건의료기관 수와 인구수 대비 공공보건의료기관 비율을 블록맵으로 시각화한다.

1) 시각화 작업에 필요한 matplotlib 패키지를 import하고 한글을 설정한다.

```
import matplotlib.pyplot as plt

plt.rc('font', family='Malgun Gothic')
plt.rc('font', size= 15)
plt.rc('axes', unicode_minus=False)
```

2) 블록맵의 행정구역 경계선을 그리기 위해 행정구역의 블록 위치인 x, y 데이터를 정의한다.

```
BORDER_LINES = [
    [(3, 2), (5, 2), (5, 3), (9, 3), (9, 1)],# 인천
    [(2, 5), (3, 5), (3, 4), (8, 4), (8, 7), (7, 7), (7, 9), (4, 9), (4, 7), (1, 7)],# 서울
    [(1, 6), (1, 9), (3, 9), (3, 10), (8, 10), (8, 9), (9, 9), (9, 8), (10, 8), (10, 5), (9, 5), (9, 3)],# 경기도
    [(9, 12), (9, 10), (8, 10)],# 강원도
    [(10, 5), (11, 5), (11, 4), (12, 4), (12, 5), (13, 5), (13, 4), (14, 4), (14, 2)],# 충청남도
    [(11, 5), (12, 5), (12, 6), (15, 6), (15, 7), (13, 7), (13, 8), (11, 8), (11, 9), (10, 9), (10, 8)],# 충청북도
    [(14, 4), (15, 4), (15, 6)],# 대전시
    [(14, 7), (14, 9), (13, 9), (13, 11), (13, 13)],# 경상북도
    [(14, 8), (16, 8), (16, 10), (15, 10), (15, 11), (14, 11), (14, 12), (13, 12)],# 대구시
    [(15, 11), (16, 11), (16, 13)],# 울산시
    [(17, 1), (17, 3), (18, 3), (18, 6), (15, 6)],# 전라북도
    [(19, 2), (19, 4), (21, 4), (21, 3), (22, 3), (22, 2), (19, 2)],# 광주시
    [(18, 5), (20, 5), (20, 6)],# 전라남도
    [(16, 9), (18, 9), (18, 8), (19, 8), (19, 9), (20, 9), (20, 10)],# 부산시
]
```

3) 블록맵의 블록에 데이터를 매핑하고 색을 표시하여 블록맵을 그린 뒤 저장하는 함수를 정의한다.

```
def draw_blockMap(blockedMap, targetData, title, color):
    whitelabelmin = (max(blockedMap[targetData]) - min(blockedMap[targetData])) * 0.25 + min(blockedMap[targetData])

    datalabel = targetData
    vmin = min(blockedMap[targetData])
    vmax = max(blockedMap[targetData])

    mapdata = blockedMap.pivot(index='y', columns='x', values=targetData)
    #print(mapdata)
    masked_mapdata = np.ma.masked_where(np.isnan(mapdata), mapdata)

    plt.figure(figsize=(8, 13))
    plt.title(title)
    plt.pcolor(masked_mapdata, vmin=vmin, vmax=vmax, cmap=color, edgecolor='#aaaaaa', linewidth=0.5)

    # 지역 이름 표시
    for idx, row in blockedMap.iterrows():
        annocolor = 'white' if row[targetData] > whitelabelmin else 'black'

        # 광역시는 구 이름이 겹치는 경우가 많아서 시단위 이름도 같이 표시한다. (중구, 서구)
        if row['광역시도'].endswith('시') and not row['광역시도'].startswith('세종'):
            dispname = '{}\n{}'.format(row['광역시도'][:2], row['행정구역'][:-1])
            if len(row['행정구역']) <= 2:
                dispname += row['행정구역'][:-1]
        else:
            dispname = row['행정구역'][:-1]

        # 서대문구, 서귀포시 같이 이름이 3자 이상인 경우에 작은 글자로 표시한다.
        if len(dispname.splitlines()[-1]) >= 3:
            fontsize, linespacing = 9.5, 1.5
        else:
            fontsize, linespacing = 11, 1.2

        plt.annotate(dispname, (row['x']+0.5, row['y']+0.5), weight='bold',
                     fontsize=fontsize, ha='center', va='center', color=annocolor, linespacing=linespacing)

    # 시도 경계 그린다.
    for path in BORDER_LINES:
        ys, xs = zip(*path)
        plt.plot(xs, ys, c='black', lw=4)

    plt.gca().invert_yaxis()
    #plt.gca().set_aspect(1)
    plt.axis('off')

    cb = plt.colorbar(shrink=.1, aspect=10)
    cb.set_label(datalabel)

    plt.tight_layout()
    #plt.savefig(f'{targetData}.png')
    plt.show()
```

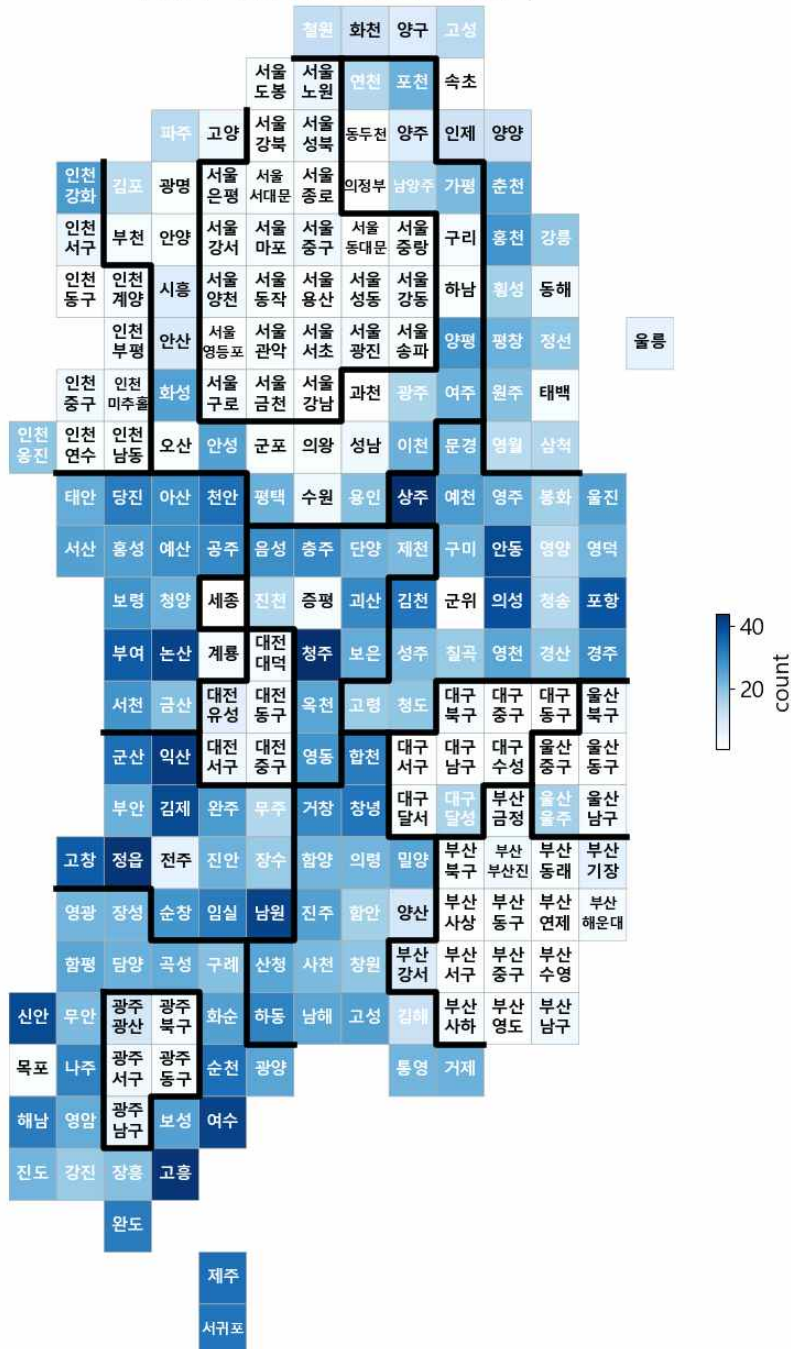
4) 데이터프레임(data_draw_korea_population_all) 'count' 칼럼 값에 대해 Blues 색상 스펙트럼을 사용하여 블록맵을 작성한다. 흰 색은 값이 없는 블록, 즉 외부병합 결과가 NaN인 행정구역이고 가장 진한 색의 블록은 공공보건의료기관이 가장 많은 행정구역이다.

```
count = data_draw_korea_population_all[['count']]
count.sort_values('count', ascending=False)[:3]
```

count	
시도군구	
충청북도 청주시	44.0
경상북도 상주시	44.0

```
draw_blockMap(data_draw_korea_population_all, 'count', '행정구역별 공공보건의료기관 수', 'Blues')
```

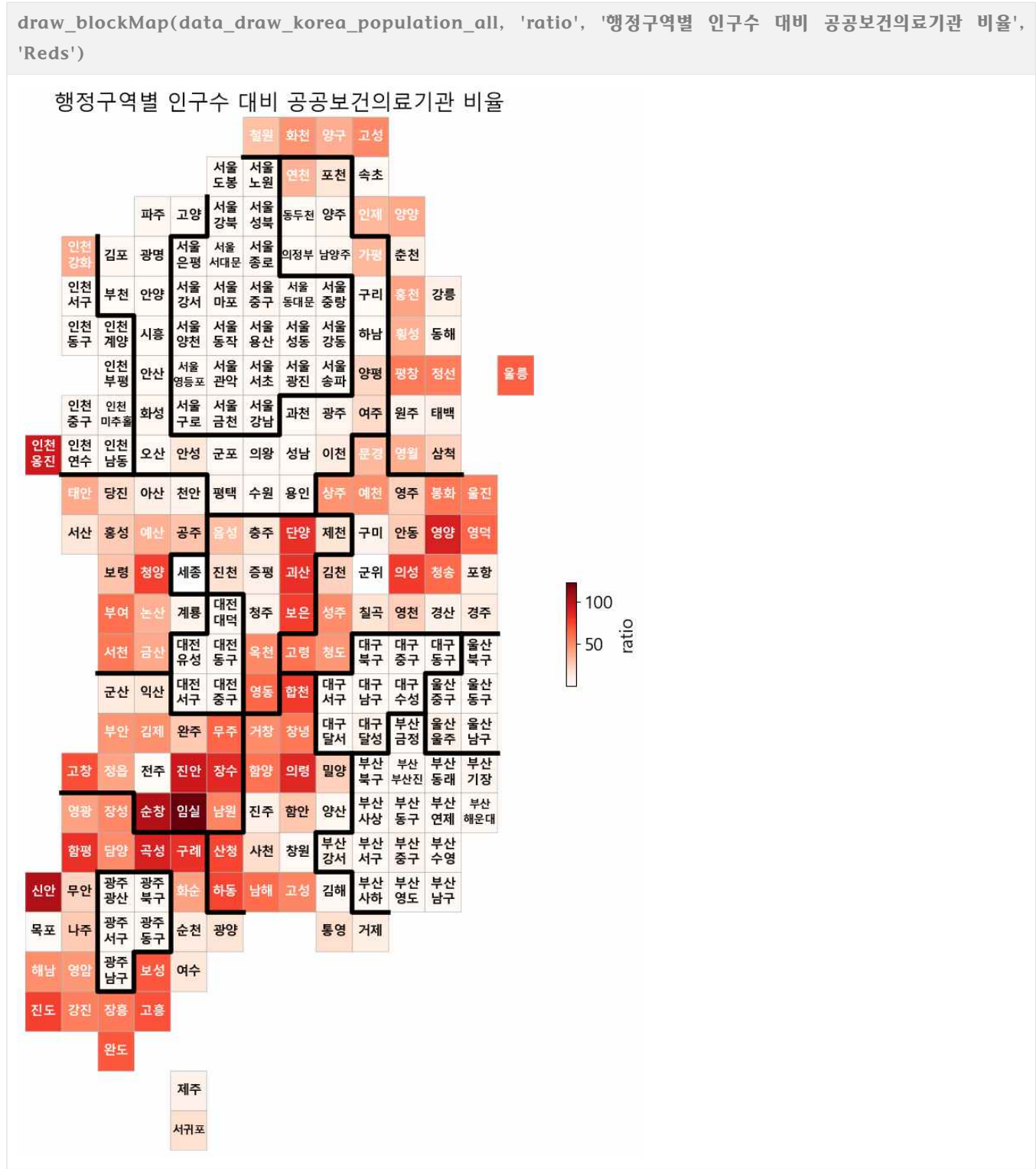
행정구역별 공공보건의료기관 수



5) 데이터프레임(data_draw_korea_population_all) 'ratio' 칼럼 값에 대해 Reds 색상 스펙트럼을 사용하여 블록맵을 작성한다.

```
ratio = data_draw_korea_population_all[['ratio']]
ratio.sort_values('ratio', ascending=False)[:2]
```

	ratio
시도군구	
전북특별자치도 임실군	123.628496
전라남도 신안군	105.141415



22. 시계열 날씨

데이터분석 단계에서 전 처리 작업으로 생성한 '전국기온강수량_2024_전처리.csv' 파일을 이용하여 데이터 시각화를 한다.

- 시각화 작업할 데이터를 읽어 데이터프레임(df_temp)에 저장한다.

```
import pandas as pd
df_temp = pd.read_csv('data/전국기온강수량_2024_전처리.csv')
```

df_temp.head()

	일시	지점	지점명	평균기온	최저기온	최고기온	일강수량	년	월	일	시도
0	2024-01-01	90	속초	2.7	0.9	6.1	0.4	2024	1	1	강원특별자치도
1	2024-01-02	90	속초	3.1	1.1	7.3	0.0	2024	1	2	강원특별자치도
2	2024-01-03	90	속초	3.1	0.2	6.7	0.0	2024	1	3	강원특별자치도
3	2024-01-04	90	속초	2.9	-2.3	6.6	0.0	2024	1	4	강원특별자치도
4	2024-01-05	90	속초	7.7	4.7	10.3	0.0	2024	1	5	강원특별자치도

df_temp.tail()

	일시	지점	지점명	평균기온	최저기온	최고기온	일강수량	년	월	일	시도
35479	2024-12-27	296	부산	1.9	-2.5	6.4	0.0	2024	12	27	부산
35480	2024-12-28	296	부산	1.3	-2.0	5.4	0.0	2024	12	28	부산
35481	2024-12-29	296	부산	1.6	-4.8	9.6	0.0	2024	12	29	부산
35482	2024-12-30	296	부산	6.0	-1.6	12.6	0.0	2024	12	30	부산
35483	2024-12-31	296	부산	5.2	-3.3	9.9	0.0	2024	12	31	부산

- 데이터프레임(df_temp) '일시' 칼럼의 타입을 확인 후 날짜 타입으로 변경한다.

```
type(df_temp.loc[0, '일시'])
```

str

```
df_temp['일시'] = pd.to_datetime(df_temp['일시'])
```

```
type(df_temp.loc[0, '일시'])
```

pandas._libs.tslibs.timestamps.Timestamp

- '인천' 지역의 일강수량을 꺾은선 그래프로 시각화한다.

1) 시각화 작업에 필요한 matplotlib 패키지를 import하고 한글을 설정한다.

```
import matplotlib.pyplot as plt

plt.rc('font', family='Malgun Gothic')
plt.rc('font', size= 10)
plt.rc('axes', unicode_minus=False)
```

2) '시도', '일시' 그룹화로 '평균기온', '일강수량'의 평균을 구한 후 '인천' 지역 데이터만 추출하여 데이터프레임(df_filt1)에 저장한다.

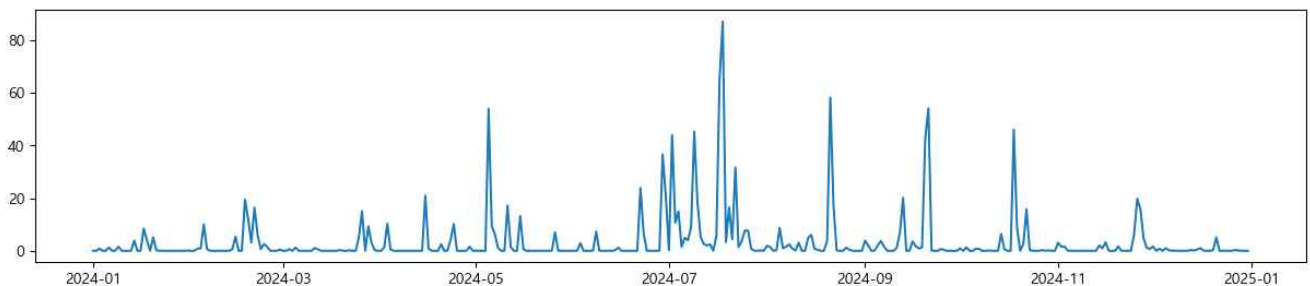
```
group = df_temp.groupby(['시도', '일시'], as_index=False)[['평균기온', '일강수량']].mean()
city1 = '인천'
filt = group['시도'] == city1
df_filt1 = group[filt]
df_filt1.head()
```

	시도	일시	평균기온	일강수량
3660	인천	2024-01-01	2.325	0.00
3661	인천	2024-01-02	2.750	0.00
3662	인천	2024-01-03	1.675	0.85
3663	인천	2024-01-04	2.000	0.00
3664	인천	2024-01-05	4.400	0.00

3) '인천' 지역의 '일시'별 '일강수량'을 꺾은선 그래프로 시각화 한다.

```
plt.figure(figsize=(15, 3))
plt.plot(df_filt1['일시'], df_filt1['일강수량'])
plt.title(f'2024년 {city1} 일강수량\n', size=20)
plt.show()
```

2024년 인천 일강수량



■ '인천', '강원특별자치도' 지역의 일강수량을 꺾은선 그래프로 시각화한다.

1) 그룹화한 데이터프레임(group)에서 '강원특별자치도' 데이터만 추출하여 데이터프레임(df_filt2)에 저장한다.

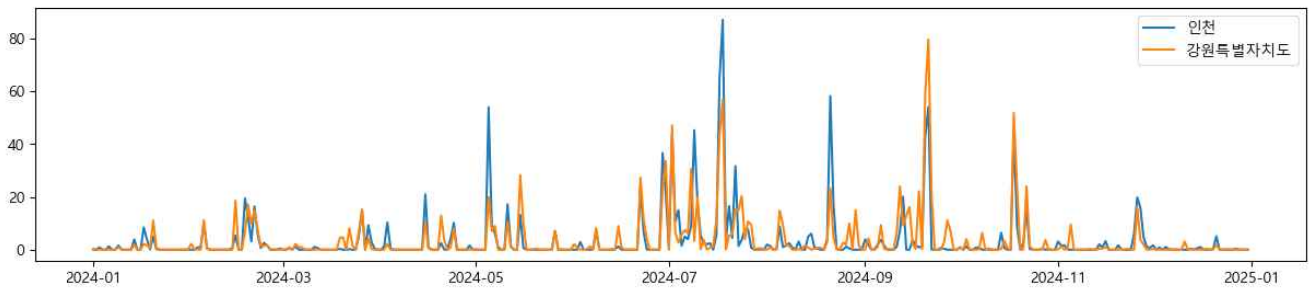
```
city2 = '강원특별자치도'
filt = group['시도'] == city2
df_filt2 = group[filt]
df_filt2.head()
```

	시도	일시	평균기온	일강수량
0	강원특별자치도	2024-01-01	2.214286	0.285714
1	강원특별자치도	2024-01-02	2.714286	0.000000
2	강원특별자치도	2024-01-03	1.085714	0.021429
3	강원특별자치도	2024-01-04	0.221429	0.000000
4	강원특별자치도	2024-01-05	4.564286	0.000000

2) '인천', '강원특별자치도' 지역의 '일시'별 '일강수량'을 꺾은선 그래프로 시각화 한다.

```
plt.figure(figsize=(15, 3))
plt.plot(df_filt1['일시'], df_filt1['일강수량'], label=city1)
plt.plot(df_filt2['일시'], df_filt2['일강수량'], label=city2)
plt.title(f'2024년 {city1}, {city2} 일강수량\n', size=20)
plt.legend()
plt.show()
```

2024년 인천, 강원특별자치도 일강수량



■ '인천', '강원특별자치도' 지역의 '일시'별 '일강수량'을 꺾은선 그래프의 X축 눈금을 변경한다.

1) '일시' 컬럼의 타입을 String으로 변경 후 '일'에 해당하는 값이 '01'만 추출하여 년, 월, 일로 list를 생성한다.

```
xticks = [str(x)[:10] for x in df_filt1['일시'] if str(x)[8:10]=='01']
```

xticks

```
['2024-01-01',
 '2024-02-01',
 '2024-03-01',
 '2024-04-01',
 '2024-05-01',
 '2024-06-01',
 '2024-07-01',
 '2024-08-01',
 '2024-09-01',
 '2024-10-01',
 '2024-11-01',
 '2024-12-01']
```

2) xticks list에 새로운 '2025-01-01' 데이터를 추가한다.

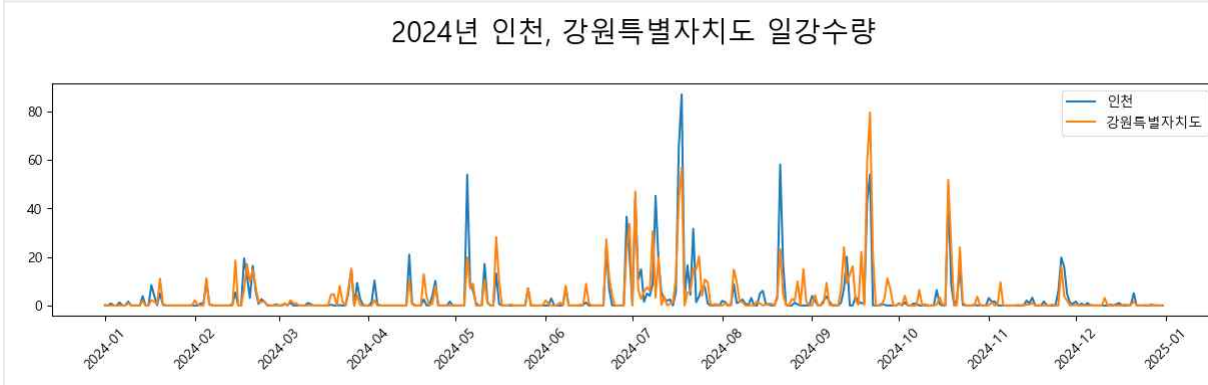
```
xticks.append('2025-01-01')
```

xticks

```
['2024-01-01',
 '2024-02-01',
 '2024-03-01',
 '2024-04-01',
 '2024-05-01',
 '2024-06-01',
 '2024-07-01',
 '2024-08-01',
 '2024-09-01',
 '2024-10-01',
 '2024-11-01',
 '2024-12-01',
 '2025-01-01']
```

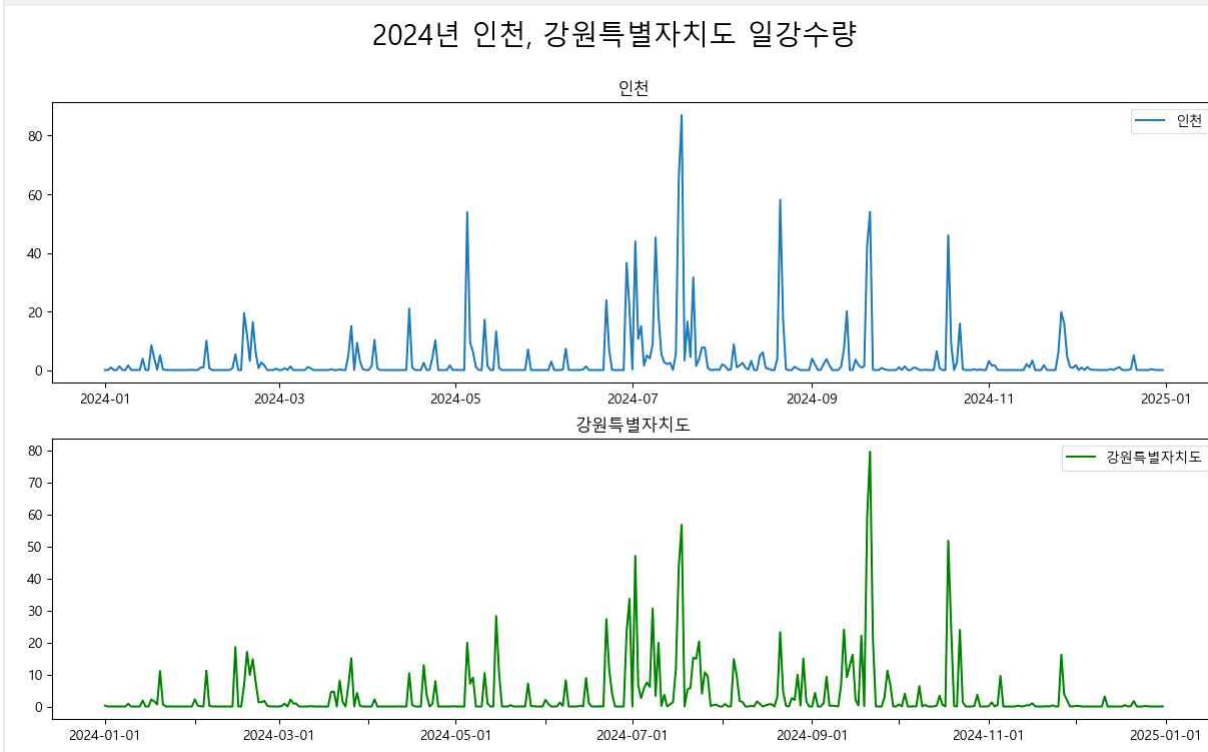

3) '인천', '강원특별자치도' 지역의 '일시'별 '일강수량' 꺾은선 그래프의 X축 눈금을 변경한다.

```
plt.figure(figsize=(15, 3))
plt.plot(df_filt1['일시'], df_filt1['일강수량'], label=city1)
plt.plot(df_filt2['일시'], df_filt2['일강수량'], label=city2)
plt.title(f'2024년 {city1}, {city2} 일강수량\n', size=20)
plt.xticks(xticks, labels=[x[:7] for x in xticks], rotation=45)
plt.legend()
```



▪ 두 개의 그래프를 subplots() 함수를 이용하여 하나의 figure에 출력한다.

```
fig, axs = plt.subplots(2, 1, figsize=(15, 8))
fig.suptitle(f'2024년 {city1}, {city2} 일강수량\n', size=20)
axs[0].plot(df_filt1['일시'], df_filt1['일강수량'], label=city1)
axs[0].set_title(f'{city1}')
axs[0].legend()
axs[1].set_title(f'{city2}')
axs[1].plot(df_filt2['일시'], df_filt2['일강수량'], label=city2, color='g')
axs[1].set_xticks(xticks, [x if int(x[5:7])%2==1 else '' for x in xticks])
axs[1].legend()
```



- 월별 최저, 평균, 최고기온을 구하여 시각화 한다.

1) 데이터프레임(df_temp)의 월별 '최고기온'의 최댓값을 구하여 데이터프레임(group_max)에 저장한다.

```
group_max = df_temp.groupby('월')[['최고기온']].max()
group_max.head(3)
```

최고기온	
월	
1	18.7
2	22.9
3	26.7

2) 데이터프레임(df_temp)의 월별 '최저기온'의 최솟값을 구하여 데이터프레임(group_min)에 저장한다.

```
group_min = df_temp.groupby('월')[['최저기온']].min()
group_min.head(3)
```

최저기온	
월	
1	-21.9
2	-15.9
3	-15.6

3) 데이터프레임(df_temp)의 월별 '평균기온'의 평균값을 구하여 데이터프레임(group_mean)에 저장한다.

```
group_mean = df_temp.groupby('월')[['평균기온']].mean()
group_mean.head(3)
```

평균기온	
월	
1	1.312936
2	4.425098
3	7.107951

4) 세 개의 데이터프레임을 index 기준으로 내부 병합해 데이터프레임(merge_month)에 저장한다.

```
merge_month=pd.merge(group_min, group_mean, how='inner', left_index=True, right_index=True)
merge_month= pd.merge(merge_month, group_max, how='inner', left_index=True, right_index=True)
merge_month.head()
```

	최저기온	평균기온	최고기온
월			
1	-21.9	1.312936	18.7
2	-15.9	4.425098	22.9
3	-15.6	7.107951	26.7
4	-3.8	14.944227	32.2
5	-1.3	17.724090	33.4

5) Y축 limit을 설정하기 위해 '최고기온'의 최댓값, '최저기온'의 최솟값을 구한다.

```
import math

max = math.ceil(merge_month['최고기온'].max())
min = math.floor(merge_month['최저기온'].min())
max, min
```

```
(40, -21)
```

6) X축 눈금의 위치 값을 구한다. 데이터행수(merge.index) * 4(최소, 평균, 최대 bar + 빈칸)의 위치 값이 생성된다.

```
xticks = merge_month.index * 4
xticks
```

```
Index([4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48], dtype='int64', name='월')
```

7) 월별 최저, 평균, 최저기온을 구하여 시각화 한다.

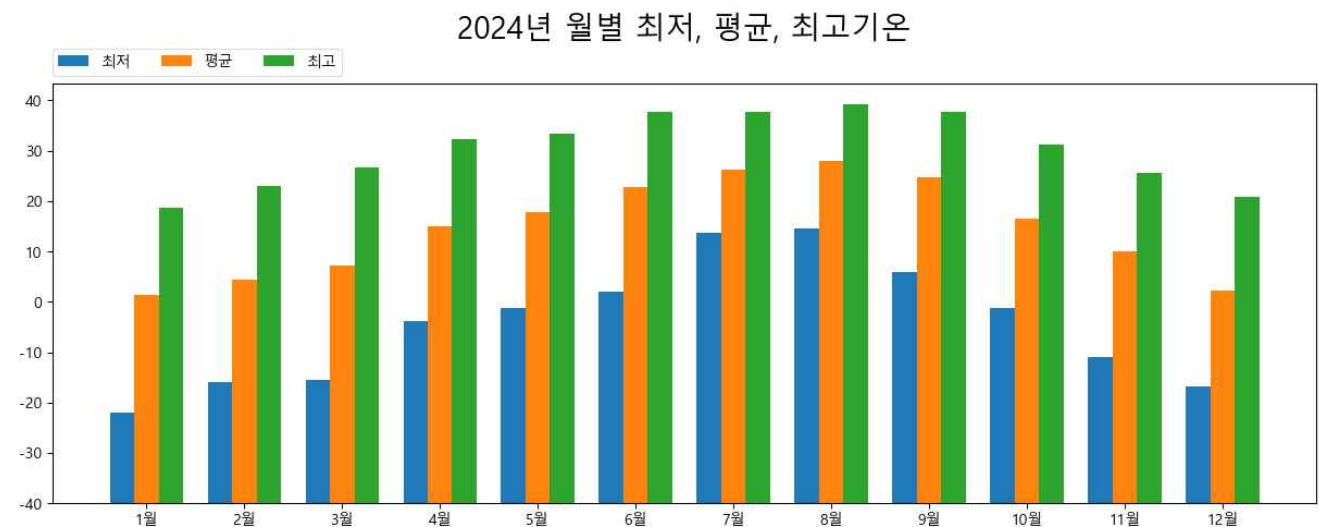
```
plt.figure(figsize=(15, 5))

plt.bar(xticks+0, merge_month['최저기온'] + 40, width=1, label='최저')
plt.bar(xticks+1, merge_month['평균기온'] + 40, width=1, label='평균')
plt.bar(xticks+2, merge_month['최고기온'] + 40, width=1, label='최고')

yticks=[y for y in range(0, 81, 10)]
plt.yticks(yticks, labels=[y-40 for y in yticks])

plt.xticks(xticks+1, [f'{x}월' for x in merge_month.index])
plt.title('2024년 월별 최저, 평균, 최고기온\n', size=20)

plt.legend(loc=(0, 1.02), ncol=3)
plt.show()
```



- 시도별 최저, 평균, 최고기온을 구하여 시각화 한다.

1) 데이터프레임(df_temp)의 시도별 '최고기온'의 최댓값을 구하여 데이터프레임(group_max)에 저장한다.

```
group_max = df_temp.groupby('시도')[['최고기온']].max()
group_max.head()
```

최고기온	
시도	
강원특별자치도	38.5
경기	37.2
경남	39.3

2) 데이터프레임(df_temp)의 시도별 '최저기온'의 최솟값을 구하여 데이터프레임(group_min)에 저장한다.

```
group_min = df_temp.groupby('시도')[['최저기온']].min()
group_min.head()
```

최저기온	
시도	
강원특별자치도	-21.9
경기	-15.7
경남	-11.5

3) 데이터프레임(df_temp)의 시도별 '평균기온'의 평균값을 구하여 데이터프레임(group_mean)에 저장한다.

```
group_mean = df_temp.groupby('시도')[['평균기온']].min()
group_mean.head()
```

평균기온	
시도	
강원특별자치도	-17.5
경기	-12.7
경남	-7.6

4) 세 개의 데이터프레임을 index 기준으로 내부 병합해 데이터프레임(merge_city)에 저장한다.

```
merge_city = pd.merge(group_min, group_mean, how='inner', left_index=True, right_index=True)
merge_city = pd.merge(merge_city, group_max, how='inner', left_index=True, right_index=True)
merge_city.head()
```

	최저기온	평균기온	최고기온
시도			
강원특별자치도	-21.9	-17.5	38.5
경기	-15.7	-12.7	37.2
경남	-11.5	-7.6	39.3
경북	-14.9	-10.8	38.9
대구	-8.8	-6.2	37.2

5) Y축 limit을 설정하기 위해 '최고기온'의 최댓값, '최저기온'의 최솟값을 구한다.

```
import math
max = math.ceil(merge_month['최고기온'].max())
min = math.floor(merge_month['최저기온'].min())
max, min
```

(40, -21)

6) X축 눈금의 위치 값을 구한다. 데이터행수(len(merge_city)) * 4(최소, 평균, 최대 bar + 빈칸)의 위치 값이 생성된다.

```
import numpy as np
xticks = np.arange(len(merge_city)) * 4
xticks
```

array([0, 4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 52, 56, 60])

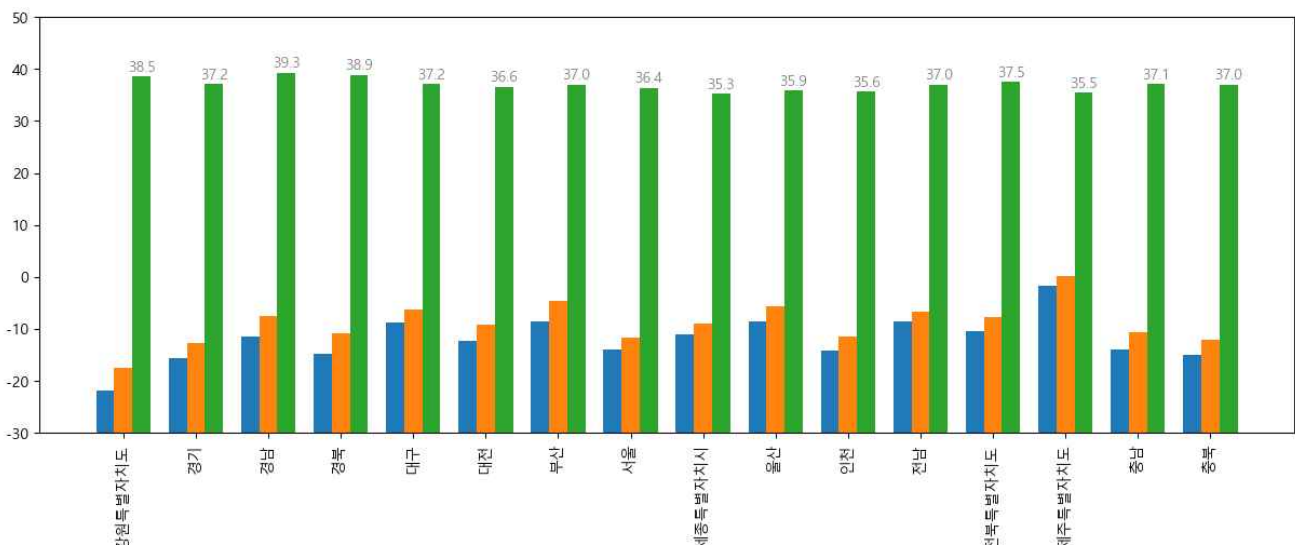
7) 시도별 최저, 평균, 최고기온을 구하여 시각화 한다.

```
plt.figure(figsize=(15, 5))
plt.bar(xticks+0, merge_city['최저기온'] + 30, width=1)
plt.bar(xticks+1, merge_city['평균기온'] + 30, width=1)
plt.bar(xticks+2, merge_city['최고기온'] + 30, width=1)

yticks=[y for y in range(0, 90, 10)]
plt.yticks(yticks, labels=[y-30 for y in yticks])
plt.xticks(xticks+1, [x for x in merge.index], rotation=90)
plt.title('2024년 시도별 최저, 평균, 최고기온\n', size=20)

for idx, val in enumerate(merge_city['최고기온']):
    plt.text(xticks[idx]+2, val+30+1, val, ha='center', size=10, color='gray')
plt.show()
```

2024년 시도별 최저, 평균, 최고기온



- 월별 '평균기온'과 '일강수량'을 구하여 '평균기온'은 막대그래프로 '일강수량'은 꺾은선그래프로 시각화한다.

1) 월별 '평균기온', '일강수량'의 평균을 구하여 데이터프레임(group_month)에 저장한다.

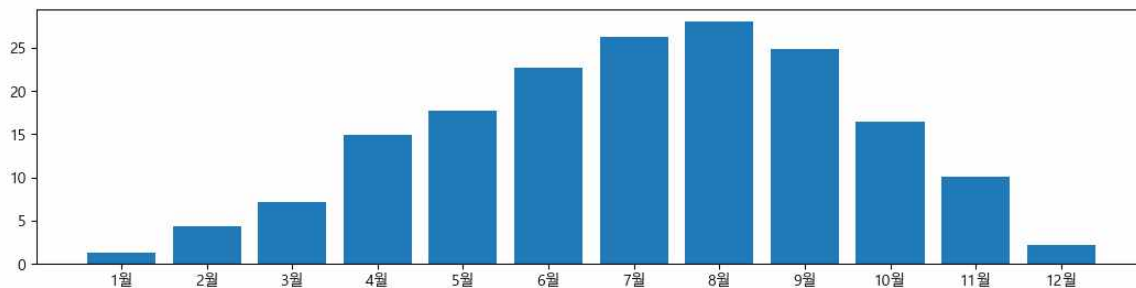
```
group_month = df_temp.groupby('월', as_index=False)[['평균기온', '일강수량']].mean()
group_month.head()
```

	월	평균기온	일강수량
0	1	1.312936	1.138577
1	2	4.425098	3.799716
2	3	7.107951	2.199701
3	4	14.944227	2.831134
4	5	17.724090	3.867858

2) 월별 '평균기온'의 평균을 막대그래프로 시각화 한다.

```
plt.figure(figsize=(13, 3))
plt.bar(group_month.월, group_month.평균기온)
plt.title('2024년 월별 평균기온\n', size=20)
plt.xticks(group_month.월, [f'{x}월' for x in group_month.월])
plt.show()
```

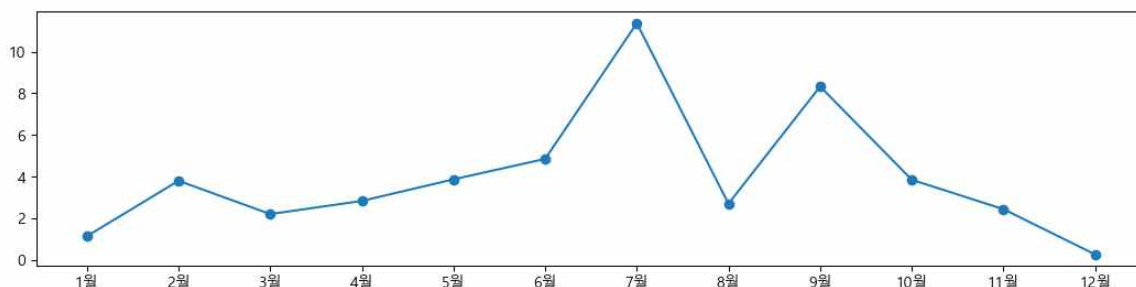
2024년 월별 평균기온



3) 월별 '일강수량'의 평균을 꺾은선 그래프로 시각화 한다.

```
plt.figure(figsize=(13, 3))
plt.plot(group_month.월, group_month.일강수량, marker='o')
plt.title('2024년 월별 일강수량\n', size=20)
plt.xticks(group_month.월, [f'{x}월' for x in group_month.월])
plt.show()
```

2024년 월별 일강수량



4) 월별 '평균기온'을 막대그래프로, '일강수량'을 꺾은선 그래프로 시각화 한다.

```
fig, ax1 = plt.subplots(figsize=(15, 5))
fig.suptitle('2024년 월별 평균기온과 강수량\n', size=20)

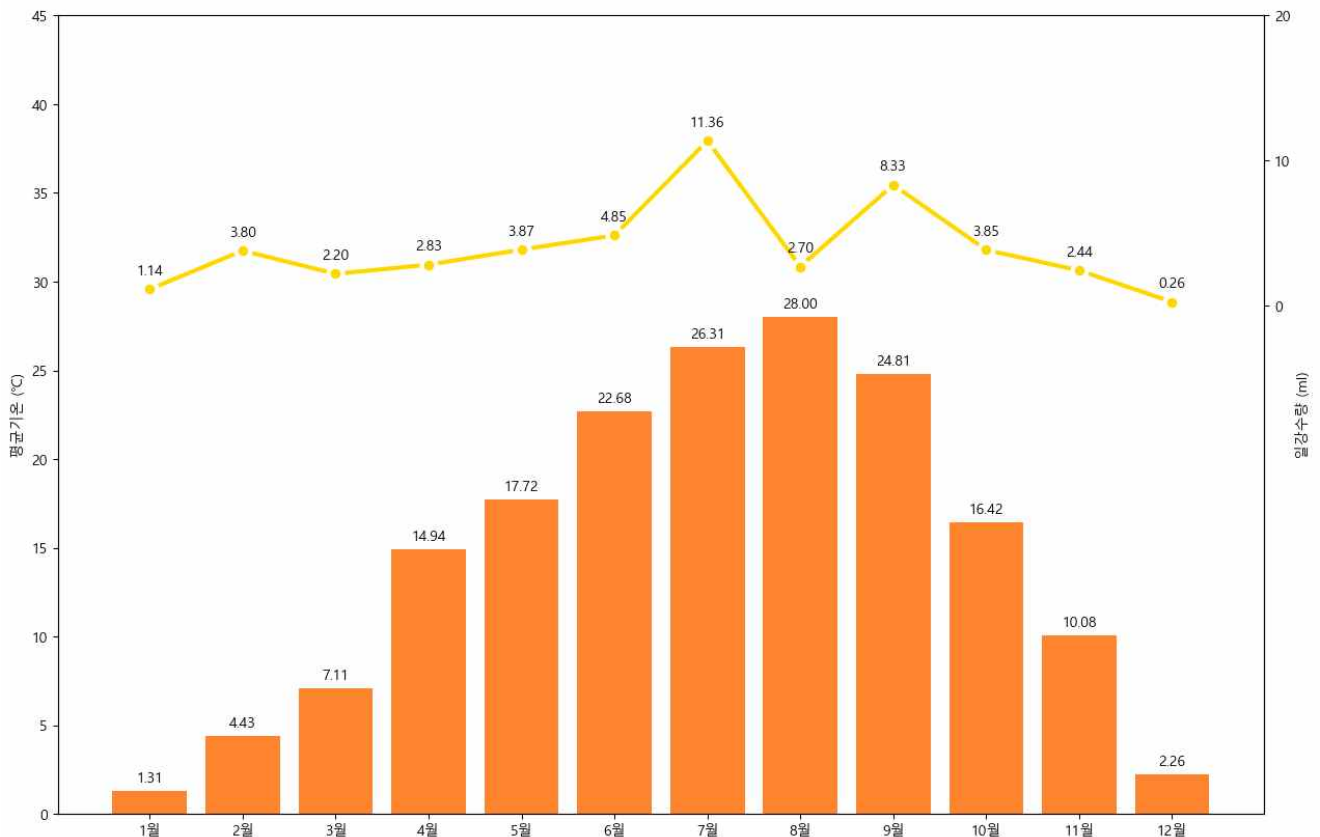
ax1.bar(group_month.월, group_month.평균기온, color='#ff812d')
ax1.set_ylabel('평균기온 (℃)')
ax1.set_ylim(0, 45)
ax1.set_xticks(group_month.월, [f'{x}월' for x in group_month.월])

for idx, val in enumerate(group_month.평균기온):
    plt.text(group_month.loc[idx, '월'], val+0.5, f'{val:.2f}', ha='center')

ax2 = ax1.twinx()
ax2.plot(group_month.월, group_month.일강수량,
         color='#ffd100',
         marker='o', ms=10, lw=3, mec='w', mew=3)
ax2.set_ylim(-35, 20)
ax2.set_yticks([x for x in range(0, 21, 10)])
ax2.set_ylabel('일강수량 (ml)')

for idx, val in enumerate(group_month.일강수량):
    plt.text(group_month.loc[idx, '월'], val+1, f'{val:.2f}', ha='center')
plt.show()
```

2024년 월별 평균기온과 강수량



- 시도별 '평균기온'과 '일강수량'을 구하여 '평균기온'은 막대그래프로 '일강수량'은 꺾은선그래프로 시각화한다.

1) 시도별 '평균기온', '일강수량'의 평균을 구하여 데이터프레임(group_month)에 저장한다.

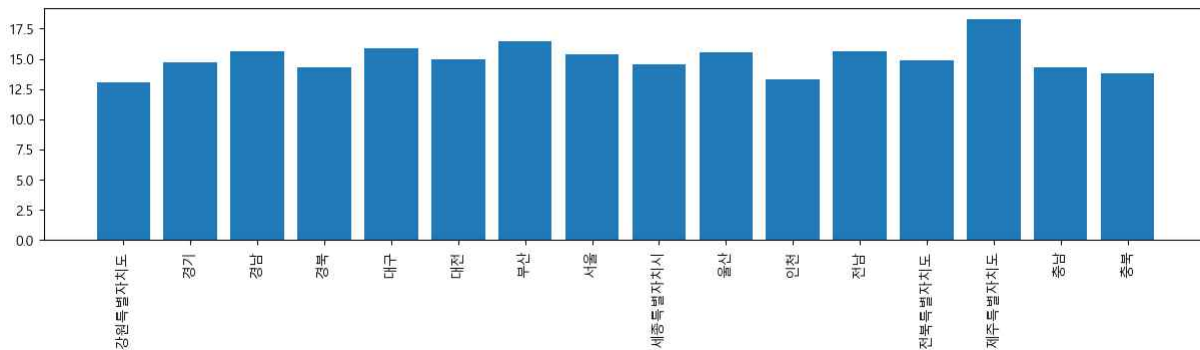
```
group_city = df_temp.groupby('시도', as_index=False)[['평균기온', '일강수량']].mean()
group_city.head(3)
```

	시도	평균기온	일강수량
0	강원특별자치도	13.014425	3.548507
1	경기	14.691829	3.694502
2	경남	15.657240	4.598244

2) 시도별 '평균기온'의 평균을 막대그래프로 시각화 한다.

```
plt.figure(figsize=(15, 3))
plt.bar(group_city.시도, group_city.평균기온)
plt.xticks(group_city.index, rotation=90)
plt.title('2024년 시도별 평균기온\n', size=20)
```

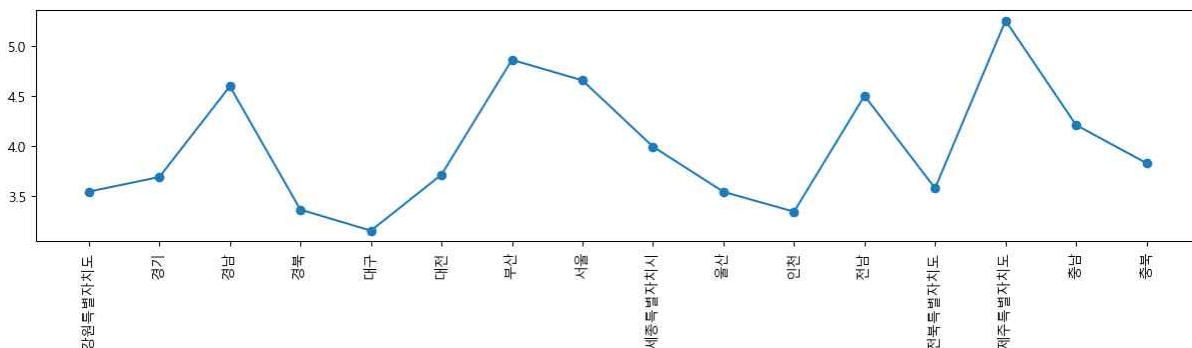
2024년 시도별 평균기온



3) 시도별 '일강수량'의 평균을 꺾은선 그래프로 시각화 한다.

```
plt.figure(figsize=(15, 3))
plt.plot(group_city.시도, group_city.일강수량, marker='o')
plt.xticks(group_city.index, rotation=90)
plt.title('2024년 시도별 평균 일강수량\n', size=20)
```

2024년 시도별 평균 일강수량



4) 시도별 '평균기온'을 막대그래프로, '일강수량'을 꺾은선 그래프로 시각화 한다.

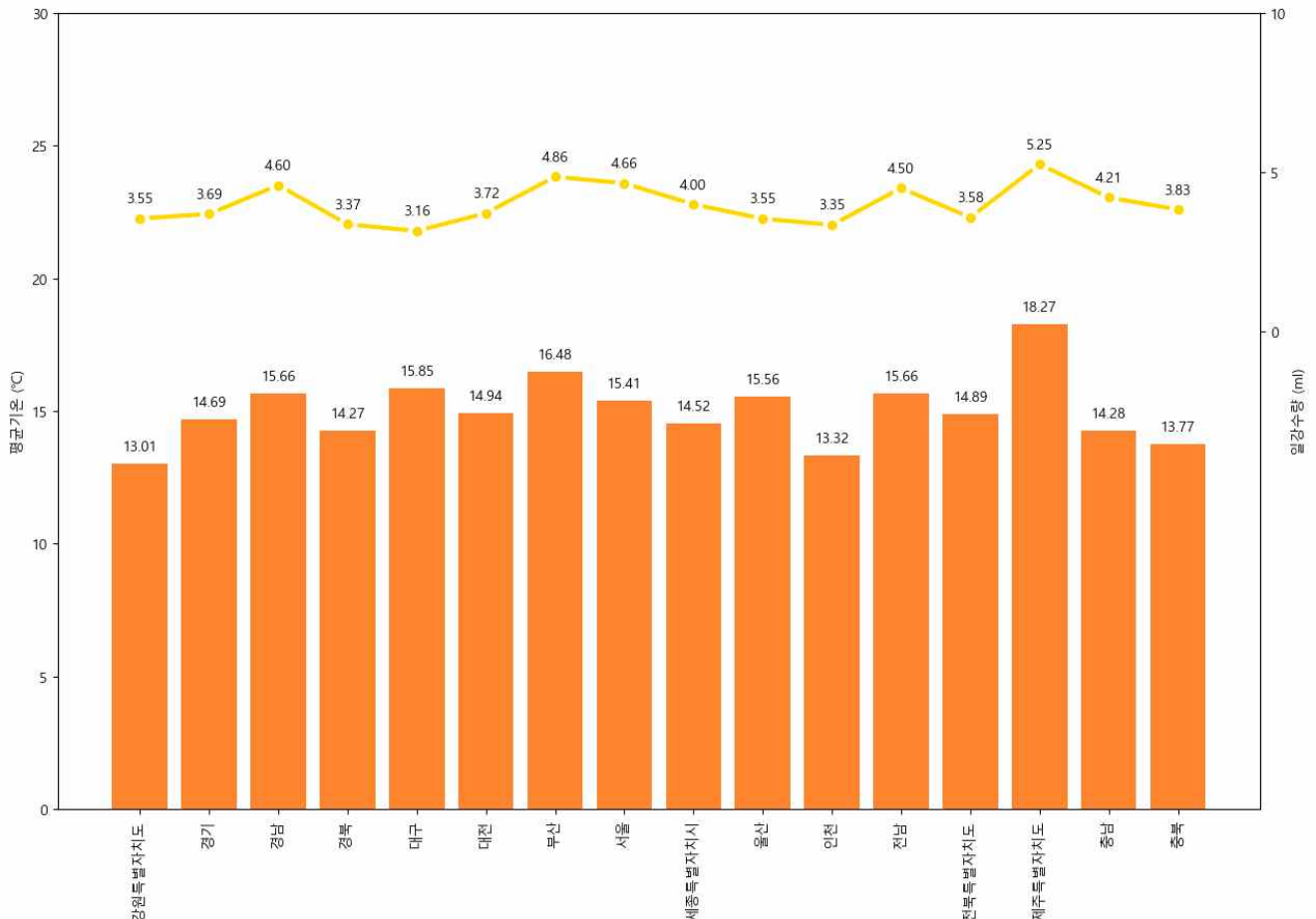
```
fig, ax1 = plt.subplots(figsize=(15, 5))
fig.suptitle('2024년 시도별 평균기온과 일강수량', size=20)

ax1.bar(group_city.시도, group_city.평균기온, color='#ff812d')
ax1.tick_params(axis='x', rotation=90)
ax1.set_ylim(0, 30)
ax1.set_ylabel('평균기온 (°C)')
for idx, val in enumerate(group_city.평균기온):
    ax1.text(idx, val+0.5, f'{val:.2f}', ha='center')

ax2=ax1.twinx()
ax2.plot(group_city.시도, group_city.일강수량, color='#ffd100', marker='o', ms=10, lw=3, mec='w',
mew=3)
ax2.set_ylim(-15, 10)
ax2.set_yticks([y for y in range(0, 11, 5)])
ax2.set_ylabel('일강수량 (ml)')
for idx, val in enumerate(group_city.일강수량):
    ax2.text(idx, val+0.5, f'{val:.2f}', ha='center')

plt.show()
```

2024년 시도별 평균기온과 일강수량



23. 시계열 주식

FinanceDataReader를 활용하면 더 이상 파일데이터로 주가 데이터를 수집해올 필요가 없으며, 가장 최신의 업데이트된 데이터를 API 콜을 통해 DataFrame으로 받아올 수 있다. 컬럼 이름에 대한 설명은 아래와 같다.

컬럼 이름	컬럼 설명
Open	시가
High	고가
Low	저가
Close	종가
Volume	거래량 (매수자와 매도자 간의 거래가 실제로 체결된 주식의 총량)
Change	대비 (주식의 현재 가격이 이전 가격보다 높아지거나 낮아지는 것을 의미)

- 새로운 파일 '23.시계열주식.ipynb'를 생성한다.
- 미국의 Apple 주가 데이터를 불러온다. 기본 오름차순 정렬이 된 데이터임을 알 수 있다.

```
import FinanceDataReader as fdr
apple = fdr.DataReader('AAPL')
apple.head()
```

	Open	High	Low	Close	Volume	Adj Close
1980-12-12	0.128348	0.128906	0.128348	0.128348	469033600	0.098597
1980-12-15	0.122210	0.122210	0.121652	0.121652	175884800	0.093453
1980-12-16	0.113281	0.113281	0.112723	0.112723	105728000	0.086594
1980-12-17	0.115513	0.116071	0.115513	0.115513	86441600	0.088737
1980-12-18	0.118862	0.119420	0.118862	0.118862	73449600	0.091309

```
apple.tail()
```

	Open	High	Low	Close	Volume	Adj Close
2025-06-26	201.429993	202.639999	199.460007	201.000000	50799100	201.000000
2025-06-27	201.889999	203.220001	200.000000	201.080002	73188600	201.080002
2025-06-30	202.009995	207.389999	199.259995	205.169998	91912800	205.169998
2025-07-01	206.669998	210.190002	206.139999	207.820007	78788900	207.820007
2025-07-02	208.910004	213.339996	208.139999	212.440002	67893400	212.440002

- 다음과 같이 2017을 같이 넘겨주면, 해당 시점 이후의 주식 데이터를 가져온다.

```
apple = fdr.DataReader('AAPL', '2017')
apple.head(2)
```

	Open	High	Low	Close	Volume	Adj Close
2017-01-03	28.950001	29.082500	28.690001	29.037500	115127600	26.827250
2017-01-04	28.962500	29.127501	28.937500	29.004999	84472400	26.797220
2017-01-05	28.980000	29.215000	28.952499	29.152500	88774400	26.933495
2017-01-06	29.195000	29.540001	29.117500	29.477501	127007600	27.233757
2017-01-09	29.487499	29.857500	29.485001	29.747499	134247600	27.483204

- 시작과 끝 날짜를 지정하여 범위 데이터를 가져올 수 있다.

```
ford = fdr.DataReader('F', '1980-01-01', '2019-12-30')
ford.head(2)
```

	Open	High	Low	Close	Volume	Adj Close
1979-12-31	1.302678	1.307747	1.262127	1.297609	3916126	0.215914
1980-01-02	1.297609	1.328022	1.287471	1.287471	2044376	0.214227

```
ford.tail(2)
```

	Open	High	Low	Close	Volume	Adj Close
2019-12-26	9.47	9.49	9.43	9.45	28961300	7.227608
2019-12-27	9.45	9.46	9.35	9.36	28272800	7.158772

- 삼성(005930) 주가 데이터를 불러와 시각화 한다.

- 1) 삼성 주가 데이터를 불러와 데이터프레임(samsung)에 저장한다.

```
code='005930'
samsung = fdr.DataReader(code)
samsung.head(2)
```

	Open	High	Low	Close	Volume	Change
Date						
2013-04-15	30060	30699	30060	30460	215346	NaN
2013-04-16	30100	30640	30020	30560	371037	0.003283

```
samsung.tail(2)
```

	Open	High	Low	Close	Volume	Change
Date						
2025-07-02	60200	60900	59800	60800	15047951	0.009967
2025-07-03	61300	63300	61100	63250	25408399	0.040296

- 2) 데이터프레임(samsung)의 index 칼럼 값을 출력한다.

```
samsung.index
```

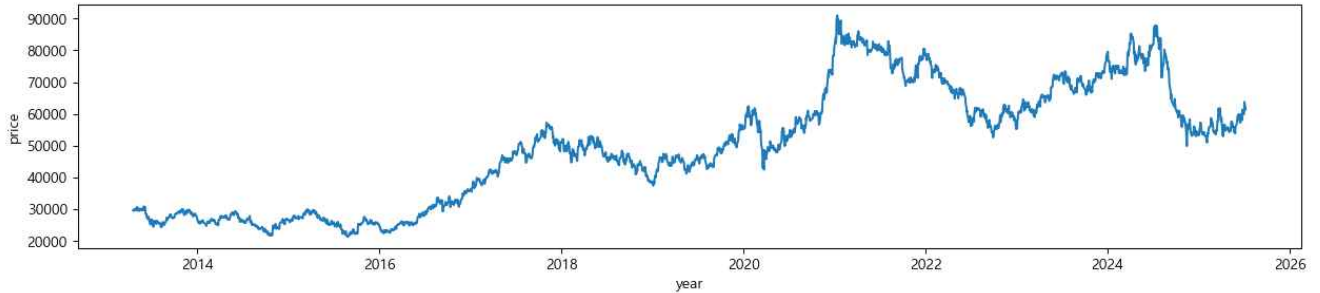
```
DatetimeIndex(['2013-04-15', '2013-04-16', '2013-04-17', '2013-04-18',
... '2025-07-02', '2025-07-03'],
              dtype='datetime64[ns]', name='Date', length=3000, freq=None)
```

- 3) 시각화 작업을 위한 matplotlib 패키지를 import하고 한글을 설정한다.

```
import matplotlib.pyplot as plt
plt.rc('font', family='Malgun Gothic')
plt.rc('font', size= 10)
plt.rc('axes', unicode_minus=False)
```

4) seaborn 패키지를 사용하여 시각화 작업을 한다. seaborn은 matplotlib을 기반으로 다양한 색상 테마와 통계용 차트 등의 기능을 추가한 시각화 패키지이다.

```
import seaborn as sns
plt.figure(figsize=(15, 4))
sns.lineplot(x=samsung.index, y=samsung['Close'])
plt.xlabel('year')
plt.ylabel('price')
```



■ 위 그래프의 X축 눈금을 6개월 단위로 X축 눈금 레이블을 년 단위로 출력되도록 xticks 옵션을 추가한다.

1) 데이터프레임(samsung)에 Year, Month, Day 칼럼을 추가한다.

```
samsung['Year'] = samsung.index.year
samsung['Month'] = samsung.index.month
samsung['Day'] = samsung.index.day
```

	Open	High	Low	Close	Volume	Change	Year	Month	Day
Date									
2013-04-18	30400	30440	29619	29660	267168	NaN	2013	4	18
2013-04-19	29299	29839	29100	29500	314080	-0.005394	2013	4	19

2) 데이터프레임(samsung)을 Year, Month 칼럼으로 그룹화해서 'Day' 칼럼의 최솟값을 구한다.

```
group = samsung.groupby(['Year', 'Month'], as_index=False)['Day'].min()
```

	Year	Month	Day
0	2013	4	18
1	2013	5	2

3) Year, Month, Day 타입을 String으로 변환한 후 '-'로 연결하여 Date 타입으로 변환하여 다시 String 타입으로 변환 한다

```
import pandas as pd
first = group.copy()
first['Date']=first['Year'].astype(str) + '-' + first['Month'].astype(str) + '-' + first['Day'].astype(str)
first = pd.to_datetime(first['Date']).astype(str)
first = list(first)
first[:5]
```

```
['2013-04-18', '2013-05-02', '2013-06-03', '2013-07-01', '2013-08-01']
```

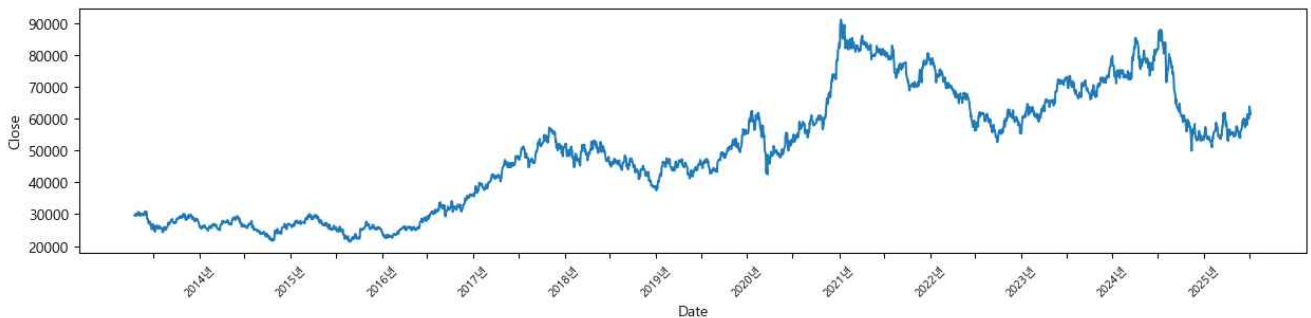
4) 데이터프레임(samsung)의 index 값이 first 배열에 존재하면서 '01'월 이나 '07'월이면 리턴 한다.

```
xticks = [x for x in samsung.index.astype(str) if (x in first) & ((x[5:7]=='01') | (x[5:7]=='07'))]

['2013-04-18', '2013-05-02', '2013-06-03', '2013-07-01', '2013-08-01']
```

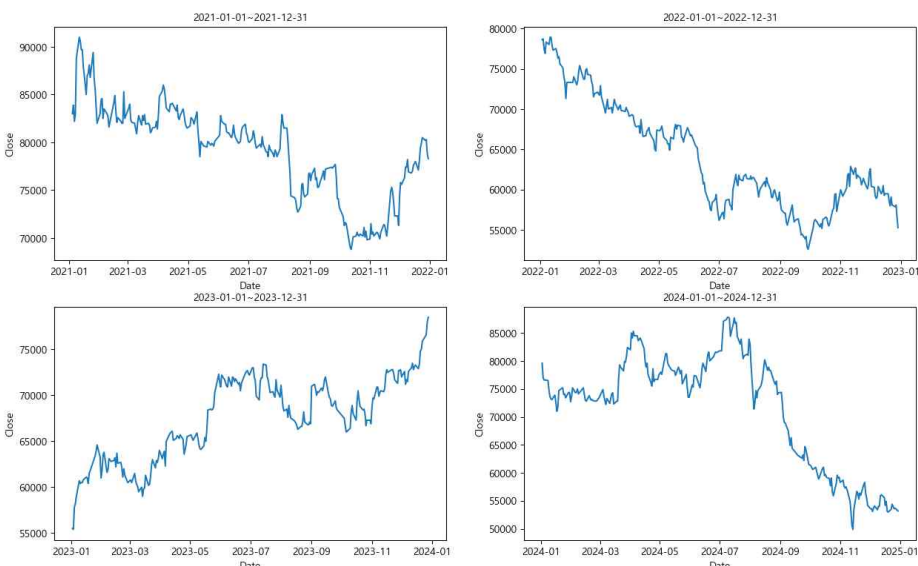
5) X축 눈금을 6개월 단위로 X축 눈금 레이블을 년 단위로 출력하는 그래프를 시각화한다.

```
plt.figure(figsize=(15, 3))
sns.lineplot(x=samsung.index, y=samsung['Close'])
plt.xticks(xticks, [f'{x[0:4]}년' if (x[5:7]=='01') else '' for x in xticks], rotation=45, size=8)
plt.show()
```



▪ 2021년, 2022년, 2023년, 2024년 여러 개의 그래프를 한 화면에 시각화한다.

```
year_steps=[['2021-01-01', '2021-12-31'], ['2022-01-01', '2022-12-31'],
            ['2023-01-01', '2023-12-31'], ['2024-01-01', '2024-12-31']]
fig, axes = plt.subplots(2, 2)
fig.set_size_inches(15, 10)
for i in range(4):
    ax = axes[i//2, i%2]
    filt = (samsung.index >= year_steps[i][0]) & (samsung.index <= year_steps[i][1])
    df = samsung.loc[filt]
    sns.lineplot(x=df.index, y=df['Close'], ax=ax)
    ax.set_title(f'\n{year_steps[i][0]}~{year_steps[i][1]}')
```



- 삼성전자(005930) '2015-01-01'~'2025-01-01' 10년간의 주가 데이터를 불러와 시각화 작업한다.

1) 삼성전자 특정 기일의 주가 데이터를 읽어 데이터프레임(samsung)에 저장한다.

```
code='005930'
start ='2015-01-01'
end = '2025-01-01'
samsung = fdr.DataReader(code, start, end)
samsung.head(3)
```

	Open	High	Low	Close	Volume	Change
Date						
2015-01-02	26799	26799	26540	26600	175499	0.002261
2015-01-05	26720	26720	26260	26660	202790	0.002256
2015-01-06	26300	26340	25760	25900	304710	-0.028507

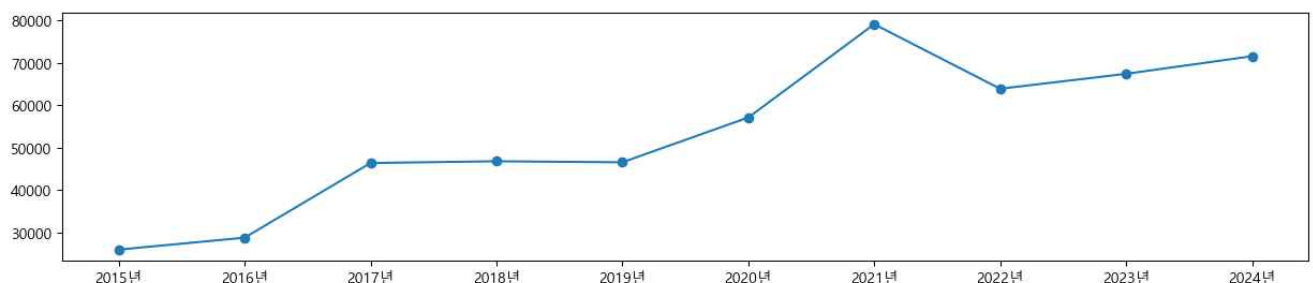
2) 특정 칼럼을 추출하고 '년', '월', '일' 칼럼을 새로 추가하여 새로운 데이터프레임(stock)에 저장한다.

```
stock = samsung.copy()
stock = stock.iloc[:, :-1]
stock.columns = ['시가', '고가', '저가', '종가', '거래량']
stock['년'] = stock.index.year
stock['월'] = stock.index.month
stock['일'] = stock.index.day
stock.head(3)
```

	시가	고가	저가	종가	거래량	년	월	일
Date								
2015-01-02	26799	26799	26540	26600	175499	2015	1	2
2015-01-05	26720	26720	26260	26660	202790	2015	1	5
2015-01-06	26300	26340	25760	25900	304710	2015	1	6

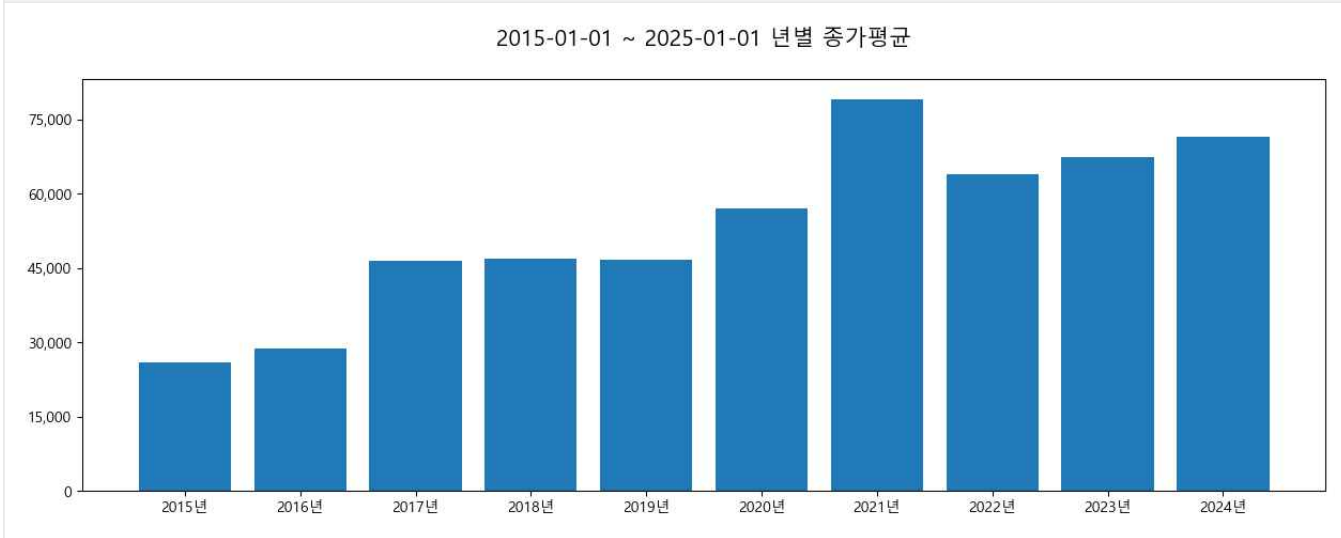
3) '년' 칼럼으로 그룹화한 후 '종가' 칼럼의 평균을 구해 꺾은선 그래프로 시각화한다.

```
group = stock.groupby('년')['종가'].mean()
plt.figure(figsize=(15, 3))
plt.plot(group.index, group.values, marker='o')
plt.xticks(group.index, labels=[f'{x}년' for x in group.index])
plt.show()
```



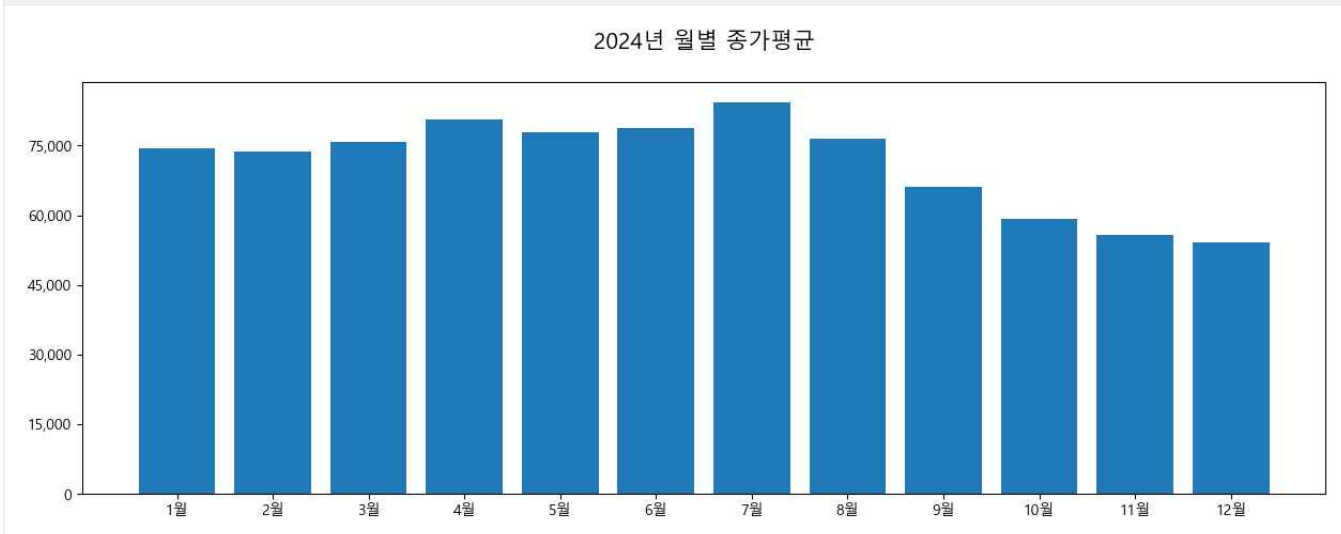
4) 위 그룹화 한 데이터프레임(group)을 막대 그래프로 시각화 한다.

```
plt.figure(figsize=(15, 5))
plt.bar(group.index, group.values)
plt.xticks(group.index, labels=[f'{x}년' for x in group.index])
plt.title(f'{start} ~ {end} 년별 증가평균\n', size=15)
yticks = [y for y in range(0, 80000, 15000)]
plt.yticks(yticks, [f'{y:,}' for y in yticks])
plt.show()
```



5) 데이터프레임(stock)에서 '2024'년 데이터를 추출하여 '년'으로 그룹화한 후 '증가' 평균을 막대 그래프로 시각화한다.

```
filt = stock['년'] == 2024
group = stock[filt].groupby('월')['증가'].mean()
plt.figure(figsize=(10, 5))
plt.bar(group.index, group.values)
plt.xticks(group.index, labels=[f'{x}월' for x in group.index])
plt.title(f'2024년 월별 증가평균\n', size=15)
yticks = [y for y in range(0, 80000, 15000)]
plt.yticks(yticks, [f'{y:,}' for y in yticks])
plt.show()
```

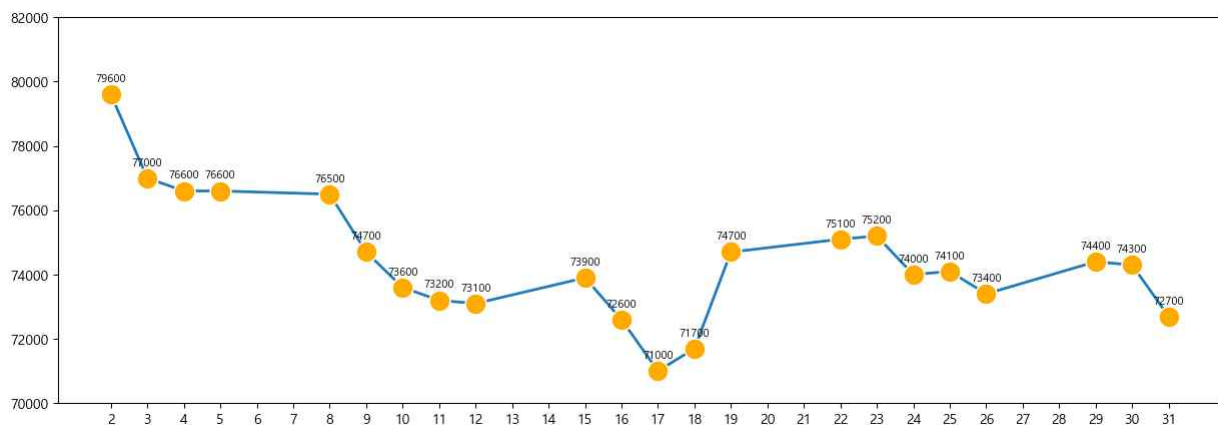


6) 데이터프레임(stock)에서 '2024'년 '1'월 데이터를 추출하여 꺾은선 그래프와 막대 그래프로 시각화 한다.

```
filt = (stock['년']==2024) & (stock['월']==1)
group = stock[filt]

plt.figure(figsize=(15, 5))
plt.plot(group['일'], group['종가'], marker='o', mfc='orange', lw=2, ms=15, mec='w', alpha=1)
plt.ylim(70000, 82000)
plt.xticks([x for x in range(group['일'].min(), group['일'].max()+1, 1)])

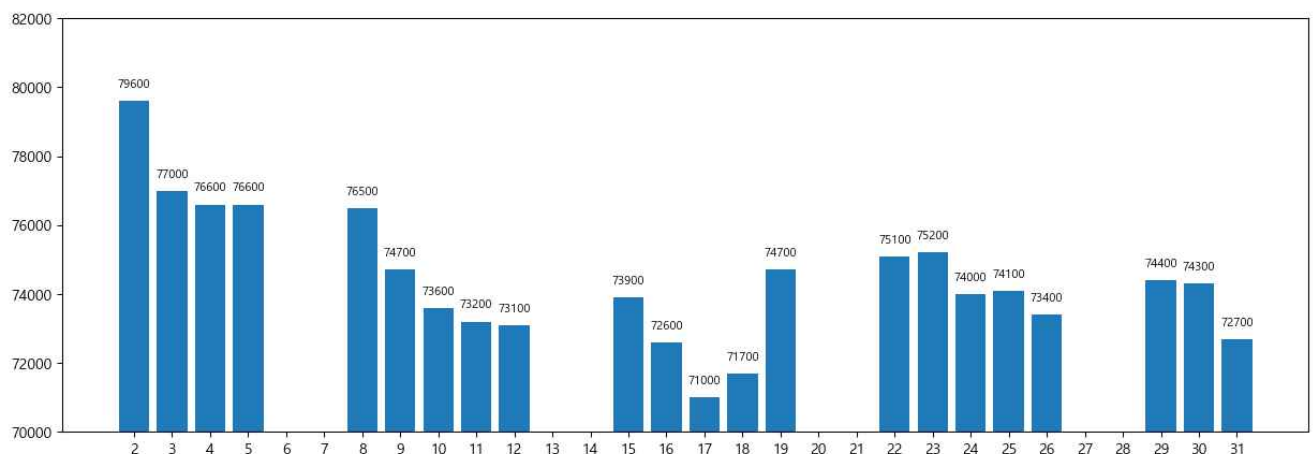
for idx, close in enumerate(group['종가']):
    plt.text(group.iloc[idx,-1], close+400, close, ha='center', size=8)
plt.show()
```



```
filt = (stock['년']==2024) & (stock['월']==1)
group = stock[filt]

plt.figure(figsize=(15, 5))
plt.bar(group['일'], group['종가'])
plt.ylim(70000, 82000)
plt.xticks([x for x in range(group['일'].min(), group['일'].max()+1, 1)])

for idx, close in enumerate(group['종가']):
    plt.text(group.iloc[idx,-1], close+400, close, ha='center', size=8)
plt.show()
```



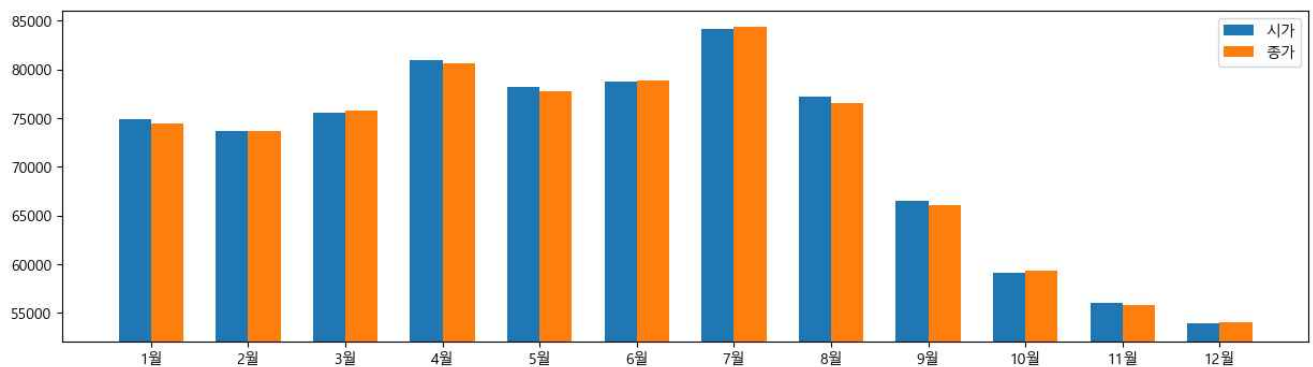
7) 데이터프레임(stock)에서 '2024'년 데이터를 추출하여 월별 '시가', '종가' 평균을 구해 막대그래프로 시각화 한다.

```

filt = stock['년'] == 2024
group = stock[filt].groupby('월')[['시가','종가']].mean()
group.reset_index(inplace=True)
group['x'] = group.index * 3

plt.figure(figsize=(15, 4))
plt.bar(group['x'], group['시가'], width=1, label='시가')
plt.bar(group['x']+1, group['종가'], width=1, label='종가')
plt.ylim(52000, 86000)
plt.xticks([x+0.5 for x in group['x']], labels=[f'{x}월' for x in group['월']])
plt.legend()

```



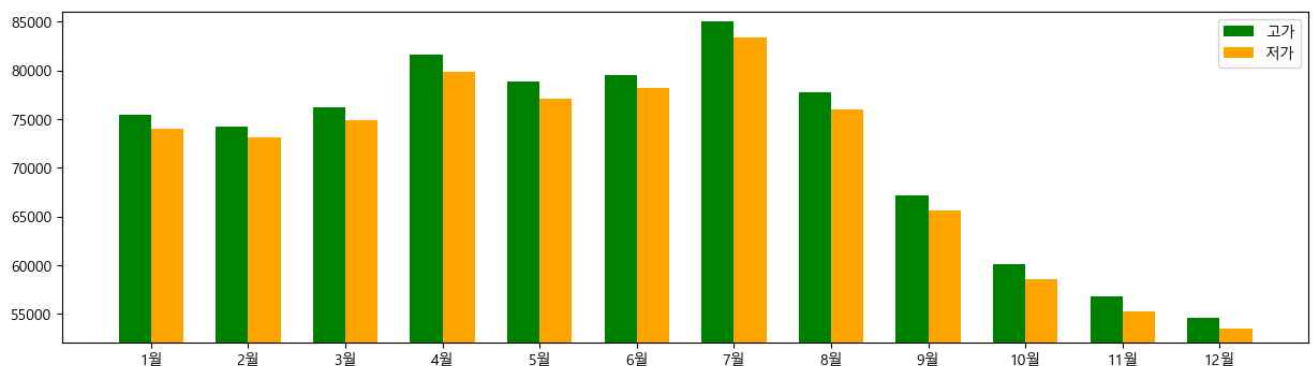
8) 데이터프레임(stock)에서 '2024'년 데이터를 추출하여 월별 '고가', '저가' 평균을 구해 막대그래프로 시각화 한다.

```

filt = stock['년'] == 2024
group = stock[filt].groupby('월')[['고가','저가']].mean()
group.reset_index(inplace=True)
group['x'] = group.index * 3

plt.figure(figsize=(15, 4))
plt.bar(group['x'], group['고가'], width=1, label='고가')
plt.bar(group['x']+1, group['저가'], width=1, label='저가')
plt.ylim(52000, 86000)
plt.xticks([x+0.5 for x in group['x']], labels=[f'{x}월' for x in group['월']])
plt.legend()

```



9) 데이터프레임(stock)의 '2024'년 데이터를 추출하여 월별 '증가'평균은 꺾은선그래프로 '거래량'은 막대그래프로 시각화한다.

```

filt = stock['년'] == 2024
group = stock[filt].groupby('월')[['증가','거래량']].mean()

fig, ax1 = plt.subplots(figsize=(12, 6))
ax1.bar(group.index, group['증가'], color='#FF812D')

ax1.set_ylim(50000, 100000)
yticks = [y for y in range(50000, 100001, 10000)]
ax1.set_yticks(yticks, labels=[f'{y:,'}월' for y in yticks])
ax1.set_ylabel('증가')

for idx, close in enumerate(group['증가']):
    plt.text(group.index[idx], close+500, f'{int(close):,'}', ha='center', size=9)

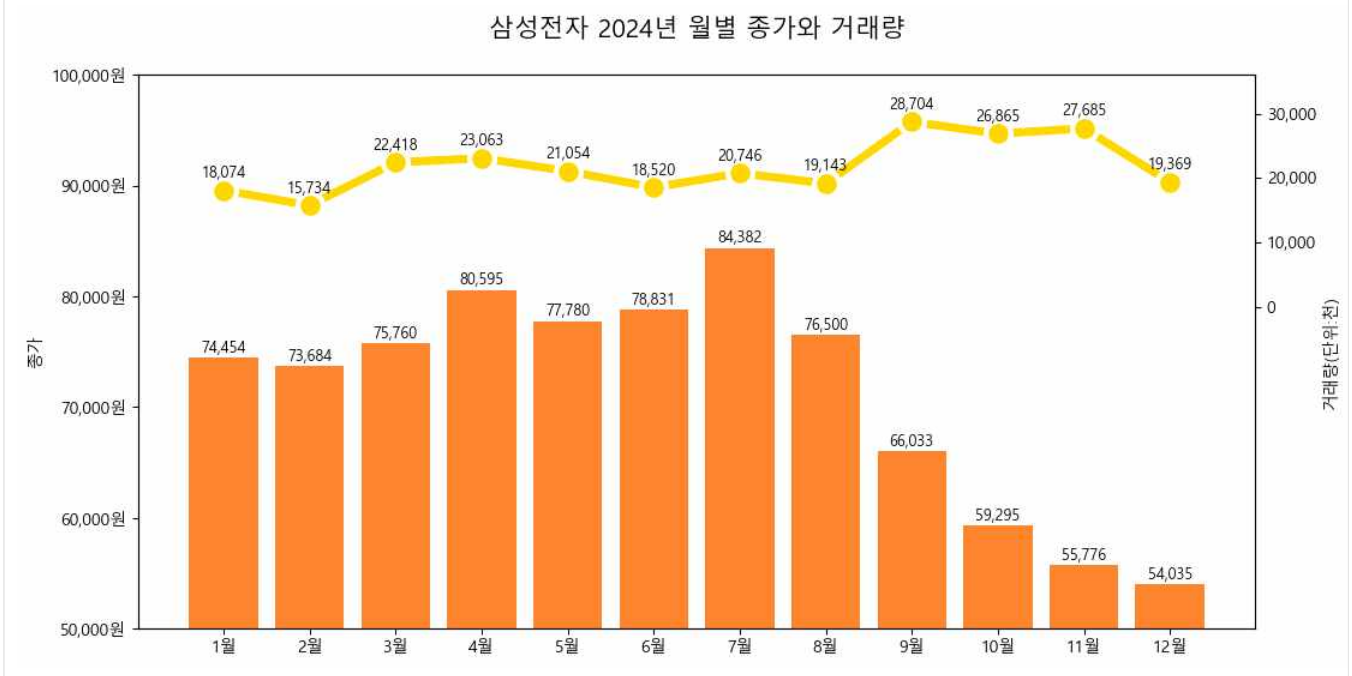
ax2 = ax1.twinx()
ax2.plot(group.index, group['거래량']/1000,
         color='#FFD100', marker='o', ms=15, mec='w', mew=3, lw=5)

ax2.set_ylim(-50000, 36000)
yticks = [y for y in range(0, 36001, 10000)]
ax2.set_yticks(yticks, labels=[f'{y:,'}' for y in yticks])
ax2.set_ylabel('거래량(단위:천)')

for idx, volumn in enumerate(group['거래량']):
    plt.text(group.index[idx], volumn/1000+2000, f'{int(volumn/1000):,'}', ha='center', size=9)

plt.xticks(group.index, labels=[f'{x}월' for x in group.index])
plt.title('삼성전자 2024년 월별 증가와 거래량\n', size=15)
plt.show()

```



- '2025-01-01' ~ 오늘까지 삼성 주가 데이터를 읽어 날짜별 종가(Close)를 찍은선 그래프로 출력한다.

1) '2025-01-01' ~ 오늘까지 삼성 주가 데이터를 읽어 데이터프레임(df_stock)에 저장한다.

```
from datetime import datetime
today = datetime.now().strftime('%Y-%m-%d')
today
```

```
'2025-07-02'
```

```
start = '2025-01-01'
end = today
df_stock = fdr.DataReader('005930', start, end)
df_stock.head(2)
```

	Open	High	Low	Close	Volume	Change
Date						
2025-01-02	52700	53600	52300	53400	16630538	0.003759
2025-01-03	52800	55100	52800	54400	19318046	0.018727

```
df_stock.tail(2)
```

	Open	High	Low	Close	Volume	Change
Date						
2025-07-01	60400	61100	60200	60200	13650991	0.006689
2025-07-02	60200	60900	59800	60800	14761799	0.009967

2) 'Year', 'Month', 'Day' 칼럼을 추가한다.

```
df_stock['Year'] = df_stock.index.year
df_stock['Month'] = df_stock.index.month
df_stock['Day'] = df_stock.index.day
df_stock.head(2)
```

	Open	High	Low	Close	Volume	Change	Year	Month	Day
Date									
2025-01-02	52700	53600	52300	53400	16630538	0.003759	2025	1	2
2025-01-03	52800	55100	52800	54400	19318046	0.018727	2025	1	3

3) 그래프 출력을 위한 matplotlib 라이브러리를 import하고 한글을 설정한다.

```
import matplotlib.pyplot as plt
plt.rc('font', family='Malgun Gothic')
plt.rc('font', size=10)
plt.rc('axes', unicode_minus=False)
```

4) 각 년, 월에 해당되는 첫 번째 날짜를 구한다.

```
min = df_stock.groupby(['Year', 'Month'], as_index=False)['Day'].min()
min
```

	Year	Month	Day
0	2025	1	2
1	2025	2	3
2	2025	3	4
3	2025	4	1
4	2025	5	2
5	2025	6	2
6	2025	7	1

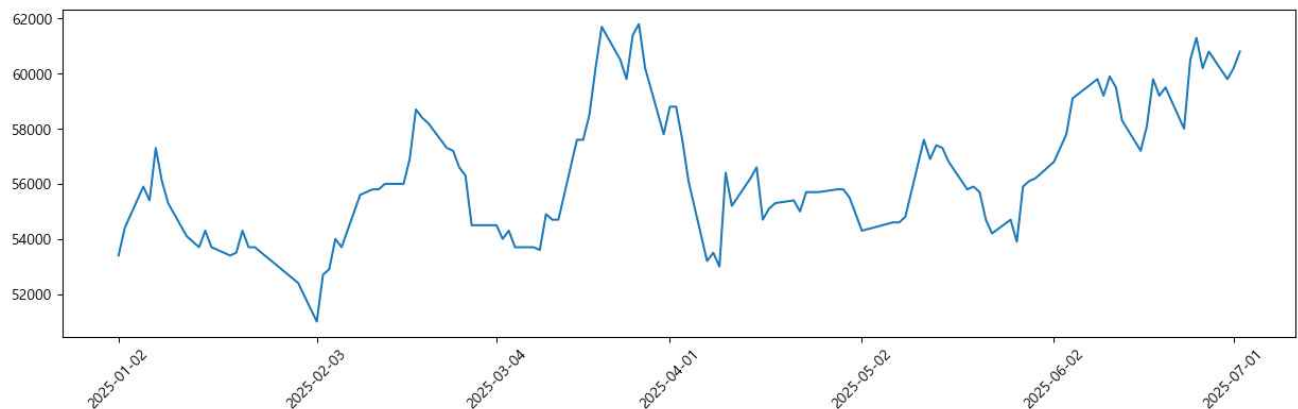
5) Year, Month, Day를 연결 후 데이트 타입으로 변경 후 다시 String 타입으로 변경한다.

```
import pandas as pd
first = min.copy()
first['Date'] = first['Year'].astype(str)+'-'+first['Month'].astype(str)+'-'+first['Day'].astype(str)
first = pd.to_datetime(first['Date']).astype(str)
first = list(first)
first
```

```
['2025-01-02',
 '2025-02-03',
 '2025-03-04',
 '2025-04-01',
 '2025-05-02',
 '2025-06-02',
 '2025-07-01']
```

6) 날짜별 종가의 변화를 시각화 한다.

```
plt.figure(figsize=(15, 4))
plt.plot(df_stock['Close'])
xticks = [x for x in df_stock.index.astype(str) if x in first]
plt.xticks(xticks, rotation=45)
plt.show()
```



- '삼성전자', '카카오', 'NAVER', 'LG전자', '현대차' 최근 시가, 고가, 저가를 막대 그래프로 시각화한다.

1) 한국증권거래소 종목리스트에서 my_names 항목 주식데이터를 데이터프레임(df_krx_my) 변수에 저장한다.

```
import FinanceDataReader as fdr
df_krx = fdr.StockListing('KRX')
my_names = ['삼성전자', '카카오', 'NAVER', 'LG전자', '현대차']
filt = df_krx['Name'].isin(my_names)
df_krx_my = df_krx[filt][['Name', 'Code', 'Open', 'High', 'Low', 'Volume']]
df_krx_my
```

	Name	Code	Open	High	Low	Volume
0	삼성전자	005930	60200	60900	59800	15047951
4	현대차	005380	215000	218000	210000	1058819
7	NAVER	035420	263000	263500	250000	1870730
16	카카오	035720	61000	61300	58100	2974555
45	LG전자	066570	75400	76200	74600	419986

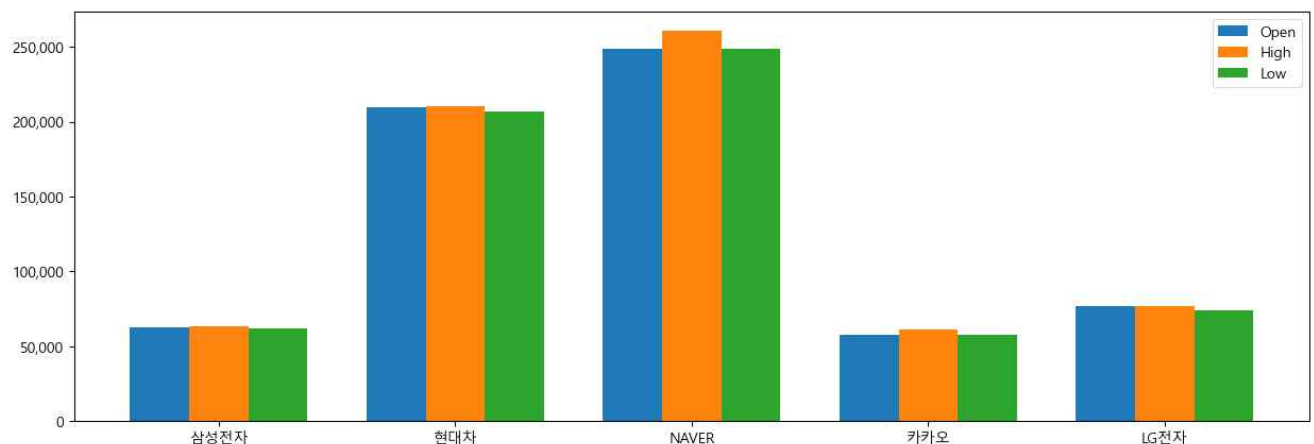
2) x축 눈금을 생성하여 xticks 변수에 저장하고 y축 최댓값을 구하여 max 변수에 저장한다.

```
import numpy as np
xticks = np.arange(len(my_names)) * 4
max = df_krx_my['High'].max()
```

```
(array([ 0,  4,  8, 12, 16]), 263500)
```

3) my_names 항목들의 시가, 고가, 저가를 막대그래프로 시각화한다.

```
plt.figure(figsize=(10,5))
plt.bar(xticks+0, df_krx_my['Open'], width=1, label='Open')
plt.bar(xticks+1, df_krx_my['High'], width=1, label='High')
plt.bar(xticks+2, df_krx_my['Low'], width=1, label='Low')
plt.xticks([xtick+1 for xtick in xticks], labels=df_krx_my['Name'])
yticks = [y for y in range(0, max, 50000)]
plt.yticks(yticks, labels=[f'{y:,'} for y in yticks])
plt.legend()
```



- my_names 항목들의 증가(Close)를 꺾은선 그래프로 시각화한다.

1) 데이터프레임(df_krx_my)에서 'Name' 칼럼 값과 'Code' 칼럼 값을 구하여 각각 변수에 저장한다.

```
my_names = list(df_krx_my['Name'])
my_codes = list(df_krx_my['Code'])
my_codes, my_names

(['005930', '005380', '035420', '035720', '066570'],
 ['삼성전자', '현대차', 'NAVER', '카카오', 'LG전자'])
```

2) my_codes 항목들의 '2025-01-01'~ 현재까지 주식 데이터를 데이터프레임(df_stock)에 저장한다.

```
df_stock = fdr.DataReader(my_codes, start, end)
df_stock.tail(2)
```

	005930	005380	035420	035720	066570
Date					
2025-07-01	60200	210000	262000	60100	75400
2025-07-02	60800	213500	252000	58700	76200

```
df_stock.columns=my_names
df_stock.head(2)
```

	삼성전자	현대차	NAVER	카카오	LG전자
Date					
2025-01-02	53400	211500	193800	37450	84000
2025-01-03	54400	213000	200500	38050	85100

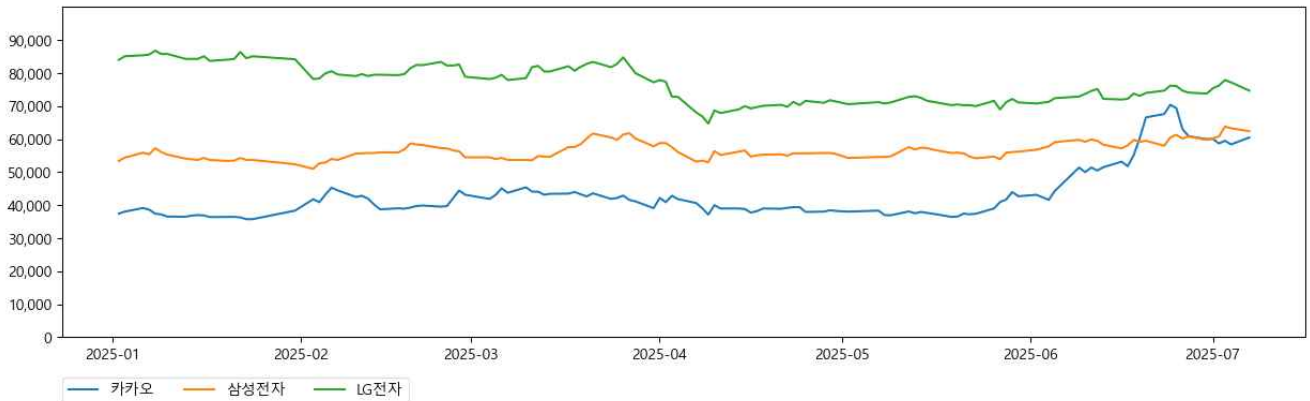
3) my_codes 항목들의 증가를 날짜별 꺾은선 그래프로 시각화한다.

```
df = df_stock.copy()
plt.figure(figsize=(15, 4))
for col in df.columns:
    plt.plot(df.index, df[col], label=col, ms=3)
plt.legend(loc=(0, 1.02), ncol=5)
yticks = [y for y in range(0, 300001, 50000)]
plt.yticks(yticks, [f'{y:,}' for y in yticks])
plt.show()
```



4) 증가가 비슷한 '카카오', '삼성전자', 'LG전자' 항목의 증가를 꺾은선 그래프로 시각화 한다.

```
df = df_stock[['카카오', '삼성전자', 'LG전자']]
plt.figure(figsize=(15, 4))
for col in df.columns:
    plt.plot(df.index, df[col], label=col, ms=3)
yticks = [y for y in range(0, 100000, 10000)]
plt.ylim(0, 100000)
plt.yticks(yticks, [f'{y:,}' for y in yticks])
plt.legend(loc=(0, -0.2), ncols=3)
plt.show()
```



▪ my_codes 항목들의 주식데이터를 년, 월별 Volume의 합계를 막대그래프로 시각화 한다.

1) my_codes 항목들의 주가 데이터를 읽어 데이터프레임(df_stock_all)에 저장한다.

```
df_stock_all = pd.DataFrame()
for idx, code in enumerate(my_codes):
    df = fdr.DataReader(code, start, end)
    df['Code'] = code
    df['Name'] = my_names[idx]
    df_stock_all = pd.concat([df_stock_all, df], axis=0)
```

df_stock_all.shape

(600, 8)

df_stock_all.head(2)

	Open	High	Low	Close	Volume	Change	Code	Name
Date								
2025-01-02	52700	53600	52300	53400	16630538	0.003759	005930	삼성전자
2025-01-03	52800	55100	52800	54400	19318046	0.018727	005930	삼성전자
2025-01-06	54400	56200	54300	55900	19034284	0.027574	005930	삼성전자
2025-01-07	56800	57300	55400	55400	17030235	-0.008945	005930	삼성전자
2025-01-08	54800	57500	54700	57300	26593553	0.034296	005930	삼성전자

df_stock_all.tail(2)

	Open	High	Low	Close	Volume	Change	Code	Name
Date								
2025-06-26	76200	76200	74300	74700	393009	-0.018397	066570	LG전자
2025-06-27	74800	75200	73700	74100	434975	-0.008032	066570	LG전자
2025-06-30	74300	74800	73700	73800	422420	-0.004049	066570	LG전자
2025-07-01	74300	76800	74200	75400	655170	0.021680	066570	LG전자
2025-07-02	75400	76200	74600	76200	419986	0.010610	066570	LG전자

2) Year, Month, Day를 구하여 새로운 칼럼을 추가한다.

```
df_stock_all['Year'] = df_stock_all.index.year
df_stock_all['Month'] = df_stock_all.index.month
df_stock_all['Day'] = df_stock_all.index.day
```

df_stock_all.head(3)

	Open	High	Low	Close	Volume	Change	Code	Name	Year	Month	Day
Date											
2025-01-02	52700	53600	52300	53400	16630538	0.003759	005930	삼성전자	2025	1	2
2025-01-03	52800	55100	52800	54400	19318046	0.018727	005930	삼성전자	2025	1	3
2025-01-06	54400	56200	54300	55900	19034284	0.027574	005930	삼성전자	2025	1	6

df_stock_all.tail(3)

	Open	High	Low	Close	Volume	Change	Code	Name	Year	Month	Day
Date											
2025-06-26	76200	76200	74300	74700	393009	-0.018397	066570	LG전자	2025	6	26
2025-06-27	74800	75200	73700	74100	434975	-0.008032	066570	LG전자	2025	6	27
2025-06-30	74300	74800	73700	73800	422420	-0.004049	066570	LG전자	2025	6	30

3) 'Year', 'Month', 'Code', 'Name'으로 그룹화 하여 'Volume'의 합계를 구해 데이터프레임(group)에 저장한다.

```
group = df_stock_all.groupby(['Year', 'Month', 'Code', 'Name'], as_index=False)[['Volume']].sum()
group.head()
```

	Year	Month	Code	Name	Volume
0	2025	1	005380	현대차	14102248
1	2025	1	005930	삼성전자	342890865
2	2025	1	035420	NAVER	11863306
3	2025	1	035720	카카오	33278622
4	2025	1	066570	LG전자	8726323

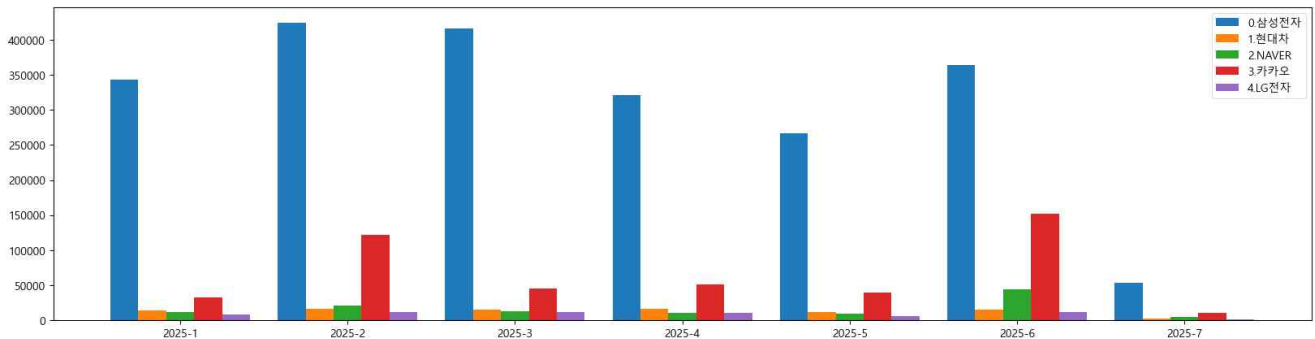
4) 데이터프레임(group)를 my_codes별로 분류하여 새로운 데이터프레임 배열(df_all)에 추가한다.

```
df_all = []
for idx, code in enumerate(my_codes):
    filt = group['Code'] == code
    df = group[filt]
    df_all.append(df)
```

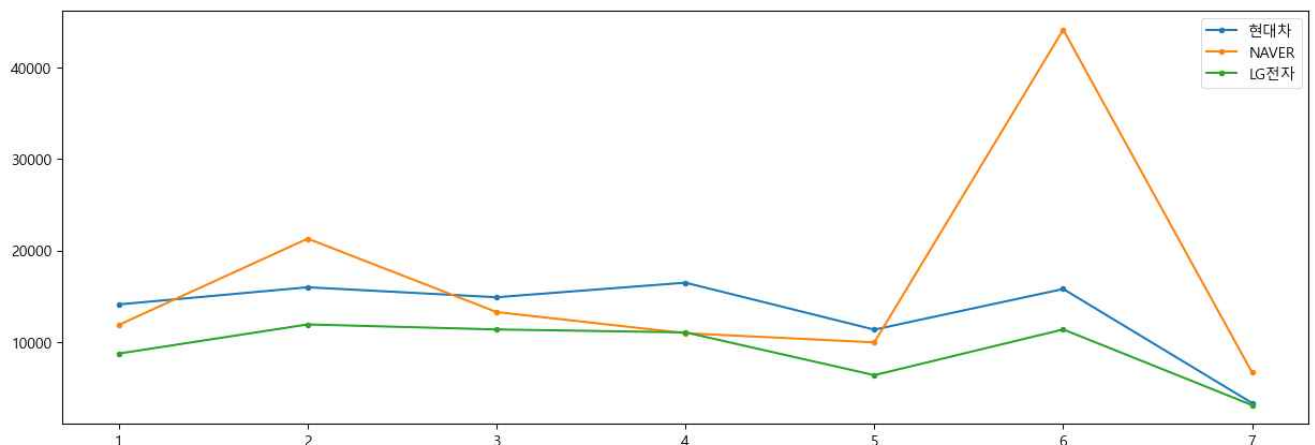
5) my_codes 항목의 'Volume' 합계를 년, 월별로 시각화한다.

```
import numpy as np
x = (len(my_codes)+1) * np.arange(len(df_all[0]))

plt.figure(figsize=(10, 5))
for idx in range(len(my_codes)):
    df = df_all[idx]
    plt.bar(x + idx, df['Volume']/1000, width=1, label=f'{idx}.{my_names[idx]}')
xticks = [xticks+2 for xticks in x]
xlabels = [f'{df_all[0].loc[x, "Year"]}-{df_all[0].loc[x, "Month"]}' for x in df_all[0].index]
plt.xticks(xticks, xlabels)
plt.legend()
```



```
plt.figure(figsize=(15, 5))
for idx in [1, 2, 4]:
    df = df_all[idx]
    plt.plot(df['Month'], df['Volume']/1000, marker='o', ms=3, label=my_names[idx])
plt.legend()
```



• Flask

'Flask' 는 소규모의 애플리케이션을 빠르게 만들 수 있고, 배포 환경에 따라 대규모 애플리케이션의 기능 확장의 역할을 하기 쉬운 장점이, 'Django'는 대규모의 애플리케이션을 빠르게 만들 수 있으며, 기본으로 제공해 주는 기능이 많은 장점이 있다.

- 가상머신을 생성한다.

```
C:\data\python\flask>python -m venv myenv
```

- 가상머신을 실행한다. (실행종료: deactivate)

```
C:\data\python\flask>.\myenv\Scripts\activate
```

- 플라스크(Flask) 패키지를 설치한다.

```
(myenv) C:\data\python\flask>pip install flask
```

1) 홈페이지 Layout을 작성한다.

[web] app.py

```
from flask import Flask, render_template
from routes import student

app = Flask(__name__, template_folder='templates')
app.register_blueprint(student.bp)

@app.route('/')
def index():
    return render_template('index.html', title='홈페이지', pageName='home.html')

if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

[web]-[templates] index.html

```
<html>
<head>
    <link rel="stylesheet" href="/static/css/style.css"/>
    <script src="http://code.jquery.com/jquery-1.9.1.js"></script>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/css/bootstrap.min.css" rel="stylesheet">
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/js/bootstrap.bundle.min.js"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/handlebars.js/3.0.1/handlebars.js"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/twbs-pagination/1.4.2/jquery.twbsPagination.min.js"></script>
    <title>{{title}}</title>
</head>
<body>
    <div class="container">
        {%include 'header.html'%}
        {%include 'menu.html'%}
        {%include pageName%}
        {%include 'footer.html'%}
    </div>
</body>
</html>
```

[web]-[templates] header.html

```
<div>
  
</div>
```

[web]-[templates] menu.html

```
<div class="mt-5">
  <a href="/" class="me-3">Home</a>
  <a href="/student/search" class="me-3">학생검색</a>
  <a href="/student/graph" class="me-3">학생그래프</a>
  <a href="/student/info" class="me-3">학생정보</a>
  <hr>
</div>
```

[web]-[templates] home.html

```
<div class="my-5">
  <h1 class="text-center">홈페이지</h1>
</div>
```

[web]-[templates] footer.html

```
<div class="my-5" style="margin-bottom: 300px;">
  <hr>
  <h4 class="text-center">Copyright 2023 홍길동 All rights reserved.</h4>
</div>
```

2) 스타일 파일을 작성한다.

[web]-[static]-[css] style.css

```
@import url('https://fonts.googleapis.com/css2?family=Sunflower:wght@300&display=swap');

* {
  font-family: "Sunflower", sans-serif;
  font-weight: 300;
  font-style: normal;
}
```

3) 웹서버를 실행하고 localhost:5000에 접속한다.

```
(myenv) C:\data\python\flask>python app.py
```

4) 학생 검색 페이지 출력을 위한 라우터와 페이지를 작성한다.

[web]-[routes] student.py

```
from flask import Blueprint, render_template

bp = Blueprint('student', __name__, url_prefix='/student')

@bp.route('/search')
def search():
    return render_template('index.html', title='학생검색', pageName='student/search.html')
```

[web]-[templates]-[student] search.html

```
<div class="my-5">
  <h1 class="text-center mb-5">{{title}}</h1>
</div>
```

5) 학생 검색 라우터와 검색 페이지를 수정한다.

[web]-[routes] student.py

```
from flask import Blueprint, render_template, request
import pandas as pd

bp = Blueprint('student', __name__, url_prefix='/student')

@bp.route('/search')
def search():
    return render_template('index.html', title='학생검색', pageName='student/search.html')

def read_data():
    df = pd.read_csv('score.txt', sep='\t')
    df['SW특기']=df['SW특기'].str.capitalize()
    df = df.fillna('')
    return df

@bp.route('/search/data')
def serch_data():
    df = read_data()
    key = request.args['key']
    word = request.args['word']
    if word == '':
        df1 = df
    else:
        filt = df[key].str.contains(word, case=False) #대소문자 관계없이 포함되는지 체크
        df1 = df[filt]
    table = df1.to_html(classes='table', table_id='tbl1')
    return table
```

[web]-[templates]-[student] search.html

```
<div class="my-5">
  <h1 class="text-center mb-5">{{title}}</h1>
  <div class="row">
    <div class="col-md-6 col-lg-4">
      <form name="frm">
        <div class="input-group">
          <select name="key" class="form-select me-2">
            <option value="학교">학교이름</option>
            <option value="이름">학생이름</option>
            <option value="SW특기">SW특기</option>
          </select>
          <input name="word" placeholder="검색어" class="form-control">
          <button class="btn btn-primary">검색</button>
        </div>
      </form>
    </div>
    <hr>
  </div>
  <div id="result"></div>
</div>
```

```

<style>
    td, th:first-child {
        text-align:center;
    }
    tbody th, thead th:first-child{
        display: none;
    }
</style>
<div class="my-5">
    <h1 class="text-center mb-5">{{title}}</h1>
    <div class="row">
        <div class="col-md-6 col-lg-4">
            <form name="frm">
                <div class="input-group">
                    <select name="key" class="form-select me-2">
                        <option value="학교">학교이름</option>
                        <option value="이름">학생이름</option>
                        <option value="SW특기">SW특기</option>
                    </select>
                    <input name="word" placeholder="검색어" class="form-control">
                    <button class="btn btn-primary">검색</button>
                </div>
            </form>
        </div>
        <hr>
    </div>
    <div id="result"></div>
</div>
<script>
    let key = $(frm.key).val();
    let word = $(frm.word).val();
    getData();

    $(frm).on("submit", function(e){
        e.preventDefault();
        key = $(frm.key).val();
        word = $(frm.word).val();
        getData();
    });

    function getData() {
        $.ajax({
            type:"get",
            url:"/student/search/data",
            data:{key, word},
            success:function(data){
                $("#result").html(data);
                $("#tbl1").addClass("table-striped table-bordered table-hover")
            }
        });
    }
</script>

```

[web]-[templates]-[student] graph.html

```
<div class="my-5">
  <h1 class="text-center mb-5">{{title}}</h1>
  <div>
    
  </div>
</div>
```

6) 학생 성적 그래프를 출력하는 라우터와 페이지를 작성한다.

[web]-[routes] student.py

```
from flask import Blueprint, render_template, request, send_file
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import numpy as np
from io import BytesIO

matplotlib.use('Agg') #Agg 백엔드는 GUI가 필요 없는 이미지 생성에 적합
matplotlib.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['font.size'] = 10
matplotlib.rcParams['axes.unicode_minus'] = False

bp = Blueprint('student', __name__, url_prefix='/student')

@bp.route('/search')
def search():
    return render_template('index.html', title='학생검색', pageName='student/search.html')

def read_data():
    df = pd.read_csv('score.txt', sep='\t')
    df['SW특기']=df['SW특기'].str.capitalize()
    df = df.fillna('')
    return df

@bp.route('/graph')
def graph():
    return render_template('index.html', title='학생그래프', pageName='student/graph.html')

@bp.route('/graph/data')
def graph_data():
    df = read_data()
    plt.figure(figsize=(10, 5))
    plt.title('학생별성적')
    w = 0.25
    index = np.arange(df.shape[0])
    plt.bar(index-w, df['국어'], width=w, label='국어')
    plt.bar(index, df['영어'], width=w, label='영어')
    plt.bar(index+w, df['수학'], width=w, label='수학')
    plt.xticks(index, df['이름'], rotation=40)
    plt.legend(ncol=3)
    #file로 저장하는 것이 아니라 binary object에 저장
    img = BytesIO()
    plt.savefig(img, format='png', dpi=200)
    plt.close()
    # object를 읽었기 때문에 처음으로 돌아감
    img.seek(0)
    return send_file(img, mimetype='image/png')
```

7) 학생 정보 출력을 위한 라우터와 출력 페이지를 작성한다.

[web]-[routes] student.py

```
from flask import Blueprint, render_template, request, send_file
import pandas as pd
```

```
bp = Blueprint('student', __name__, url_prefix='/student')
...
@bp.route('/info')
def info():
    df = read_data()
    df1 = df.loc[:, ['학교', '이름', '국어', '영어', '수학', '과학', '사회']]
    df2 = df[['이름', '학교', '키', 'SW특기']]
    return render_template(
        'index.html',
        title='학생정보',
        pageName='student/info.html',
        table1=df1.to_html(classes='table', table_id='tbl1'),
        table2=df2.to_html(classes='table', table_id='tbl2')
    )
```

```
@bp.route('/json')
def json():
    df = read_data()
    records = df.to_json(orient='records')
    #records = json.loads(records)
    columns = df.to_json(orient='columns')
    #columns = json.loads(columns)
    data = {'columns':columns, 'records':records}
    return data
```

[web]-[templates]-[student] info.html

```
<style>
    td, th:first-child {
        text-align:center;
    }
    tbody th, thead th:first-child{
        display: none;
    }
</style>
<div class="my-5">
    <h1 class="text-center mb-5">{{title}}</h1>
    <div class="row px-5">
        <div class="col-md-6">{{table1|safe}}</div>
        <div class="col">{{table2|safe}}</div>
    </div>
</div>
<script>
    $("#tbl1, #tbl2").addClass("table-striped table-bordered table-hover")
    $.ajax({
        type:"get",
        url:"/student/json",
        success:function(data){
            console.log(JSON.parse(data.records));
            console.log(JSON.parse(data.columns));
        }
    });
</script>
```